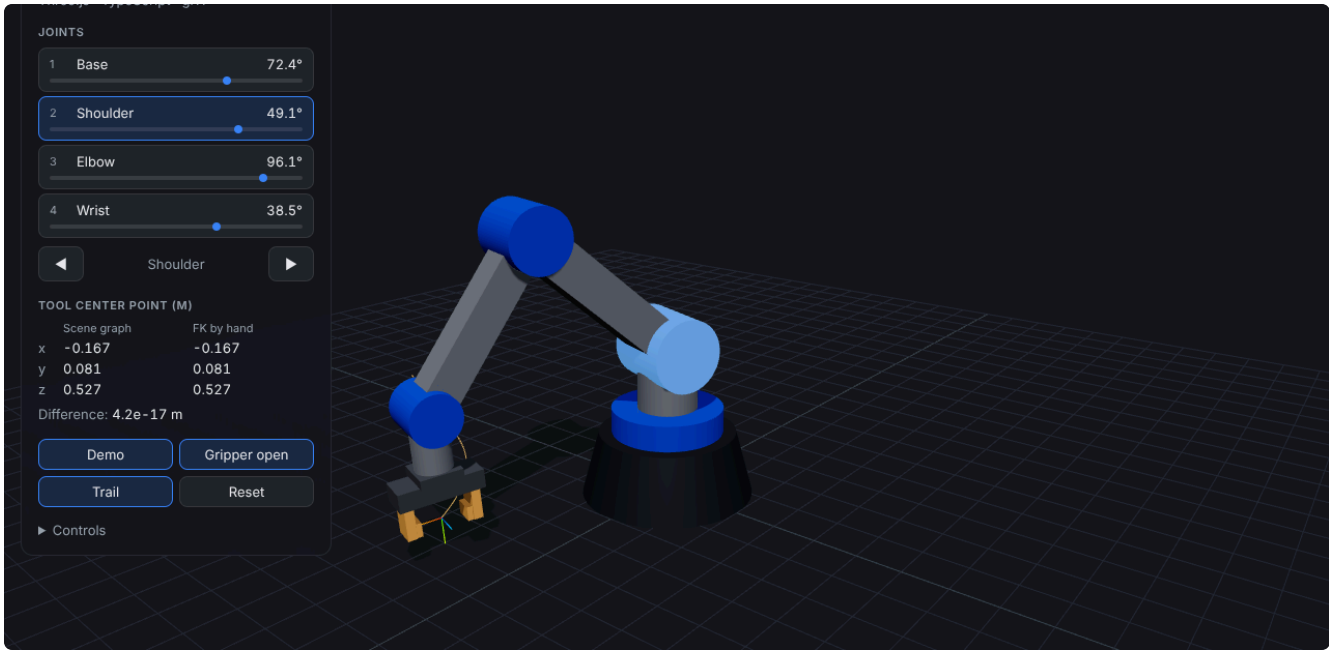
VENTION · PORTFOLIO WALKTHROUGH

Snack 1, Line by Line

A robot arm in Three.js: scene graphs and forward kinematics, a glTF model built by hand, conveyor belts, rule-driven splitters and machines, and the tests and CI that keep it working, every file explained.



1 · The Robot Arm

Projects in `Portfolio/vention/` · Code quoted directly from the repository, with file and line numbers

Contents

How to read this book 4

Chapter 1 · The Robot Arm 5

1.1 Coordinate frames and transforms 6
1.2 Forward kinematics 7
1.3 Meshes, as the GPU sees them 8
1.4 glTF in one page 9
1.5 The frame loop and frame-rate independence 11
1.6 Conveyor belts: frames, kinematics and fixed steps 11
1.7 Rules as data 13
1.8 Stations, holding and turn-taking 13
1.9 Bins, and stock that leaves the shelf once 14
Line by line 15
The arm as data 15
Forward kinematics by hand 18
The arm as a scene graph 22
Loading and driving the gripper 29
The stage 33
The tool path trail 36
The side panel 38
Wiring it together 42
Testing the kinematics 51
Building the gripper model by hand 55
Shapes in the scene: the data and the reach rule 66
Inventory parts as shapes 70
Drawing the shapes 74
The Shapes panel 79
Conveyor belts: how a belt carries things 92
Drawing the belts 104
The Conveyors tab, and running the line 110
Splitters: rules that decide where items go 127
Gates on the running line 133
Drawing gates, and the Splitters tab 138
Connection points and joining geometry 150
Mergers and machines: stations on the line 158
Running the line with machines, and the Machines tab 169
Bins: catching and feeding 177
The Bins tab, and loading many items at once 186

Interview questions 192
Exercises 196

Appendix A · Running the projects 198

Appendix B · Glossary 199

Appendix C · Keeping Snack 1 working 203

BEFORE YOU START

How to read this book

This walkthrough covers Snack 1 on its own. It's also Chapter 1 of the combined book, *Five Snacks, Line by Line*, where every chapter has the same five parts:

1. **Concepts.** The ideas the code relies on, with diagrams, before any code.
2. **Line by line.** Every source file that matters, quoted from the repository in order, with each chunk explained. The code is sliced from the real files, so the line numbers match what you'll see in your editor.
3. **Going deeper.** Boxes inside the line-by-line parts that step back to what a production system, a full engine or a large team would do differently.
4. **Interview questions.** Questions this project invites, with model answers grounded in the code.
5. **Exercises.** Extensions to try, each with a hint.

Two reading paths work well. To prepare for a conversation about the project, read the concepts and the questions. To understand or extend the code, read the line-by-line parts with the project open next to the book.

Appendix C, *Keeping Snack 1 working*, covers how the project manages its dependencies and versions, and the tests and CI pipeline that check every change.

The project

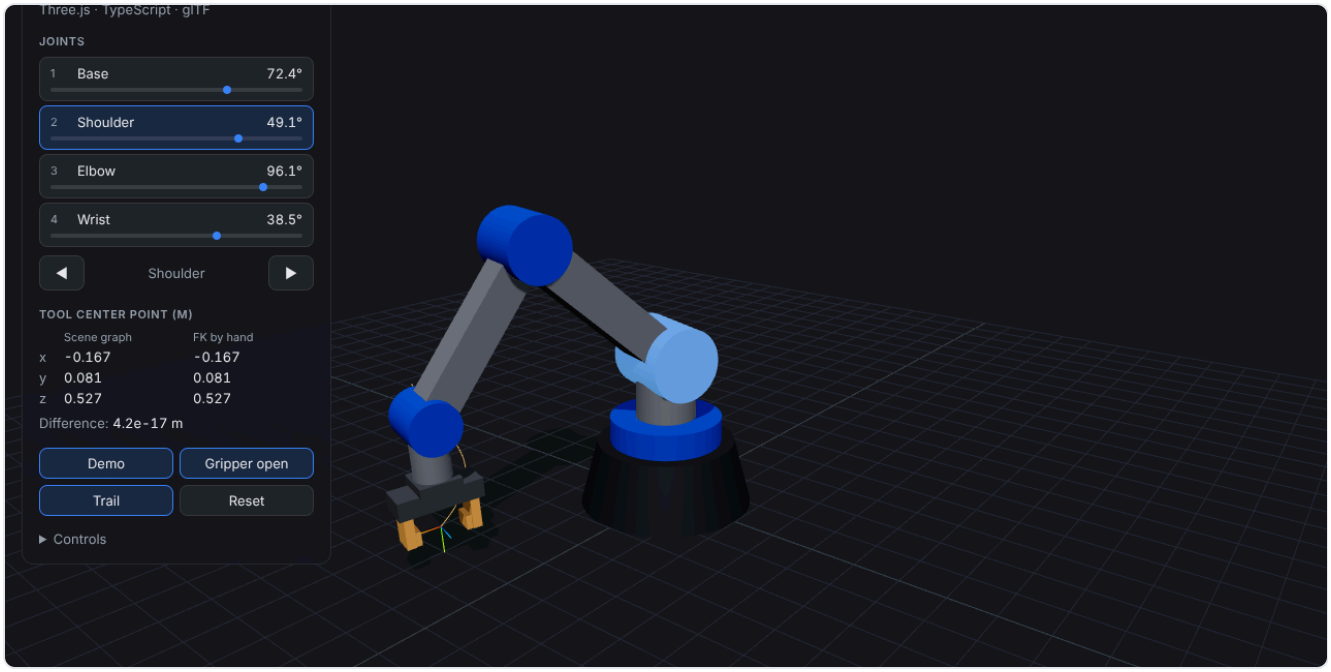
	Project	Stack	What it shows
1	**The Robot Arm**	Three.js · TypeScript · glTF	A four-joint arm as a Three.js scene graph, forward kinematics written by hand and checked against it every frame, a gripper model built byte by byte as a glTF file, shapes and inventory parts placed on its floor from the database, and conveyor belts that carry them.

CHAPTER 1 · `vention/snack1-robot-arm`

The Robot Arm

A four-joint arm as a Three.js scene graph, forward kinematics written by hand and checked against it every frame, a gripper model built byte by byte as a glTF file, shapes and inventory parts placed on its floor from the database, and conveyor belts that carry them.

Three.js · TypeScript · glTF



Demo mode. The panel's TCP readout shows the scene graph's answer and the hand-written FK side by side; they differ by $4 \cdot 10^{-17}$ m.

The one idea to take away

The same question, “where is the tool?”, answered two independent ways, every frame:

`joint angles` → `Three.js scene graph` → `tcp.getWorldPosition()`

`joint angles` → `forwardKinematics() by hand` → `{ x, y, z }`

If the hierarchy were built wrong, the two would disagree immediately. The gripper is a file the script writes from scratch:

`boxes` → `vertices + normals + indices` →

`buffer → bufferView → accessor → mesh → node` → `.glb`

CONCEPTS · 1.1

Coordinate frames and transforms

Every object in a 3D scene has a **local transform**: a position, rotation and scale relative to its parent. Three.js packs the three into one 4×4 matrix (`object.matrix`) and computes the **world** matrix by multiplying down the tree: `matrixWorld = parent.matrixWorld × matrix`. Nothing more is needed to make a robot arm move like a robot arm: put each joint inside the previous one, and rotating a joint carries everything after it.

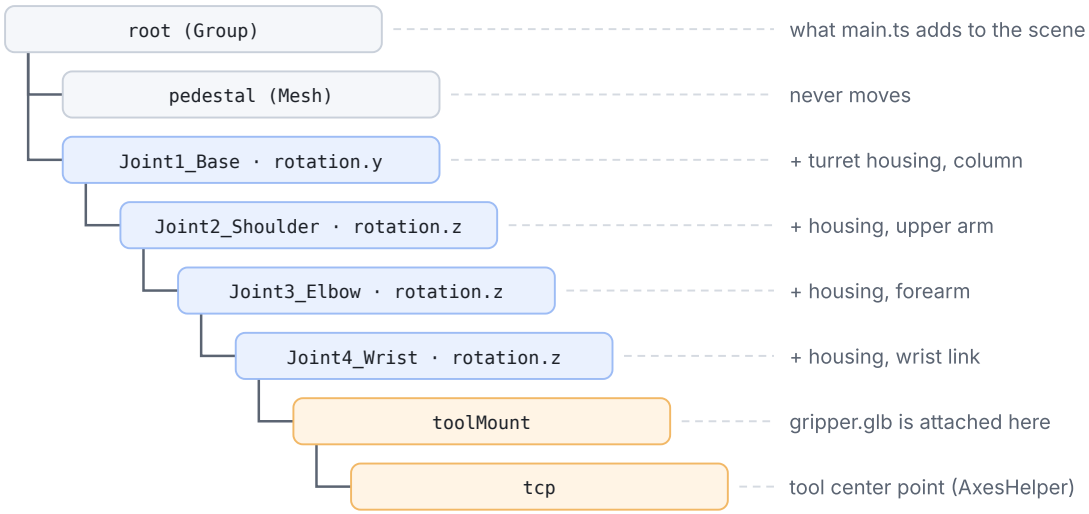


Figure 1.1. The arm's hierarchy. Blue nodes are empty joints that only rotate; the meshes hang off them. Rotating a joint moves everything below it in the tree.

The conventions are worth saying out loud, because most 3D bugs are convention bugs:

- **Right-handed, +Y up, meters.** Three.js, glTF and this project agree on all three, so no conversion happens anywhere.
- **Right-hand rule for rotation.** Point your right thumb along the axis; your fingers curl in the positive direction. A positive rotation about Z turns +Y toward −X, which is why a positive shoulder angle leans the arm toward −X.
- **Radians in the API, degrees for people.** The project stores degrees and converts once, in `RobotArm.setAngle`.

A transform matrix applies scale, then rotation, then translation (T·R·S). Read the arm's chain as alternating pairs: “translate up the link, then rotate about the joint”. That's exactly the list of matrices the test file multiplies together.

> Each joint here rotates about one axis only, so Euler-angle order never matters and **gimbal lock** can't happen. Free-floating orientations, like the tumbling boxes in Snack 3, use **quaternions** instead: four numbers that compose and interpolate smoothly without singular poses.

CONCEPTS · 1.2

Forward kinematics

Forward kinematics (FK) maps joint space (four angles) to task space (where the tool is). It always has exactly one answer and is cheap to compute, which makes it the foundation for everything else: drawing, checking clearances, and testing inverse kinematics in Snack 2.

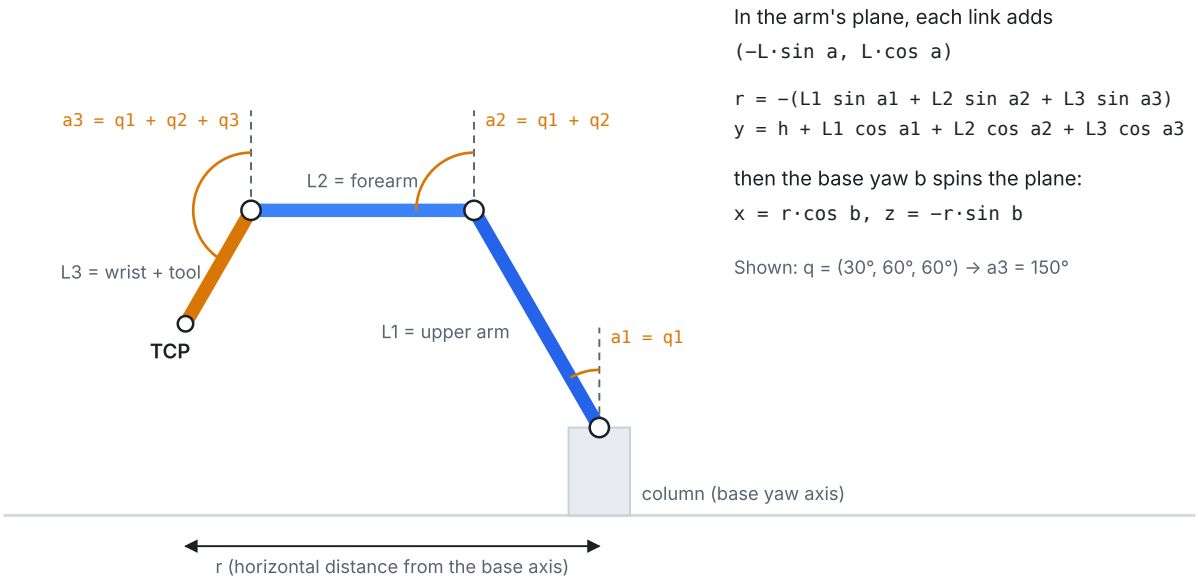


Figure 1.2. Side view of the arm's plane. Because the pitch joints are parallel, each link's angle from vertical is the sum of the joint angles before it, and the tool position is a sum of three link vectors.

The general method is a chain of 4×4 matrices, one per joint, usually described by **Denavit–Hartenberg** (DH) parameters: four numbers per joint (link length, twist, offset and angle). This arm doesn't need that machinery, because its pitch axes are parallel: everything after the base is a 2D problem in one plane. The test file still builds the general matrix chain, as an independent reference.

The **workspace** is the set of points the tool can reach. With every joint free it would be a sphere of radius 1.085 m around the shoulder; the joint limits carve pieces out of it. The demo-motion test checks one practical corner of it: the fingertips must stay above the floor.

CONCEPTS · 1.3

Meshes, as the GPU sees them

A mesh is a list of **vertices**, each with attributes (position, normal, maybe texture coordinates), plus an **index** list saying which three vertices form each triangle. The vertex shader moves every vertex by the model, view and projection matrices; the GPU fills in the triangles; the fragment shader colors each pixel using the normal interpolated across the triangle.

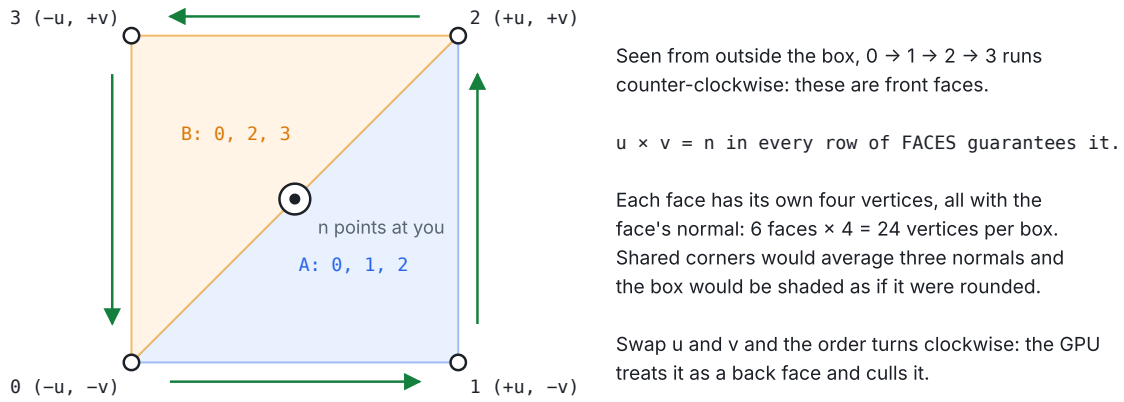


Figure 1.3. One face of a box, split into two triangles. The corner order decides which side is the front.

- **Winding** decides the front of a triangle: counter-clockwise as seen by the camera. Back faces are culled by default, which halves the work for closed shapes. Wrong winding looks like a model turned inside out.
- **Normals** drive lighting. Shared vertices give smooth shading (one averaged normal); duplicated vertices give hard edges. A box needs 24 vertices, not 8, to look like a box.
- **Indexed drawing** saves memory: 24 vertices and 36 indices instead of 36 full vertices, and the GPU can reuse vertices it has already transformed.

CONCEPTS · 1.4

glTF in one page

glTF 2.0 is the Khronos Group's open format for delivering 3D scenes, often called “the JPEG of 3D”. It holds a node hierarchy, meshes, PBR materials, textures, cameras, animations and skins. A `.gltf` is JSON plus separate binary and image files; a `.glb` packs everything into one binary file.

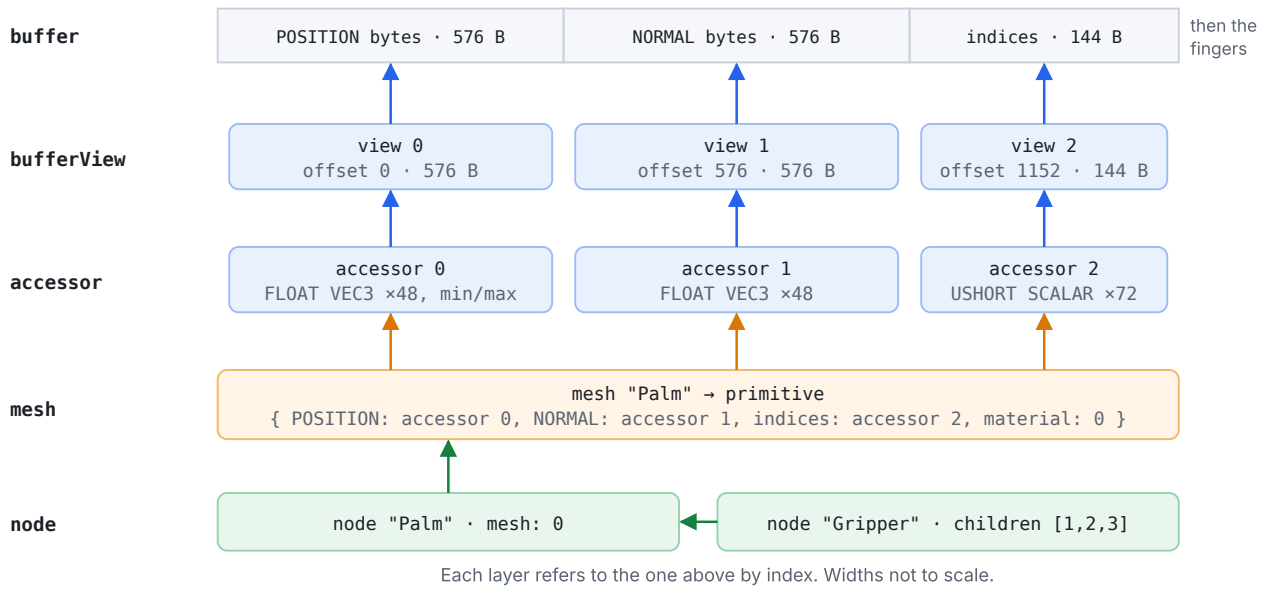


Figure 1.4. How the palm is stored. Raw bytes, then byte ranges, then typed views of those ranges, then a drawable mesh, then a node that places it.

Why three layers between bytes and meshes? A **bufferView** is a byte range that can be uploaded to the GPU as-is, and can **interleave** several attributes using a stride. An **accessor** says how to read part of a view (type, count, offset). So many meshes can share one blob, and a loader can hand byte ranges straight to WebGL with no parsing of individual numbers. That's what makes glTF fast to load.

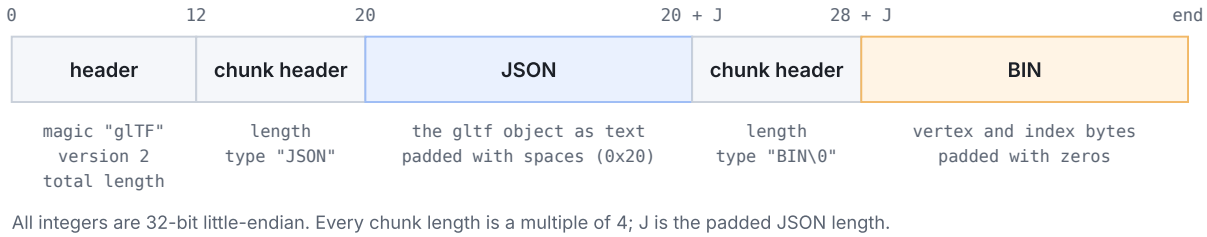


Figure 1.5. The .glb container: one file holding the JSON and the binary buffer, so the model loads with a single request.

Materials use the **metallic-roughness** PBR model: a base color, how metallic the surface is, and how rough. Three.js maps these directly to `MeshStandardMaterial`, so a model looks the same in any glTF viewer.

CONCEPTS · 1.5

The frame loop and frame-rate independence

`renderer.setAnimationLoop` (a wrapper around `requestAnimationFrame`) calls you once per display refresh: 60 times a second on most monitors, 120 or 144 on others. Anything that moves must therefore scale by the frame time `dt`, or it will run twice as fast on a 120 Hz screen.

Constant speeds are easy: `angle += speed × dt`. Smoothing is subtler. Moving a fixed fraction k of the remaining gap each frame leaves $(1 - k)^n$ of the gap after n frames, so the result depends on how many frames there were. Choosing **k = 1 − e^(−λ·dt)** fixes it: after any sequence of frames adding up to time T, the remaining gap is e^(−λT), however the time was sliced. With λ = 3 (`DEMO_SMOOTHING`), 95% of the gap closes in one second.

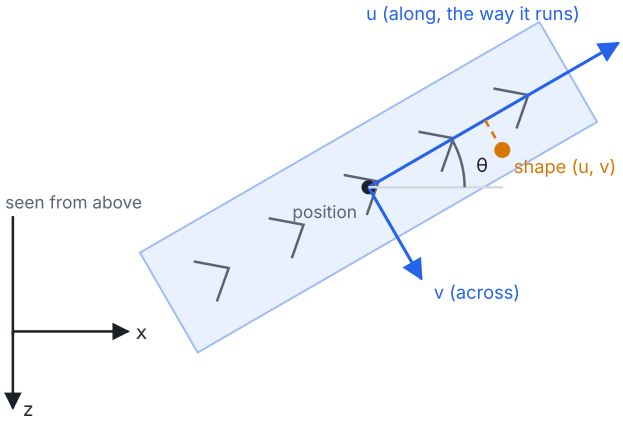
Two more frame-loop rules from this project: clamp `dt` (here to 50 ms), because a hidden tab stops calling you and the first frame back would otherwise see a huge step; and remember that Three.js updates world matrices lazily, during `render()`, so code that needs a world position between changes must ask for it (`getWorldPosition` does).

CONCEPTS · 1.6

Conveyor belts: frames, kinematics and fixed steps

A conveyor looks like a physics problem, but most of it is geometry. Three ideas carry the whole feature.

Local frames. Every belt can be turned to any angle, which makes “is this point on the belt?” awkward in world coordinates. Moved into the belt's own frame (subtract its position, rotate by minus its angle), every belt becomes the same axis-aligned rectangle. Robotics does this constantly: a part's position in the gripper's frame, a target in the camera's frame. The one rule is to use the same angle convention as the renderer, or what's drawn and what moves will disagree.



toBelt: world → belt

```
dx = x − position.x, dz = z − position.z
u = dx·cos θ − dz·sin θ
v = dx·sin θ + dz·cos θ
```

on the belt if

```
|u| ≤ length/2 and |v| ≤ width/2
```

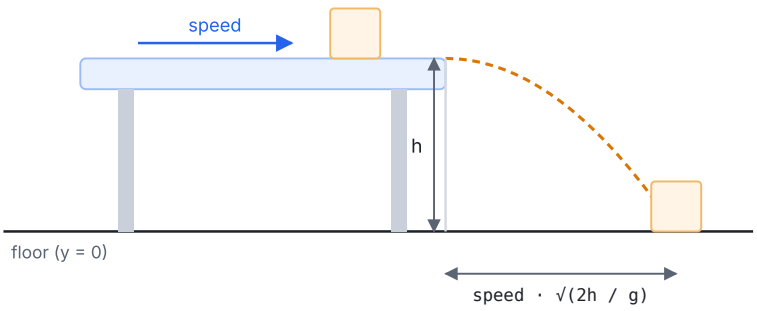
surface velocity

```
(vx, vz) = speed · (cos θ, −sin θ)
```

Same convention as Three.js's rotation.y, so the drawing and the motion always agree.

Figure 1.6. A belt's own frame. Turning the world point into (u, v) makes every belt an axis-aligned rectangle, whatever its angle.

Kinematic, not dynamic. A dynamic simulation computes forces (friction between belt and part, contact between parts) and gets motion from them. A kinematic one sets the motion directly: on a belt, move at the belt's speed. It can't show a box tipping over or two boxes jamming, but it's exact, cheap and testable, and it answers the layout question (where does each part end up?) that a line designer asks first. Off the end, gravity is the one force simulated, because where an item lands decides whether the next belt catches it.



Each step of dt, in the air:

```
vy ← vy − g·dt
y ← y + vy·dt
x, z ← x, z + (vx, vz)·dt
```

On landing:

y = the support under it
velocity = 0
next step, a belt there picks it up again

h = 0.3 m at 1 m/s:
0.25 s in the air, lands 0.25 m out

Figure 1.7. Off the end of a belt: the item keeps its forward speed and falls with gravity, a projectile, until something holds it up.

Fixed steps. The display runs at whatever rate the machine manages; the simulation shouldn't care. So each frame's time is cut into steps of at most 1/120 s. Without that, a slow frame would make a falling item drop further in one step than in several, landing in a different place, and a fast item could jump over a short belt entirely. Games call this decoupling the simulation rate from the frame rate.

CONCEPTS · 1.7

Rules as data

A splitter's behaviour could have been code: an `if` per case. Instead it's **data**, a list of field/operator/value conditions that a small interpreter (`route`) evaluates. That one choice is what makes everything else possible:

- **Stored and shared.** Rules live in the database like any other record, so every browser and every API replica sees the same ones.
- **Editable without a deploy.** A person changes sorting in the panel, a script changes it through the API, an operator changes it with SQL. None of them touches the app's code.
- **Explainable.** Because the interpreter is tiny, it can also report *which* rule fired, and preview decisions before anything is saved.
- **Safe by construction.** A condition can only name a known field and an operator that suits it. There's no way to put arbitrary code in a rule, which is why rules from the API can be trusted to run in every browser.

This is the pattern behind firewall rules, email filters, feature flags and the routing tables of real sortation systems. The usual trade-off applies: a rule language is limited by design, and when people start wanting “or”, grouping and arithmetic, it's time to decide how far to grow it.

Rules that read **live** data add one more property: the decision depends on the world when the item arrives, not when the rule was written. Change a part's category in the inventory and its items take a different exit, with no rule edited. That's the behaviour asked for here, and also the thing to be careful with: the same rule can sort the same part differently on two runs, so the panel shows the last decision and why.

CONCEPTS · 1.8

Stations, holding and turn-taking

Mergers and machines look different but share one shape: a **station** puts lines on belts, and every step it's asked whether to hold the item at its line. Everything a station does is a decision over the current step's state: where every item is, and a little memory of its own (who's waiting, who went last, what a machine holds). There are no timers except a machine's cycle, so the line behaves the same at 120 frames a second or 3.

A merger is a tiny **traffic controller**. Its safety rule is a clear gap at the merge point; its fairness rule is the priority. “Main belt first” is a yield sign; “feeders first” is a yield sign facing the other way; “take turns” is a zipper merge, and needs one bit of memory: who went last. The same few lines run a Y merger's two feeders.

A machine is an **assembly cell**: it waits for a matched set of inputs, works for a cycle time, and emits one output. Real cells use buffers and side feeders so a second base can wait without blocking the attach item behind it. This one lets extra items pass instead, because on a single belt waiting could **deadlock**: each item waiting for one that can't arrive until it moves.

Joining uses **connection points**, named spots defined once per kind of object as fractions of its size. A join stores the attached object's offset in the base's frame, which is the scene-graph idea from the start of this chapter (a child's transform is relative to its parent) applied to data.

CONCEPTS · 1.9

Bins, and stock that leaves the shelf once

A bin turns the scene from a layout into a small **material flow**: items come out of stock into a bin, ride the line, get joined, and land in another bin. The inventory has to stay true through all of it, which comes down to one rule: **a part leaves stock exactly once**. Each shape carries a flag saying whether its part has left, and every operation that would take stock (loading, catching, joining) checks it first.

Choosing what to load by **rule** reuses the splitter's rule engine on things that don't exist yet. Each kind of object becomes a stand-in shape (how it would be drawn, plus its part), and the same `matches` that sorts items on a belt decides what goes into a bin. One small language, three uses: sorting, accepting, loading.

Push and pull. A steady bin pushes items onto a belt whenever there's room. An on-demand bin pulls: it sends an item only when a machine downstream has an empty slot for it and none is already on the way. That's the idea behind kanban in factories: produce what the next step asks for, not whatever you can.

CHAPTER 1

Line by line

25 parts, in the order data flows through the code.

PART 1.1 · `src/arm/jointSpecs.ts`

The arm as data

Every number that describes the arm lives in one file: link lengths, joint limits, the home pose and the demo motion. The arm, the panel and the tests all read from here.

Joint specs

src/arm/jointSpecs.ts · lines 1–12

```ts
import type { ArmDimensions } from "./kinematics";

export type Axis = "x" | "y" | "z";

/** One rotary joint. Angles are in degrees. */
export interface JointSpec {
  name: string;
  axis: Axis;
  min: number;
  max: number;
  home: number;
}
```

`Axis` is a **string literal union**: the only values TypeScript accepts are `"x"`, `"y"` and `"z"`. That pays off in RobotArm.ts, where `rotation[spec.axis]` is type-checked: a typo like `"w"` is a compile error, not a silent no-op.

A `JointSpec` is pure data, in degrees, because degrees are what people read and type. Conversion to radians happens at exactly one place, where the angle is handed to Three.js.

Link lengths

src/arm/jointSpecs.ts · lines 14–22

```
/** Link lengths in meters (glTF and Three.js both treat 1 unit as 1 m). */
export const ARM: ArmDimensions = {
  baseHeight: 0.15,
  shoulderHeight: 0.2,
  upperArm: 0.45,
  forearm: 0.4,
  wrist: 0.12,
  tool: 0.115,
};
```

Meters, because glTF and Three.js both treat one unit as one meter, and keeping real units means numbers like “45 cm upper arm” can be checked against a spec sheet. The arm stretches to 1.085 m from the shoulder (0.45 + 0.4 + 0.12 + 0.115), roughly the size of a small collaborative robot.

`tool` is the distance from the wrist flange to the **tool center point** (TCP), the point between the fingertips that everything else in the project aims.

Joints and the home pose

src/arm/jointSpecs.ts · lines 24–33

```
/**
 * The arm is built from this list, so adding or re-limiting a joint is a data
 * change. Home pose: gripper pointing straight down (30 + 90 + 60 = 180 degrees).
 */
export const JOINTS: readonly JointSpec[] = [
  { name: "Base", axis: "y", min: -180, max: 180, home: 45 },
  { name: "Shoulder", axis: "z", min: -100, max: 100, home: 30 },
  { name: "Elbow", axis: "z", min: -140, max: 140, home: 90 },
  { name: "Wrist", axis: "z", min: -120, max: 120, home: 60 },
];
```

One yaw joint about Y (the base turns the whole arm), then three pitch joints about Z. The array is the **single source of truth**: RobotArm.ts builds one joint per entry, the panel builds one row per entry, and the keyboard handler accepts keys 1 to `JOINTS.length`.

The home angles are chosen so the three pitch angles add up to 180° (30 + 90 + 60). Pitch angles accumulate along a chain of parallel joints, so a total of 180° means the last link points straight down: the gripper is ready to pick something off the floor. A test checks exactly this.

`readonly JointSpec[]` stops any code from pushing to or reassigning entries of the array. It's a compile-time promise only, which is all that's needed.

The demo motion

src/arm/jointSpecs.ts · lines 35–56

```
/**
 * Demo motion: each joint sways around its home angle at its own frequency, so
 * the combined path never quite repeats. Amplitudes are chosen so the gripper
 * stays above the floor (checked in tests/kinematics.test.ts).
 */
const DEMO_MOTION = [
  { amplitude: 70, frequency: 0.07 },
  { amplitude: 20, frequency: 0.11 },
  { amplitude: 18, frequency: 0.13 },
  { amplitude: 30, frequency: 0.17 },
] as const;

/** Target angle in degrees for joint `index` at `time` seconds into the demo. */
export function demoTarget(index: number, time: number): number {
  const { amplitude, frequency } = DEMO_MOTION[index];
  return JOINTS[index].home + amplitude * Math.sin(2 * Math.PI * frequency * time + index);
}

/** Whether the gripper should be open at `time` seconds into the demo. */
export function demoGripperOpen(time: number): boolean {
  return Math.sin(2 * Math.PI * 0.15 * time) > 0;
}
```

Each joint sways around its home angle as a sine wave: `home + amplitude · sin(2π · frequency · t + index)`. The frequencies (0.07, 0.11, 0.13, 0.17 Hz) share no simple ratio, so the combined motion takes a very long time to repeat, and the `+ index` phase offset stops all joints from starting in step.

`as const` freezes the table's types to its literal values and makes it read-only. Amplitudes were tuned down after the floor-clearance test failed with the first values: the gripper dipped through the floor at certain moments. That test, and not eyeballing, is what decided them.

`demoGripperOpen` is a slow square wave: open for half of every 6.7 s cycle (0.15 Hz), closed for the other half.

PART 1.2 · `src/arm/kinematics.ts`

Forward kinematics by hand

Given the four joint angles, where is the tool? Three.js already knows, through its scene graph. This file works it out independently, with no Three.js at all, so the two answers can check each other.

Why the problem is small

src/arm/kinematics.ts · lines 1–11

```
/**
 * Forward kinematics for the arm, written by hand with no Three.js dependency.
 *
 * The arm is a base that yaws about Y, followed by three joints that all pitch
 * about their local Z axis. Because the pitch joints are parallel, everything
 * after the base yaw lies in one vertical plane, so FK reduces to summing link
 * vectors in that plane, then rotating the plane about Y by the base angle.
 *
 * main.ts compares this against Three.js's own scene-graph result every frame,
 * which is a good sanity check that the hierarchy is built the way we think.
 */
```

The comment states the insight that makes the code short. The three pitch joints are **parallel** (all rotate about their own Z axis), so every link after the base lies in one vertical plane. Forward kinematics (FK) becomes a 2D problem in that plane, followed by one rotation of the whole plane about the vertical axis by the base angle.

A general 6-axis arm needs a full chain of 4×4 matrices. That's what the test file builds as an independent reference.

Data shapes

src/arm/kinematics.ts · lines 13–32

```
export interface ArmDimensions {
  /** Floor to the base (yaw) joint. */
  baseHeight: number;
  /** Base joint to the shoulder joint. */
  shoulderHeight: number;
  /** Shoulder joint to elbow joint. */
  upperArm: number;
  /** Elbow joint to wrist joint. */
  forearm: number;
  /** Wrist joint to the tool flange. */
  wrist: number;
  /** Tool flange to the tool center point (between the gripper fingertips). */
  tool: number;
}

export interface Vec3 {
  x: number;
  y: number;
  z: number;
}
```

`ArmDimensions` documents each length with a doc comment, so editors show “Floor to the base (yaw) joint” on hover. `Vec3` is a plain structural type: Three.js's `Vector3` satisfies it, but so does `{ x: 1, y: 2, z: 3 }`, which keeps this module testable in plain Node with no WebGL.

The forward kinematics function

src/arm/kinematics.ts · lines 34–45

```
/**
 * World position of the tool center point.
 * @param q joint angles in radians: [base yaw, shoulder, elbow, wrist]
 */
export function forwardKinematics(d: ArmDimensions, q: readonly number[]): Vec3 {
  const [base = 0, shoulder = 0, elbow = 0, wrist = 0] = q;

  // Each pitch joint adds to the angle of every link after it.
  const a1 = shoulder;
  const a2 = shoulder + elbow;
  const a3 = shoulder + elbow + wrist;
  const distal = d.wrist + d.tool;
```

Destructuring with defaults (`base = 0`) means a short array still works. Then the key step: the **absolute** angle of each link from vertical is the sum of every pitch joint before it. Tilt the shoulder 30° and the elbow 90°, and the forearm is 120° from vertical. `distal` treats the wrist link and tool as one straight segment, since nothing bends between them.

Summing link vectors

src/arm/kinematics.ts · lines 47–54

```
// Rotating (0, L, 0) about Z by a gives (-L sin a, L cos a, 0).
const r = -(d.upperArm * Math.sin(a1) + d.forearm * Math.sin(a2) + distal * Math.sin(a3));
const y =
  d.baseHeight +
  d.shoulderHeight +
  d.upperArm * Math.cos(a1) +
  d.forearm * Math.cos(a2) +
  distal * Math.cos(a3);
```

In the arm's plane, a link of length L at angle a from vertical contributes (−L sin a, L cos a). The minus sign is the arm's convention: a positive rotation about Z tips +Y toward −X, so the arm leans toward −X. `r` is the signed horizontal distance from the base axis to the TCP and `y` is its height.

Height starts from the shoulder joint (`baseHeight + shoulderHeight`), since the column below it never tilts.

Turning the plane

src/arm/kinematics.ts · lines 56–58

```
  // Rotating (r, y, 0) about Y by `base` gives (r cos b, y, -r sin b).
  return { x: r * Math.cos(base), y, z: -r * Math.sin(base) };
}
```

Rotating a point (r, y, 0) about the vertical axis by angle b gives (r cos b, y, −r sin b). Height doesn't change, since yaw only spins the plane. The sign on z is again the right-hand rule: positive rotation about +Y turns +X toward −Z.

`main.ts` calls this every frame and the panel shows its answer next to Three.js's, with the difference: around 10^{-17} m, which is floating-point noise. If the hierarchy were built wrong, that number would jump to centimeters immediately.

Small helpers

src/arm/kinematics.ts · lines 60–69

```
/** Distance from the shoulder joint to the TCP with the arm fully stretched. */
export function maxReach(d: ArmDimensions): number {
  return d.upperArm + d.forearm + d.wrist + d.tool;
}

export function clamp(value: number, min: number, max: number): number {
  return Math.min(max, Math.max(min, value));
}

export const DEG = Math.PI / 180;
```

`maxReach` is the stretched-out length from the shoulder, used by the tests as an upper bound. `clamp` keeps a value within limits; `Math.min(max, Math.max(min, value))` reads inside out: first raise to at least `min`, then lower to at most `max`. `DEG` converts degrees to radians when multiplied (and radians to degrees when divided).

GOING DEEPER

The Jacobian, and singularities

The next step after FK is velocity. The **Jacobian** J is the matrix of partial derivatives ∂(tool position)/∂(joint angle): 3 rows by 4 columns here. Multiply it by joint speeds and you get the tool's speed. Real controllers invert it to execute straight-line moves (joint speeds = J^{-1} × desired tool speed), and numerical IK solvers are built on it.

Where J loses rank, the arm is at a **singularity**: some tool direction can't be produced at any joint speed. For this arm the obvious one is full stretch, where the tool can't move further outward. Near a singularity, a straight-line move demands enormous joint speeds, which is why industrial controllers slow down or refuse near them.

PART 1.3 · `src/arm/RobotArm.ts`

The arm as a scene graph

The arm is a chain of nested Three.js objects. Rotating one joint moves everything below it, so there's no kinematics code here at all: the hierarchy is the kinematics.

The hierarchy

src/arm/RobotArm.ts · lines 8–23

```
/**
 * The arm as a Three.js scene-graph hierarchy:
 *
 *   root
 *   ├── pedestal
 *   └── Joint1 (yaw, Y)        ← turret, column
 *       └── Joint2 (pitch, Z)  ← shoulder housing, upper arm
 *           └── Joint3 (pitch, Z)  ← elbow housing, forearm
 *               └── Joint4 (pitch, Z)  ← wrist housing, wrist link
 *                   └── toolMount  ← gripper (glTF) attaches here
 *                       └── tcp    ← tool center point
 *
 * Each joint is an empty Object3D placed at the end of its parent link. Rotating
 * a joint moves everything below it, exactly like parenting Transforms in Unity.
 * Link meshes are offset by half their length so they extend from the joint.
 */
```

Each joint is an empty `Object3D`, a transform with no geometry, placed at the end of its parent link. The visible meshes hang off the joints. This separation is the main design choice: rotation lives on the joint, shape lives on the mesh, and a mesh can be offset or swapped without touching the motion.

Unity developers will recognise it as parenting empty GameObjects. Three.js computes each object's world transform as its parent's world transform times its own local one, all the way up the tree.

Fields

src/arm/RobotArm.ts · lines 24–34

```
export class RobotArm {
  readonly root = new THREE.Group();
  readonly joints: THREE.Object3D[] = [];
  /** Clickable joint housings; userData.jointIndex says which joint each belongs to. */
  readonly housings: THREE.Mesh<THREE.BufferGeometry, THREE.MeshStandardMaterial>[] = [];
  readonly toolMount = new THREE.Object3D();
  readonly tcp = new THREE.Object3D();

  private readonly dims: ArmDimensions;
  private readonly specs: readonly JointSpec[];
  private readonly angles: number[]; // degrees
```

`root` is what main.ts adds to the scene. `joints` is indexed the same way as `JOINTS`, so `joints[2]` is the elbow. `housings` are the blue cylinders the user can click. `toolMount` is where the gripper attaches, and `tcp` marks the tool center point.

`angles` stores degrees as the source of truth. Reading an angle back from `rotation.z` would work, but would give radians and float round-trip noise; keeping the commanded value avoids both.

Constructor

src/arm/RobotArm.ts · lines 36–42

```
constructor(dims: ArmDimensions, specs: readonly JointSpec[]) {
  this.dims = dims;
  this.specs = specs;
  this.angles = specs.map((s) => s.home);
  this.build();
  specs.forEach((s, i) => this.setAngle(i, s.home));
}
```

Build the hierarchy once, then apply the home pose through `setAngle`, so the same clamping and conversion run as for any later change. There is only one code path that rotates a joint.

Setting an angle

src/arm/RobotArm.ts · lines 44–51

```
/** Sets joint `index` in degrees, clamped to its limits. Returns the applied angle. */
setAngle(index: number, degrees: number): number {
  const spec = this.specs[index];
  const value = clamp(degrees, spec.min, spec.max);
  this.angles[index] = value;
  this.joints[index].rotation[spec.axis] = value * DEG;
  return value;
}
```

Clamp to the joint's limits, remember the value, then set the rotation on the joint's axis in radians. `rotation[spec.axis]` is computed property access: for the base it's `rotation.y`, for the others `rotation.z`. Returning the applied angle lets a caller see that a request was clamped.

Setting `rotation` doesn't recompute any matrices. Three.js does that lazily, when the scene is rendered or when someone asks for a world position.

Reading angles and highlighting

src/arm/RobotArm.ts · lines 53–70

```
getAngle(index: number): number {
  return this.angles[index];
}

get anglesDeg(): readonly number[] {
  return this.angles;
}

anglesRad(): number[] {
  return this.angles.map((a) => a * DEG);
}

/** Makes the selected joint's housing glow; pass null to clear. */
highlight(index: number | null): void {
  this.housings.forEach((h, i) => {
    h.material.emissive.setHex(i === index ? SELECTED_GLOW : 0x000000);
  });
}
```

`anglesDeg` returns the internal array typed as `readonly`, so the panel can read it every frame without a copy. `anglesRad` makes a new array, which is fine because it's only used for the FK comparison.

Highlighting sets the material's **emissive** color, light the surface gives off regardless of lighting, so the selected housing glows. That only works because each housing has its own material (see `addHousing`).

Building the chain

src/arm/RobotArm.ts · lines 72–110

```
private build(): void {
  const d = this.dims;
  const linkMat = new THREE.MeshStandardMaterial({ color: 0xc9ced6, metalness: 0.7, roughness:
0.35 });
  const baseMat = new THREE.MeshStandardMaterial({ color: 0x2a2e35, metalness: 0.4, roughness:
0.6 });

  const pedestal = solid(new THREE.CylinderGeometry(0.16, 0.2, d.baseHeight, 48), baseMat);
  pedestal.position.y = d.baseHeight / 2;
```

```
  this.root.add(pedestal);

  // Joint 1: base yaw
  const j1 = this.addJoint(this.root, d.baseHeight);
  const turret = this.addHousing(j1, new THREE.CylinderGeometry(0.13, 0.13, 0.06, 48));
  turret.position.y = 0.03;
  j1.add(along(solid(new THREE.CylinderGeometry(0.07, 0.07, d.shoulderHeight, 32), linkMat),
d.shoulderHeight));

  // Joint 2: shoulder pitch
  const j2 = this.addJoint(j1, d.shoulderHeight);
  this.addHousing(j2, zCylinder(0.085, 0.17));
  j2.add(along(solid(new THREE.BoxGeometry(0.09, d.upperArm, 0.09), linkMat), d.upperArm));

  // Joint 3: elbow pitch
  const j3 = this.addJoint(j2, d.upperArm);
  this.addHousing(j3, zCylinder(0.07, 0.15));
  j3.add(along(solid(new THREE.BoxGeometry(0.075, d.forearm, 0.075), linkMat), d.forearm));

  // Joint 4: wrist pitch
  const j4 = this.addJoint(j3, d.forearm);
  this.addHousing(j4, zCylinder(0.055, 0.12));
  j4.add(along(solid(new THREE.CylinderGeometry(0.04, 0.04, d.wrist, 32), linkMat), d.wrist));

  this.toolMount.name = "ToolMount";
  this.toolMount.position.y = d.wrist;
  j4.add(this.toolMount);

  this.tcp.name = "TCP";
  this.tcp.position.y = d.tool;
  this.tcp.add(new THREE.AxesHelper(0.06));
  this.toolMount.add(this.tcp);
}
```

Read it as the arm, bottom to top. The pedestal is a tapered cylinder centered at half its height so it sits on the floor. Each joint is created at the end of the previous link (`addJoint(parent, offsetY)`), gets a housing, and then gets its link mesh, which `along()` shifts up by half its length so it runs from the joint to the next one.

Every link points along its joint's local +Y. That's why FK can use (0, L, 0) for every link: all the geometry is “straight up” in local space, and only the rotations differ.

`toolMount` sits at the end of the wrist link and `tcp` a further `d.tool` beyond it. An `AxesHelper` on the TCP draws small red, green and blue axis lines, which is how you can see the tool's orientation in the screenshot.

Adding joints and housings

src/arm/RobotArm.ts · lines 112–135

```
private addJoint(parent: THREE.Object3D, offsetY: number): THREE.Object3D {
  const index = this.joints.length;
  const joint = new THREE.Object3D();
  joint.name = `Joint${index + 1}_${this.specs[index].name}`;
  joint.position.y = offsetY;
  parent.add(joint);
  this.joints.push(joint);
  return joint;
}

private addHousing(joint: THREE.Object3D, geometry: THREE.BufferGeometry) {
  // Each housing gets its own material so it can be highlighted on its own.
  const material = new THREE.MeshStandardMaterial({
    color: HOUSING_COLOR,
    metalness: 0.3,
    roughness: 0.45,
    emissiveIntensity: 0.6,
  });
  const housing = solid(geometry, material);
  housing.userData.jointIndex = this.housings.length;
  joint.add(housing);
  this.housings.push(housing);
  return housing;
}
```

`addJoint` names each joint (`Joint2_Shoulder`), which makes the hierarchy readable in Three.js's devtools, and pushes it so `joints[i]` lines up with `specs[i]`.

`addHousing` stores the joint index in `userData`, Three.js's free-form slot for app data on any object. When a click ray hits a housing, main.ts reads `userData.jointIndex` to know which joint was clicked. Each housing gets its **own** material, since a shared material would make all of them glow at once.

Mesh helpers

src/arm/RobotArm.ts · lines 138–154

```
function solid<M extends THREE.Material>(geometry: THREE.BufferGeometry, material: M) {
  const mesh = new THREE.Mesh(geometry, material);
  mesh.castShadow = true;
  mesh.receiveShadow = true;
  return mesh;
}

/** Shifts a link mesh so it runs from its joint (y = 0) to y = length. */
function along<T extends THREE.Object3D>(mesh: T, length: number): T {
  mesh.position.y = length / 2;
  return mesh;
}

/** Cylinder whose axis is Z, matching a pitch joint's rotation axis. */
function zCylinder(radius: number, length: number): THREE.BufferGeometry {
  return new THREE.CylinderGeometry(radius, radius, length, 40).rotateX(Math.PI / 2);
}
```

`solid` turns on shadows for every part. `along` shifts a mesh's center up by half its length. `zCylinder` exists because Three.js cylinders stand along Y, but a pitch joint's housing should lie along its rotation axis, Z: rotating the geometry 90° about X lays it down. `rotateX` on a geometry changes the vertices once, so the mesh itself keeps an identity rotation.

The generic `<M extends THREE.Material>` keeps the precise material type, so `housings` can be typed as meshes with a `MeshStandardMaterial`, and `material.emissive` type-checks.

GOING DEEPER

When world matrices are actually computed

Setting `rotation.z` only stores a number. During `renderer.render()`, Three.js calls `scene.updateMatrixWorld()`, which walks the tree: each object rebuilds `matrix` from position, quaternion and scale (if `matrixAutoUpdate` is on, the default) and multiplies by its parent's world matrix. For a 20-node arm that's nothing; for 50,000 static objects it adds up, which is why static scenery often sets `matrixAutoUpdate = false` and updates by hand.

`getWorldPosition` calls `updateWorldMatrix(true, false)`: update the parents, then itself, but not the children. That's why main.ts can read the TCP right after changing angles and get the right answer.

PART 1.4 · `src/arm/Gripper.ts`

Loading and driving the gripper

The gripper is a glTF file (built by the script at the end of this chapter). This class loads it, finds the fingers by name, and slides them.

Constants and fields

src/arm/Gripper.ts · lines 1–25

```
import * as THREE from "three";
import { GLTFLoader } from "three/addons/loaders/GLTFLoader.js";

const CLOSED_X = 0.025;
const OPEN_X = 0.07;
const SPEED = 2.5; // full open/close strokes per second

/**
 * Parallel two-finger gripper loaded from glTF. The model's finger nodes are
 * found by name and slid along X, so the asset stays a plain static file with
 * no animation clips baked in.
 */
export class Gripper {
  readonly object: THREE.Object3D;
  private readonly left: THREE.Object3D;
  private readonly right: THREE.Object3D;
  private openness = 1; // 0 = closed, 1 = open
  private target = 1;

  private constructor(object: THREE.Object3D, left: THREE.Object3D, right: THREE.Object3D) {
    this.object = object;
    this.left = left;
    this.right = right;
    this.apply();
  }
```

The fingers sit 2.5 cm from the center when closed and 7 cm when open. `openness` is the animated value from 0 to 1, `target` is where it's heading. The constructor is **private**, so the only ways to get a `Gripper` are the two static factories below, each of which guarantees both finger nodes exist.

Loading the model

src/arm/Gripper.ts · lines 27–41

```
static async load(url: string): Promise<Gripper> {
  const gltf = await new GLTFLoader().loadAsync(url);
  const root = gltf.scene;
  const left = root.getObjectByName("FingerLeft");
  const right = root.getObjectByName("FingerRight");
  if (!left || !right) throw new Error(`${url} has no FingerLeft/FingerRight nodes`);

  root.traverse((child) => {
    if (child instanceof THREE.Mesh) {
      child.castShadow = true;
      child.receiveShadow = true;
    }
  });
  return new Gripper(root, left, right);
}
```

`loadAsync` fetches and parses the `.glb` and returns a promise. `gltf.scene` is an ordinary Three.js `Group` built from the file's nodes. `getObjectByName` searches the tree for the two node names the build script wrote; if they're missing, loading fails loudly instead of producing a gripper that silently doesn't move.

`traverse` visits every descendant. glTF has no notion of shadows, so they're switched on here for each mesh.

A fallback

src/arm/Gripper.ts · lines 43–61

```
/** Simple boxes, used if the glTF file can't be loaded. */
static placeholder(): Gripper {
  const material = new THREE.MeshStandardMaterial({ color: 0x888888 });
  const root = new THREE.Group();
  const palm = new THREE.Mesh(new THREE.BoxGeometry(0.18, 0.06, 0.07), material);
  palm.position.y = 0.03;
  const finger = () => {
    const pivot = new THREE.Object3D();
    pivot.position.y = 0.06;
    const mesh = new THREE.Mesh(new THREE.BoxGeometry(0.02, 0.07, 0.05), material);
    mesh.position.y = 0.035;
    pivot.add(mesh);
    return pivot;
  };
  const left = finger();
  const right = finger();
  root.add(palm, left, right);
  return new Gripper(root, left, right);
}
```

If the file can't be loaded (wrong path, blocked request), main.ts uses this instead: grey boxes with the same structure, a pivot 6 cm up for each finger. Because the class only ever moves the two pivots, everything else works unchanged. A missing asset degrades the look, not the app.

Opening, closing and animating

src/arm/Gripper.ts · lines 63–86

```
  get isOpen(): boolean {
    return this.target === 1;
  }

  setOpen(open: boolean): void {
    this.target = open ? 1 : 0;
  }

  update(dt: number): void {
    if (this.openness === this.target) return;
    const step = SPEED * dt;
    this.openness =
      this.openness < this.target
        ? Math.min(this.target, this.openness + step)
        : Math.max(this.target, this.openness - step);
    this.apply();
  }

  private apply(): void {
    const x = CLOSED_X + (OPEN_X - CLOSED_X) * this.openness;
    this.left.position.x = -x;
    this.right.position.x = x;
  }
}
```

`setOpen` only changes the target, so callers never deal with animation. `update(dt)` moves `openness` toward the target at 2.5 strokes per second, using `Math.min` / `Math.max` so it lands exactly on the target instead of overshooting. Returning early when there's nothing to do keeps the per-frame cost near zero.

`apply` maps openness to a distance and moves the fingers symmetrically: left to −x, right to +x. The fingers are animated in code rather than with glTF animation clips, which keeps the asset a plain static file.

PART 1.5 · `src/scene/stage.ts`

The stage

Everything except the arm: renderer, camera, orbit controls, lights and floor.

Renderer

src/scene/stage.ts · lines 5–13

```
export function createStage(canvas: HTMLCanvasElement) {
  const renderer = new THREE.WebGLRenderer({ canvas, antialias: true });
  renderer.setPixelRatio(Math.min(window.devicePixelRatio, 2));
  renderer.toneMapping = THREE.ACESFilmicToneMapping;
  renderer.shadowMap.enabled = true;
  renderer.shadowMap.type = THREE.PCFSoftShadowMap;

  const scene = new THREE.Scene();
  scene.background = new THREE.Color(0x15181d);
```

`setPixelRatio` renders at the screen's real resolution on high-DPI displays, capped at 2× because 3× costs 2.25 times the pixels for a barely visible gain. **ACES filmic tone mapping** compresses bright highlights the way film does, instead of clipping them to white. `PCFSoftShadowMap` blurs shadow edges.

Camera and orbit controls

src/scene/stage.ts · lines 15–24

```
const camera = new THREE.PerspectiveCamera(45, 1, 0.01, 50);
camera.position.set(1.3, 1.0, 1.6);

const controls = new OrbitControls(camera, canvas);
controls.target.set(0, 0.4, 0);
controls.enableDamping = true;
controls.minDistance = 0.5;
controls.maxDistance = 5;
controls.maxPolarAngle = Math.PI * 0.49; // stay above the floor
controls.update();
```

A 45° field of view, with near and far clip planes at 1 cm and 50 m. Keeping the near plane as far out as possible matters: depth precision is spread between near and far, and a tiny near plane causes flickering surfaces.

`OrbitControls` lets the user drag to orbit around `target` (40 cm up, roughly the arm's middle). Damping gives the motion inertia, which requires `controls.update()` every frame. `maxPolarAngle` just under 90° stops the camera going below the floor.

Lights and shadows

src/scene/stage.ts · lines 26–35

```
scene.add(new THREE.HemisphereLight(0xdfe8ff, 0x2a2622, 1.2));

const sun = new THREE.DirectionalLight(0xffffff, 2.2);
sun.position.set(2, 4, 2.5);
sun.castShadow = true;
sun.shadow.mapSize.set(2048, 2048);
Object.assign(sun.shadow.camera, { left: -1.5, right: 1.5, top: 1.5, bottom: -1.5, near: 0.5,
far: 10 });
sun.shadow.camera.updateProjectionMatrix();
sun.shadow.normalBias = 0.02;
scene.add(sun);
```

A **hemisphere light** gives cool light from above and warm bounce from below, a cheap stand-in for sky and ground. The directional “sun” casts the shadows. Its shadow camera is an orthographic box (±1.5 m) that only needs to cover the arm's reach: the smaller the box, the sharper the 2048×2048 shadow map. `normalBias` nudges the lookup along the surface normal to avoid “shadow acne”, stripes caused by a surface shadowing itself.

Floor, grid and resizing

src/scene/stage.ts · lines 37–60

```
  // An invisible floor that only shows shadows, plus a grid at 10 cm spacing.
  const floor = new THREE.Mesh(
    new THREE.PlaneGeometry(10, 10).rotateX(-Math.PI / 2),
    new THREE.ShadowMaterial({ opacity: 0.35 }),
  );
  floor.receiveShadow = true;
  scene.add(floor);

  const grid = new THREE.GridHelper(4, 40, 0x3a414d, 0x262b33);
  grid.position.y = 0.001; // avoid z-fighting with the shadow plane
  scene.add(grid);

  function resize() {
    const width = canvas.clientWidth;
    const height = canvas.clientHeight;
    renderer.setSize(width, height, false);
    camera.aspect = width / height;
    camera.updateProjectionMatrix();
  }
  window.addEventListener("resize", resize);
  resize();

  return { renderer, scene, camera, controls };
}
```

The floor is a `ShadowMaterial` plane, invisible except where shadows fall on it, with a grid drawn 1 mm above to avoid **z-fighting** (two surfaces at the same depth flickering as they win the depth test on alternate pixels).

`resize` reads the canvas's CSS size and sets the drawing buffer to match. Passing `false` stops Three.js from overwriting the canvas's CSS size. The camera's aspect ratio must match, followed by `updateProjectionMatrix`, or the scene stretches.

PART 1.6 · `src/scene/Trail.ts`

The tool path trail

A line tracing where the TCP has been. Small, but a good example of writing per-frame code that never allocates.

A fixed buffer

src/scene/Trail.ts · lines 3–32

```
/**
 * A line tracing the recent path of a point. The vertex buffer is allocated once
 * and reused, so adding points never allocates or creates new GPU buffers.
 */
export class Trail {
  readonly line: THREE.Line;
  private readonly positions: Float32Array;
  private readonly attribute: THREE.BufferAttribute;
  private readonly capacity: number;
  private count = 0;
  private readonly last = new THREE.Vector3(Infinity, Infinity, Infinity);

  constructor(capacity = 800) {
    this.capacity = capacity;
    this.positions = new Float32Array(capacity * 3);
    this.attribute = new THREE.BufferAttribute(this.positions, 3);
    this.attribute.setUsage(THREE.DynamicDrawUsage);

    const geometry = new THREE.BufferGeometry();
    geometry.setAttribute("position", this.attribute);
    geometry.setDrawRange(0, 0);

    this.line = new THREE.Line(
      geometry,
      new THREE.LineBasicMaterial({ color: 0xffa94d, transparent: true, opacity: 0.85 }),
    );
    // The bounding sphere would be computed once from the empty buffer and then
    // cull the line wrongly, so skip frustum culling for this one object.
    this.line.frustumCulled = false;
  }
```

The positions live in one `Float32Array` sized for 800 points, allocated once. `DynamicDrawUsage` tells WebGL the data will change often. `setDrawRange(0, 0)` means draw nothing yet: the buffer is full size, but only the first `count` points are used.

Frustum culling skips objects outside the camera's view using a bounding sphere. Three.js computes that sphere once, here from an empty buffer, so the trail would be culled as soon as it grew. Turning culling off for this one object is simpler than recomputing the sphere.

Adding a point

src/scene/Trail.ts · lines 34–47

```
push(point: THREE.Vector3): void {
  if (this.last.distanceToSquared(point) < 1e-6) return; // moved less than 1 mm
  this.last.copy(point);

  if (this.count === this.capacity) {
    this.positions.copyWithin(0, 3); // drop the oldest point
    this.count--;
  }
  point.toArray(this.positions, this.count * 3);
  this.count++;

  this.attribute.needsUpdate = true;
  this.line.geometry.setDrawRange(0, this.count);
}
```

Points closer than 1 mm to the last one are skipped (comparing squared distance avoids a square root). When the buffer is full, `copyWithin(0, 3)` shifts every value back by one point, dropping the oldest. That's 2,400 floats moved, a few microseconds, and it keeps the line in order for drawing. A ring buffer would avoid the copy but would need two draw ranges.

`point.toArray(positions, offset)` writes x, y, z straight into the buffer. `needsUpdate` tells Three.js to upload it to the GPU before the next draw.

Clearing and visibility

src/scene/Trail.ts · lines 49–58

```
  clear(): void {
    this.count = 0;
    this.last.set(Infinity, Infinity, Infinity);
    this.line.geometry.setDrawRange(0, 0);
  }

  set visible(on: boolean) {
    this.line.visible = on;
  }
}
```

Clearing doesn't zero the buffer, it just sets the count to 0: old values are overwritten as new points arrive. A setter (`set visible`) lets main.ts write `trail.visible = on` like a plain property.

GOING DEEPER

Why per-frame allocation matters

JavaScript frees memory with a garbage collector. Short-lived objects are cheap to collect, but not free: allocate a few `Vector3`s per object per frame and, at 60 fps, the collector runs often enough to show up as occasional dropped frames. The habits in this project avoid that: reuse scratch objects (`tcpWorld` in main.ts), preallocate typed arrays (the trail), and keep closures out of hot loops. Snack 3 takes the same idea to its conclusion with structure-of-arrays physics.

PART 1.7 · `src/ui/panel.ts`

The side panel

Plain DOM, no framework. The panel only displays state and reports user intent through handlers; it never touches the arm.

Handlers and helpers

src/ui/panel.ts · lines 4–32

```
export type JogDirection = -1 | 0 | 1;
```

```
export interface PanelHandlers {
  onSelect(index: number): void;
  onJog(direction: JogDirection): void;
  onToggleDemo(): void;
  onToggleGripper(): void;
  onToggleTrail(): void;
  onReset(): void;
}

function byId<T extends HTMLElement = HTMLElement>(id: string): T {
  const el = document.getElementById(id);
  if (!el) throw new Error(`Missing #${id} in index.html`);
  return el as T;
}

/** Writes only when the text changed, to avoid needless DOM work every frame. */
function setText(el: HTMLElement, text: string) {
  if (el.textContent !== text) el.textContent = text;
}

function formatDeg(value: number) {
  return `${(Math.abs(value) < 0.05 ? 0 : value).toFixed(1)}°`;
}

function formatMeters(value: number) {
  return (Math.abs(value) < 0.0005 ? 0 : value).toFixed(3);
}
```

`PanelHandlers` is the panel's whole contract with the app: six callbacks. main.ts decides what they do. `byId` fails loudly if index.html is missing an element, instead of a confusing `null` error later.

`setText` only writes when the text changed. `update` runs every frame, and writing the same text 60 times a second would still make the browser do layout work. `formatDeg` and `formatMeters` snap tiny values to 0, so the display never shows “−0.0°”.

Building the joint list

src/ui/panel.ts · lines 35–55

```
export function createPanel(specs: readonly JointSpec[], handlers: PanelHandlers) {
  const list = byId("joints");
  const rows = specs.map((spec, i) => {
    const button = document.createElement("button");
    button.type = "button";
    button.className = "joint";
    button.innerHTML = `
      <span class="key">${i + 1}</span>
      <span class="name">${spec.name}</span>
      <span class="value"></span>
      <span class="bar" title="${spec.min}° to ${spec.max}°"><span class="marker"></span>
</span>`;
    button.addEventListener("click", () => handlers.onSelect(i));
    const item = document.createElement("li");
    item.append(button);
    list.append(item);
    return {
      button,
      value: button.querySelector<HTMLElement>(".value")!,
      marker: button.querySelector<HTMLElement>(".marker")!,
    };
  });
```

One button per joint spec, with its key number, name, live value and a bar whose `title` shows the limits on hover. Each row keeps references to the elements it will update, so the per-frame code never queries the DOM.

Press-and-hold jog buttons

src/ui/panel.ts · lines 57–67

```
for (const button of document.querySelectorAll<HTMLButtonElement>("[data-jog]")) {
  const direction = Number(button.dataset.jog) as JogDirection;
  button.addEventListener("pointerdown", (e) => {
    button.setPointerCapture(e.pointerId);
    handlers.onJog(direction);
  });
  const stop = () => handlers.onJog(0);
  button.addEventListener("pointerup", stop);
  button.addEventListener("pointercancel", stop);
  button.addEventListener("lostpointercapture", stop);
}
```

Jogging must last exactly as long as the button is held. `setPointerCapture` keeps sending pointer events to the button even if the pointer slides off it, so the release is never missed. Stopping on `pointerup`, `pointercancel` and `lostpointercapture` covers every way a press can end, including touch interruptions.

Updating every frame

src/ui/panel.ts · lines 83–98

```
return {
  update(anglesDeg: readonly number[], scenePos: Vec3, fkPos: Vec3) {
    rows.forEach((row, i) => {
      const spec = specs[i];
      setText(row.value, formatDeg(anglesDeg[i]));
      row.marker.style.left = `${((anglesDeg[i] - spec.min) / (spec.max - spec.min)) * 100}%`;
    });
    const s = [scenePos.x, scenePos.y, scenePos.z];
    const f = [fkPos.x, fkPos.y, fkPos.z];
    for (let k = 0; k < 3; k++) {
      setText(scene[k], formatMeters(s[k]));
      setText(fk[k], formatMeters(f[k]));
    }
    const error = Math.hypot(s[0] - f[0], s[1] - f[1], s[2] - f[2]);
    setText(diff, `${error.toExponential(1)} m`);
  },
```

Sets each joint's value and moves its marker along the bar as a percentage of the joint's range. Then the TCP readout: both positions and the distance between them, in exponent notation, since it's always around 10^{-17}.

Toggles

src/ui/panel.ts · lines 100–122

```
  setSelected(index: number) {
    rows.forEach((row, i) => row.button.classList.toggle("selected", i === index));
    setText(selectedName, specs[index].name);
  },

  setDemo(on: boolean) {
    demo.setAttribute("aria-pressed", String(on));
  },

  setGripperOpen(open: boolean) {
    gripper.setAttribute("aria-pressed", String(open));
    setText(gripper, open ? "Gripper open" : "Gripper closed");
  },

  setTrail(on: boolean) {
    trail.setAttribute("aria-pressed", String(on));
  },

  showStatus(message: string) {
    status.textContent = message;
    status.hidden = false;
  },
};
```

Toggle buttons use `aria-pressed`, which screen readers announce as “pressed” or “not pressed”, and the CSS styles the same attribute. One attribute serves accessibility and appearance, so they can't disagree.

PART 1.8 · `src/main.ts`

Wiring it together

The entry point: builds the scene, connects input, and runs the frame loop. All app state lives here.

Setup and state

src/main.ts · lines 14–76

```
const JOG_SPEED = 60; // degrees per second while a key or jog button is held
const DEMO_SMOOTHING = 3; // higher = joints follow the demo path more tightly

const canvas = document.getElementById("scene") as HTMLCanvasElement;
const { renderer, scene, camera, controls } = createStage(canvas);

const arm = new RobotArm(ARM, JOINTS);
scene.add(arm.root);

const trail = new Trail();
scene.add(trail.line);

let gripper: Gripper | null = null;

// Shapes on the floor, stored in the inventory database (Snack 5's API).
const shapes = createShapes({ root: document.getElementById("shapes-panel")!, scene, camera,
canvas, arm: ARM });

// Conveyor belts, also stored there. Shapes on a running belt ride along.
const belts = createConveyors({
  root: document.getElementById("conveyors-panel")!,
  runButton: document.getElementById("btn-run") as HTMLButtonElement,
  scene, camera, canvas, shapes,
});

// Splitters: gates on the belts that send each item left, right or straight on, by your rules.
const splitters = createSplitters({ root: document.getElementById("splitters-panel")!, scene,
camera, canvas, shapes, conveyors: belts });

// Mergers and joining machines, plus the connection point editor in the Shapes tab.
const line = createLine({
  root: document.getElementById("machines-panel")!, binsRoot: document.getElementById("bins-
panel")!,
  scene, camera, canvas, shapes, conveyors: belts,
});

// The right-hand panel's tabs.
const tabs = {
```

```
  shapes: document.getElementById("tab-shapes")!,
  conveyors: document.getElementById("tab-conveyors")!,
  splitters: document.getElementById("tab-splitters")!,
  machines: document.getElementById("tab-machines")!,
  bins: document.getElementById("tab-bins")!,
};
function showTab(name: keyof typeof tabs) {
  for (const [key, tab] of Object.entries(tabs)) {
    const on = key === name;
    tab.setAttribute("aria-selected", String(on));
    document.getElementById(tab.getAttribute("aria-controls")!)!.hidden = !on;
  }
}
tabs.shapes.addEventListener("click", () => showTab("shapes"));
tabs.conveyors.addEventListener("click", () => showTab("conveyors"));
tabs.splitters.addEventListener("click", () => showTab("splitters"));
tabs.machines.addEventListener("click", () => showTab("machines"));
tabs.bins.addEventListener("click", () => showTab("bins"));
belts.onShowTab(() => showTab("conveyors"));
splitters.onShowTab(() => showTab("splitters"));

const state = {
  selected: 1,
  demo: true,
  demoTime: 0,
  trail: true,
  jog: 0 as JogDirection,
};
```

`JOG_SPEED` is 60°/s while a key is held. The arm and the trail are added to the scene immediately; the gripper is `null` until its file loads, which is why later code says `gripper?.update(dt)`.

The Shapes panel and the Conveyors tab are created here too, with the scene, camera and canvas they need, and the two tabs that switch between them (all covered in the last parts of this chapter). Everything about shapes and belts lives in their own modules; main.ts only hands them the pointer, the keyboard and the frame loop.

`state` is one plain object for everything the UI can change. `0 as JogDirection` tells TypeScript the field holds the type −1 | 0 | 1, not just the literal 0.

Connecting the panel

src/main.ts · lines 78–88

```
const panel = createPanel(JOINTS, {
  onSelect: select,
  onJog: (direction) => {
    state.jog = direction;
    if (direction !== 0) setDemo(false);
  },
  onToggleDemo: () => setDemo(!state.demo),
  onToggleGripper: toggleGripper,
  onToggleTrail: () => setTrail(!state.trail),
  onReset: reset,
});
```

Each handler is one line. Jogging from the panel stops the demo, because the demo would otherwise overwrite the angle on the very next frame. `select` and `reset` are function declarations further down, which JavaScript hoists, so they can be referenced here.

Loading the gripper

src/main.ts · lines 90–102

```
// ---- Gripper (glTF) ----

Gripper.load(`${import.meta.env.BASE_URL}models/gripper.glb`)
  .catch((err: unknown) => {
    console.error(err);
    panel.showStatus("Couldn't load models/gripper.glb, so a placeholder gripper is shown.");
    return Gripper.placeholder();
  })
  .then((loaded) => {
    gripper = loaded;
    arm.toolMount.add(loaded.object);
    panel.setGripperOpen(loaded.isOpen);
  });
```

A promise chain: try the real model, and on any failure log it, tell the user, and fall back to the placeholder. Either way `.then` receives a working `Gripper` and mounts it on the arm's tool mount, so it moves with the wrist.

`import.meta.env.BASE_URL` is Vite's base path, so the URL is right whether the app runs at the site root or in a subfolder.

Actions

src/main.ts · lines 106–140

```
function select(index: number) {
  state.selected = (index + JOINTS.length) % JOINTS.length;
  arm.highlight(state.selected);
  panel.setSelected(state.selected);
}

function setDemo(on: boolean) {
  state.demo = on;
  panel.setDemo(on);
}

function toggleGripper() {
  setDemo(false); // otherwise the demo would immediately override it
  setGripperOpen(!(gripper?.isOpen ?? true));
}

function setGripperOpen(open: boolean) {
  if (!gripper || gripper.isOpen === open) return;
  gripper.setOpen(open);
  panel.setGripperOpen(open);
}

function setTrail(on: boolean) {
  state.trail = on;
  trail.visible = on;
  if (!on) trail.clear();
  panel.setTrail(on);
}

function reset() {
  setDemo(false);
  JOINTS.forEach((spec, i) => arm.setAngle(i, spec.home));
  setGripperOpen(true);
  trail.clear();
}
```

Every user action goes through one of these functions, whether it came from a key, a button or a click on the arm. Each updates `state`, the 3D scene and the panel together, so they can't drift apart.

`(index + JOINTS.length) % JOINTS.length` wraps around in both directions: pressing up on the first joint selects the last. Adding the length first matters, because in JavaScript `-1 % 4` is −1, not 3.

Keyboard

src/main.ts · lines 144–180

```
const held = new Set<string>();

window.addEventListener("keydown", (e) => {
  // Typing a shape's name or size must not move the arm (R would reset it, G open the
gripper…).
  const t = e.target;
  if (t instanceof HTMLInputElement || t instanceof HTMLSelectElement || t instanceof
HTMLTextAreaElement) return;
  if (line.handleKey(e) || splitters.handleKey(e) || belts.handleKey(e) || shapes.handleKey(e))
{
    e.preventDefault();
    return;
  }
  // Let a focused button handle its own Space/Enter instead of toggling twice.
  if (e.target instanceof HTMLButtonElement && (e.key === " " || e.key === "Enter")) return;
  const key = e.key.length === 1 ? e.key.toLowerCase() : e.key;

  if (/^[1-9]$/.test(key) && Number(key) <= JOINTS.length) select(Number(key) - 1);
  else if (key === "ArrowUp") select(state.selected - 1);
  else if (key === "ArrowDown") select(state.selected + 1);
  else if (key === "ArrowLeft" || key === "ArrowRight" || key === "a" || key === "d") {
    held.add(key);
    setDemo(false);
  } else if (key === "g" && !e.repeat) toggleGripper();
  else if (key === " " && !e.repeat) setDemo(!state.demo);
  else if (key === "t" && !e.repeat) setTrail(!state.trail);
  else if (key === "r" && !e.repeat) reset();
  else return;

  e.preventDefault();
});
```

```
window.addEventListener("keyup", (e) => held.delete(e.key.length === 1 ? e.key.toLowerCase() :
e.key));
window.addEventListener("blur", () => held.clear());

function keyboardDirection(): number {
  const negative = held.has("ArrowLeft") || held.has("a") ? 1 : 0;
  const positive = held.has("ArrowRight") || held.has("d") ? 1 : 0;
  return positive - negative;
}
```

Held keys go into a `Set`, so jogging continues while the key is down and stops on keyup, with no dependence on the operating system's key-repeat rate. Toggle keys check `!e.repeat`, so holding G doesn't flicker the gripper. `preventDefault` stops arrows and space from scrolling the page, but only for keys the app used.

The first lines are new with the Shapes panel. A key typed into a text field must stay in that field: typing “r” in a shape's name would otherwise reset the arm. Then the shapes module gets the first look at every key (Esc cancels placing, Delete removes the selected shape), and only keys it doesn't use reach the arm.

On `blur` (the window loses focus) all held keys are cleared. Otherwise, releasing a key while another window has focus would leave the joint jogging forever.

Clicking a joint

src/main.ts · lines 184–217

```
const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
let pressX = 0;
let pressY = 0;

function housingAt(e: PointerEvent) {
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
  return raycaster.intersectObjects(arm.housings, false)[0]?.object;
}

canvas.addEventListener("pointerdown", (e) => {
  pressX = e.clientX;
  pressY = e.clientY;
});
```

```
canvas.addEventListener("pointerup", (e) => {
  if (Math.hypot(e.clientX - pressX, e.clientY - pressY) > 4) return; // that was an orbit drag
  // Whoever is placing something gets the click; then shapes (they sit on top of belts), then
  // splitter gates (they stand over belts), then belts.
  if (line.isActive() && line.handleClick(e)) return; // putting a connection point on an
object, or placing a bin
  if (splitters.isActive() && splitters.handleClick(e)) return;
  if (belts.isActive() && belts.handleClick(e)) return;
  if (shapes.handleClick(e)) return; // placing, moving or selecting a shape
  if (splitters.handlePick(e)) return;
  if (belts.handlePick(e)) return;
  const hit = housingAt(e);
  if (hit) select(hit.userData.jointIndex as number);
});
canvas.addEventListener("pointermove", (e) => {
  if (e.buttons !== 0) return;
  if (line.isActive() ? line.handleHover(e) : splitters.isActive() ? splitters.handleHover(e) :
belts.isActive() ? belts.handleHover(e) : shapes.handleHover(e)) return;
  canvas.style.cursor = housingAt(e) || splitters.hoversGate(e) || belts.hoversBelt(e) ?
"pointer" : "";
});
```

To find what's under the mouse, convert the pointer position to **normalized device coordinates** (−1 to 1 on both axes, with +y up, hence the minus sign), then cast a ray from the camera through that point. `intersectObjects` returns hits sorted by distance; the first is the nearest housing.

Orbit controls also use the mouse, so a click is only a click if the pointer moved less than 4 pixels between press and release. Anything more was a drag to orbit the camera.

A genuine click goes to the shapes module first: while placing or moving a shape, a click on the floor is its drop point, and a click on a shape selects it. Only when the shapes module doesn't want the click does it select a joint. Hover works the same way, so the cursor is a crosshair while placing and a pointer over a shape.

The frame loop

`src/main.ts · lines 221–248`

```
const tcpWorld = new THREE.Vector3();
let lastTime = performance.now();

select(state.selected);
panel.setDemo(state.demo);
panel.setTrail(state.trail);
```

```
renderer.setAnimationLoop((time) => {
  const dt = Math.min((time - lastTime) / 1000, 0.05); // clamp after a hidden tab resumes
  lastTime = time;

  if (state.demo) {
    state.demoTime += dt;
    // Frame-rate independent easing toward the demo path, so toggling the demo
    // on from any pose glides there instead of jumping.
    const k = 1 - Math.exp(-DEMO_SMOOTHING * dt);
    JOINTS.forEach((_, i) => {
      const current = arm.getAngle(i);
      arm.setAngle(i, current + (demoTarget(i, state.demoTime) - current) * k);
    });
    setGripperOpen(demoGripperOpen(state.demoTime));
  } else {
    const direction = keyboardDirection() + state.jog;
    if (direction !== 0) {
      const i = state.selected;
      arm.setAngle(i, arm.getAngle(i) + Math.sign(direction) * JOG_SPEED * dt);
    }
  }
```

`setAnimationLoop` calls the function once per display refresh. `dt` is the frame time in seconds, capped at 50 ms so a tab that was hidden for a minute doesn't make the arm jump.

In demo mode, each joint eases toward its demo target with `k = 1 − e^(−3·dt)`. That's **frame-rate-independent smoothing**: a plain “move 5% of the way each frame” would converge twice as fast at 120 fps as at 60. With the exponential form, the result after one second is the same however it's sliced into frames. Turning the demo on from any pose glides instead of jumping.

Otherwise, the selected joint moves at `JOG_SPEED` in the direction of whatever is held. Multiplying by `dt` makes the speed the same in degrees per second at any frame rate.

Measuring and drawing

src/main.ts · lines 250–262

```
  gripper?.update(dt);
  belts.update(dt); // moves shapes riding the belts while running
  splitters.update(dt); // swings the gates' flaps

  // getWorldPosition walks up the hierarchy and refreshes world matrices first,
  // so it's correct even though the joints changed since the last render.
  arm.tcp.getWorldPosition(tcpWorld);
  if (state.trail) trail.push(tcpWorld);
  panel.update(arm.anglesDeg, tcpWorld, forwardKinematics(ARM, arm.anglesRad()));

  controls.update();
  renderer.render(scene, camera);
});
```

`getWorldPosition` updates the world matrices up the chain and returns the TCP's position: the scene graph's answer. `forwardKinematics` gives the hand-written answer from the same angles. Both go to the panel side by side, so every frame is a live test that the hierarchy and the math agree.

GOING DEEPER

Picking at scale

`Raycaster.intersectObjects` tests the ray against every triangle of the objects it's given, after a quick bounding-sphere check. Four housings is trivial. A factory layout with thousands of parts needs either a **bounding volume hierarchy** (the three-mesh-bvh library makes raycasts logarithmic) or **GPU picking**: render each object's ID as a color into an off-screen buffer and read back the one pixel under the mouse.

PART 1.9 · `tests/kinematics.test.ts`

Testing the kinematics

Seven tests, run with Node's built-in test runner. The interesting one doesn't check hand-picked answers: it checks the function against a completely different way of computing the same thing.

Setup

tests/kinematics.test.ts · lines 1–12

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { DEG, forwardKinematics, maxReach, type Vec3 } from "../src/arm/kinematics";
import { ARM, JOINTS, demoTarget } from "../src/arm/jointSpecs";

const H = ARM.baseHeight + ARM.shoulderHeight; // shoulder joint height
const REACH = maxReach(ARM);

function assertClose(actual: Vec3, expected: Vec3, tolerance = 1e-12) {
  const err = Math.hypot(actual.x - expected.x, actual.y - expected.y, actual.z - expected.z);
  assert.ok(err < tolerance, `expected ${JSON.stringify(expected)}, got
${JSON.stringify(actual)} (error ${err})`);
}
```

`H` is the shoulder height and `REACH` the stretched length. `assertClose` compares positions by distance with a tolerance of 10^{-12}, since floating-point results are rarely exactly equal, and puts both values in the failure message.

An independent reference

tests/kinematics.test.ts · lines 16–44

```
type Mat4 = number[]; // row-major

function multiply(a: Mat4, b: Mat4): Mat4 {
  const out = new Array<number>(16).fill(0);
  for (let r = 0; r < 4; r++)
    for (let c = 0; c < 4; c++)
      for (let k = 0; k < 4; k++) out[r * 4 + c] += a[r * 4 + k] * b[k * 4 + c];
  return out;
}
const translateY = (y: number): Mat4 => [1, 0, 0, 0, 0, 1, 0, y, 0, 0, 1, 0, 0, 0, 0, 1];
const rotateY = (t: number): Mat4 => {
  const c = Math.cos(t), s = Math.sin(t);
  return [c, 0, s, 0, 0, 1, 0, 0, -s, 0, c, 0, 0, 0, 0, 1];
};
const rotateZ = (t: number): Mat4 => {
  const c = Math.cos(t), s = Math.sin(t);
  return [c, -s, 0, 0, s, c, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1];
```

```
};

function matrixChainFK(q: number[]): Vec3 {
  const chain = [
    translateY(ARM.baseHeight), rotateY(q[0]),
    translateY(ARM.shoulderHeight), rotateZ(q[1]),
    translateY(ARM.upperArm), rotateZ(q[2]),
    translateY(ARM.forearm), rotateZ(q[3]),
    translateY(ARM.wrist + ARM.tool),
  ].reduce(multiply);
  return { x: chain[3], y: chain[7], z: chain[11] }; // translation column
}
```

The same arm as a chain of 4×4 matrices: translate up a link, rotate about the joint's axis, translate, rotate… and multiply them all together (`reduce(multiply)`). The position of the tool is the translation column of the result. This is exactly what a scene graph does internally, written out with no shortcuts and no shared code with kinematics.ts.

Known poses

tests/kinematics.test.ts · lines 48–66

```
test("zero pose points straight up", () => {
  assertClose(forwardKinematics(ARM, [0, 0, 0, 0]), { x: 0, y: H + REACH, z: 0 });
});

test("shoulder +90 degrees lays the arm along -X", () => {
  assertClose(forwardKinematics(ARM, [0, 90 * DEG, 0, 0]), { x: -REACH, y: H, z: 0 });
});

test("base +90 degrees swings that pose to +Z", () => {
  assertClose(forwardKinematics(ARM, [90 * DEG, 90 * DEG, 0, 0]), { x: 0, y: H, z: REACH });
});

test("home pose has the gripper pointing straight down", () => {
  const q = JOINTS.map((j) => j.home * DEG);
  const tcp = forwardKinematics(ARM, q);
  const flange = forwardKinematics({ ...ARM, wrist: 0, tool: 0 }, q); // the wrist joint
  assert.ok(Math.abs(tcp.x - flange.x) < 1e-12 && Math.abs(tcp.z - flange.z) < 1e-12);
  assert.ok(tcp.y < flange.y);
});
```

Poses you can check in your head: everything at 0 points straight up; shoulder at +90° lays the arm flat along −X; base +90° swings that to +Z. These pin down the sign conventions, which are the most common source of FK bugs.

The home-pose test computes the wrist joint's position by running FK with zero-length wrist and tool, then checks the TCP is directly below it: the gripper points straight down.

Random poses, and limits

tests/kinematics.test.ts · lines 68–96

```
test("matches a 4x4 matrix chain for 1,000 random poses", () => {
  let seed = 42; // deterministic pseudo-random numbers
  const random = () => ((seed = (seed * 1664525 + 1013904223) % 2 ** 32) / 2 ** 32);
  for (let i = 0; i < 1000; i++) {
    const q = JOINTS.map((j) => (j.min + random() * (j.max - j.min)) * DEG);
    assertClose(forwardKinematics(ARM, q), matrixChainFK(q));
  }
});

test("never reaches farther than the stretched arm", () => {
  for (let i = 0; i < 500; i++) {
    const q = [i * 0.7, Math.sin(i), Math.cos(i * 1.3), Math.sin(i * 0.4) * 2];
    const p = forwardKinematics(ARM, q);
    assert.ok(Math.hypot(p.x, p.y - H, p.z) <= REACH + 1e-12);
  }
});

test("demo motion stays within joint limits and keeps the gripper above the floor", () => {
  const FINGERTIP_BELOW_TCP = 0.015; // fingertips extend a little past the TCP
  let lowest = Infinity;
  for (let t = 0; t < 600; t += 0.05) {
    const anglesDeg = JOINTS.map((_, i) => demoTarget(i, t));
    anglesDeg.forEach((a, i) => {
      assert.ok(a >= JOINTS[i].min && a <= JOINTS[i].max, `${JOINTS[i].name} = ${a} at
t=${t}`);
    });
    lowest = Math.min(lowest, forwardKinematics(ARM, anglesDeg.map((a) => a * DEG)).y);
  }
  assert.ok(lowest - FINGERTIP_BELOW_TCP > 0.02, `gripper dips to ${lowest.toFixed(3)} m`);
});
```

1,000 random poses within the joint limits, each compared with the matrix chain. A small inline random generator with a fixed seed makes the “random” poses identical on every run, so a failure can always be reproduced.

The last test samples ten minutes of the demo motion every 50 ms and checks every angle is within its limits and the fingertips stay at least 2 cm above the floor. That's the test that forced the demo amplitudes down.

PART 1.10 · `scripts/build-gripper.ts`

Building the gripper model by hand

This script writes `public/models/gripper.glb`, a binary glTF 2.0 file, with no 3D software and no libraries. Doing it by hand shows exactly what's inside a glTF file.

The header comment

scripts/build-gripper.ts · lines 1–14

```
/**
 * Writes public/models/gripper.glb: a two-finger parallel gripper as a binary glTF 2.0 file.
 *
 * No dependencies: geometry, buffer layout and the GLB container are built by hand, which
 * keeps the asset reproducible from source. Run with `npm run build:gripper`.
 *
 * Node hierarchy (names matter, Gripper.ts looks them up):
 *   Gripper
 *   ├── Palm
 *   ├── FingerLeft   (origin at the finger root, slides along X)
 *   └── FingerRight
 *
 * Units are meters, +Y points away from the wrist flange, matching the glTF convention.
 */
```

Documentation only. It says what the script produces, that it has no dependencies, how to run it, and the node hierarchy the app expects. The names matter because Gripper.ts looks up `FingerLeft` and `FingerRight` by name.

Imports

scripts/build-gripper.ts · lines 15–17

```
import { mkdirSync, writeFileSync } from "node:fs";
import { dirname } from "node:path";
import { fileURLToPath } from "node:url";
```

- `mkdirSync`: creates a folder, used to make sure `public/models/` exists.
- `writeFileSync`: writes the finished bytes to disk.
- `dirname`: gets the folder part of a path (`.../public/models/gripper.glb` becomes `.../public/models`).
- `fileURLToPath`: turns a `file:///...` URL into a normal path. ES modules know their own location as a URL, not a path.

The `node:` prefix means “Node's built-in module”, not something from npm.

Types

scripts/build-gripper.ts · lines 19–24

```
type V3 = [number, number, number];

interface Box {
  center: V3;
  size: V3;
}
```

A 3D vector as a fixed-length tuple, and a box described by its center and its full width, height and depth. The whole gripper is built from boxes.

The face table

scripts/build-gripper.ts · lines 26–36

```
// For each face: outward normal n, and in-plane axes u, v with u × v = n,
// so the corners below wind counter-clockwise when seen from outside (glTF front faces).
const FACES: { n: V3; u: V3; v: V3 }[] = [
  { n: [1, 0, 0], u: [0, 1, 0], v: [0, 0, 1] },
  { n: [-1, 0, 0], u: [0, 0, 1], v: [0, 1, 0] },
  { n: [0, 1, 0], u: [0, 0, 1], v: [1, 0, 0] },
  { n: [0, -1, 0], u: [1, 0, 0], v: [0, 0, 1] },
  { n: [0, 0, 1], u: [1, 0, 0], v: [0, 1, 0] },
  { n: [0, 0, -1], u: [0, 1, 0], v: [1, 0, 0] },
];
const CORNERS: [number, number][] = [[-1, -1], [1, -1], [1, 1], [-1, 1]];
```

One row per face of a box. `n` is the **normal**, the direction the face points; `u` and `v` are two directions lying flat on the face, used to step from the face's center to its corners.

The comment says `u × v = n`. That's the important part: it guarantees the corners come out **counter-clockwise when viewed from outside**, which is how glTF and WebGL decide which side of a triangle is the front. Get it backwards and the faces are invisible from outside, because back faces are culled.

`CORNERS` lists the four corners as (u, v) steps: bottom-left, bottom-right, top-right, top-left. Going round in this order is the counter-clockwise loop.

Turning boxes into vertex data

scripts/build-gripper.ts · lines 38–56

```
/** Boxes merged into one flat-shaded mesh: 4 vertices and 2 triangles per face. */
function boxesToMesh(boxes: Box[]) {
  const positions: number[] = [];
  const normals: number[] = [];
  const indices: number[] = [];
  for (const { center, size } of boxes) {
    for (const { n, u, v } of FACES) {
      const first = positions.length / 3;
      for (const [su, sv] of CORNERS) {
        for (let k = 0; k < 3; k++) {
          positions.push(center[k] + ((n[k] + su * u[k] + sv * v[k]) * size[k]) / 2);
        }
        normals.push(...n);
      }
      indices.push(first, first + 1, first + 2, first, first + 2, first + 3);
    }
  }
  return { positions, normals, indices };
}
```

Three flat arrays, which is what the GPU wants: `positions` (x, y, z per vertex), `normals` (one direction per vertex, for lighting) and `indices` (three vertex numbers per triangle). For every box and every face, `first` remembers the number of the first vertex the face adds (three numbers per vertex, hence ÷ 3).

The corner formula reads: **start at the box center, step out to the face (`n`), step sideways to the corner (± `u`, ± `v`), and scale each step by half the box size on that axis.** It works because `n`, `u` and `v` each point along a different axis, so on any one axis only one of them is nonzero.

Every corner of a face gets that face's normal. That's why each face has its own four vertices instead of sharing corners with its neighbours: shared corners would average the normals and the box would look rounded instead of crisp.

Finally the 4-corner face is split into two triangles, 0-1-2 and 0-2-3, both keeping the counter-clockwise order. One box is 24 vertices and 12 triangles.

The gripper's shape

scripts/build-gripper.ts · lines 58–71

```
// ---- Geometry ----

const palm: Box[] = [
  { center: [0, 0.01, 0], size: [0.09, 0.02, 0.09] }, // mounting flange
  { center: [0, 0.04, 0], size: [0.18, 0.04, 0.07] }, // body, top face at y = 0.06
];

/** side = +1 for the left finger (pad faces +X, toward the center), -1 for the right. */
function finger(side: 1 | -1): Box[] {
  return [
    { center: [0, 0.035, 0], size: [0.02, 0.07, 0.05] }, // finger
    { center: [side * 0.016, 0.055, 0], size: [0.012, 0.03, 0.044] }, // grip pad
  ];
}
```

The palm is two boxes, in meters: a 9 × 2 × 9 cm mounting flange from y = 0 to 2 cm, sitting flush on the wrist, and an 18 cm wide body from 2 to 6 cm.

A finger is a 2 × 7 × 5 cm bar plus a grip pad near its tip, on the inner side. `side` flips which side that is: `+1` puts the pad at +X for the left finger, `−1` at −X for the right. The finger's y runs from 0 to 7 cm because its origin is its root, where it attaches to the palm.

glTF constants and storage

scripts/build-gripper.ts · lines 73–84

```
// ---- glTF JSON + binary buffer ----

const ARRAY_BUFFER = 34962;
const ELEMENT_ARRAY_BUFFER = 34963;
const FLOAT = 5126;
const UNSIGNED_SHORT = 5123;

const binChunks: Buffer[] = [];
let binLength = 0;
const bufferViews: object[] = [];
const accessors: object[] = [];
const meshes: object[] = [];
```

WebGL's numeric codes, which glTF reuses: `ARRAY_BUFFER` marks vertex data, `ELEMENT_ARRAY_BUFFER` triangle indices, `FLOAT` a 32-bit float and `UNSIGNED_SHORT` a 16-bit whole number (0 to 65,535, plenty for 48 vertices).

A glTF file has two parts: a **JSON** description and one **binary blob** of raw numbers. `binChunks` collects pieces of the blob and `binLength` tracks its size; the other three arrays are the JSON sections that point into it.

Appending to the blob

scripts/build-gripper.ts · lines 86–96

```
function addBufferView(data: Buffer, target: number): number {
  const padding = (4 - (binLength % 4)) % 4; // keep every view 4-byte aligned
  if (padding) {
    binChunks.push(Buffer.alloc(padding));
    binLength += padding;
  }
  bufferViews.push({ buffer: 0, byteOffset: binLength, byteLength: data.length, target });
  binChunks.push(data);
  binLength += data.length;
  return bufferViews.length - 1;
}
```

glTF requires each block of float data to start at a byte offset divisible by 4. `(4 - (binLength % 4)) % 4` is how many filler bytes get there: if the blob is 10 bytes long, 10 % 4 = 2, so 2 bytes of padding. The outer `% 4` turns “4 needed” into 0 when already aligned.

It matters because index data (2 bytes each) can leave the blob at an odd length. Then a **bufferView** is recorded: “in buffer 0, starting at this byte, this many bytes, used for vertices or indices”. The function returns the view's index, because glTF refers to everything by its position in an array.

Accessors

scripts/build-gripper.ts · lines 98–101

```
function addAccessor(accessor: object): number {
  accessors.push(accessor);
  return accessors.length - 1;
}
```

An **accessor** says how to read a bufferView: “48 items, each 3 floats (VEC3)”. Three layers, then: the buffer holds raw bytes, a bufferView marks a byte range within it, and an accessor says how to interpret that range as numbers.

One complete mesh

scripts/build-gripper.ts · lines 103–113

```
function addMesh(name: string, boxes: Box[], material: number): number {
  const { positions, normals, indices } = boxesToMesh(boxes);
  const pos = new Float32Array(positions);
  const count = pos.length / 3;

  const min: V3 = [Infinity, Infinity, Infinity];
  const max: V3 = [-Infinity, -Infinity, -Infinity];
  for (let i = 0; i < pos.length; i++) {
    min[i % 3] = Math.min(min[i % 3], pos[i]);
    max[i % 3] = Math.max(max[i % 3], pos[i]);
  }
```

Build the vertex data and convert the positions to a `Float32Array`, the exact byte format glTF stores. Then compute the smallest and largest x, y and z: glTF **requires** these bounds for positions, and loaders use them for culling. `i % 3` picks the axis as the loop walks the flat list.

The bounds are read from the `Float32Array`, not the original numbers, on purpose: converting to 32 bits rounds slightly, and the bounds must match what's actually stored.

scripts/build-gripper.ts · lines 115–130

```
  const position = addAccessor({
    bufferView: addBufferView(Buffer.from(pos.buffer), ARRAY_BUFFER),
    componentType: FLOAT, count, type: "VEC3", min, max,
  });
  const normal = addAccessor({
    bufferView: addBufferView(Buffer.from(new Float32Array(normals).buffer), ARRAY_BUFFER),
    componentType: FLOAT, count, type: "VEC3",
  });
  const index = addAccessor({
    bufferView: addBufferView(Buffer.from(new Uint16Array(indices).buffer),
ELEMENT_ARRAY_BUFFER),
    componentType: UNSIGNED_SHORT, count: indices.length, type: "SCALAR",
  });

  meshes.push({ name, primitives: [{ attributes: { POSITION: position, NORMAL: normal },
indices: index, material }] });
  return meshes.length - 1;
}
```

Each accessor call also writes its bytes to the blob. `pos.buffer` is the raw memory behind the typed array and `Buffer.from` wraps it without copying. Normals don't need bounds; indices are 16-bit, one number each (`SCALAR`).

The mesh entry ties it together. A **primitive** is one drawable piece: “positions from accessor N, normals from M, triangles from the index accessor, with this material”. Triangles are the default mode, so it isn't stated.

Building the three meshes

scripts/build-gripper.ts · lines 132–136

```
const ANODIZED = 0;
const ACCENT = 1;
const palmMesh = addMesh("Palm", palm, ANODIZED);
const leftMesh = addMesh("FingerLeft", finger(1), ACCENT);
const rightMesh = addMesh("FingerRight", finger(-1), ACCENT);
```

`ANODIZED` and `ACCENT` name the two material slots. The dark palm and two orange, mirror-image fingers are built, keeping each mesh's index.

The glTF JSON

scripts/build-gripper.ts · lines 138–148

```
const OPEN_X = 0.07;
const gltf = {
  asset: { version: "2.0", generator: "robot-arm/scripts/build-gripper.ts" },
  scene: 0,
  scenes: [{ name: "Gripper", nodes: [0] }],
  nodes: [
    { name: "Gripper", children: [1, 2, 3] },
    { name: "Palm", mesh: palmMesh },
    { name: "FingerLeft", mesh: leftMesh, translation: [-OPEN_X, 0.06, 0] },
    { name: "FingerRight", mesh: rightMesh, translation: [OPEN_X, 0.06, 0] },
  ],
```

`asset.version` is required; `generator` records which tool made the file. One scene, whose top node is node 0.

The **nodes** form the hierarchy, like Object3Ds in Three.js or GameObjects in Unity. “Gripper” is an empty parent with three children. Each finger is placed 7 cm to the side and 6 cm up (the top of the palm), so the model starts open. These are the names Gripper.ts searches for, and moving a finger means changing its node's position.

scripts/build-gripper.ts · lines 149–163

```
  meshes,
  materials: [
    {
      name: "Anodized",
      pbrMetallicRoughness: { baseColorFactor: [0.16, 0.17, 0.19, 1], metallicFactor: 0.6,
roughnessFactor: 0.45 },
    },
    {
      name: "Accent",
      pbrMetallicRoughness: { baseColorFactor: [0.95, 0.42, 0.08, 1], metallicFactor: 0.1,
roughnessFactor: 0.55 },
    },
  ],
  accessors,
  bufferViews,
  buffers: [{ byteLength: binLength }],
};
```

Two **PBR** (physically based) materials: `baseColorFactor` is red, green, blue and alpha from 0 to 1, `metallicFactor` runs from plastic (0) to metal (1), `roughnessFactor` from mirror (0) to matte (1). Three.js's `GLTFLoader` turns them into `MeshStandardMaterial`, the same material the arm uses. The single buffer's size is declared last; in a .glb that buffer is the binary chunk after the JSON.

Packing it into a .glb

scripts/build-gripper.ts · lines 165–178

```
// ---- GLB container: 12-byte header, JSON chunk (space padded), BIN chunk (zero padded) ----

function pad(data: Buffer, fill: number): Buffer {
  const padding = (4 - (data.length % 4)) % 4;
  return padding ? Buffer.concat([data, Buffer.alloc(padding, fill)]) : data;
}

const json = pad(Buffer.from(JSON.stringify(gltf)), 0x20);
const bin = pad(Buffer.concat(binChunks), 0x00);

const header = Buffer.alloc(12);
header.writeUInt32LE(0x46546c67, 0); // "glTF"
header.writeUInt32LE(2, 4);
header.writeUInt32LE(12 + 8 + json.length + 8 + bin.length, 8);
```

A `.glb` is one binary file: a 12-byte header, a JSON chunk, a binary chunk. Each chunk's length must be a multiple of 4. The JSON is padded with spaces (`0x20`) so it still parses; the binary with zeros.

The header is three 32-bit numbers in **little-endian** order (lowest byte first, which glTF requires): the magic number `0x46546c67`, which reads `g l T F` as bytes and lets a loader recognise the file; the container version, 2; and the total size, 12 for the header plus 8 + JSON and 8 + binary for the two chunks.

scripts/build-gripper.ts · lines 180–193

```
function chunkHeader(length: number, type: number): Buffer {
  const h = Buffer.alloc(8);
  h.writeUInt32LE(length, 0);
  h.writeUInt32LE(type, 4);
  return h;
}

const glb = Buffer.concat([
  header,
  chunkHeader(json.length, 0x4e4f534a), // "JSON"
  json,
  chunkHeader(bin.length, 0x004e4942), // "BIN\0"
  bin,
]);
```

Every chunk starts with 8 bytes: its length, then a type code. Like the magic number, the codes spell `JSON` and `BIN` when read as bytes. Then the five pieces are concatenated in order.

Writing the file

scripts/build-gripper.ts · lines 195–198

```
const out = fileURLToPath(new URL("../public/models/gripper.glb", import.meta.url));
mkdirSync(dirname(out), { recursive: true });
writeFileSync(out, glb);
console.log(`Wrote ${out} (${glb.length} bytes)`);
```

`import.meta.url` is this script's own location, so resolving `../public/models/gripper.glb` against it puts the output in the right place **whichever folder you run the command from**. `recursive: true` makes any missing folders and doesn't complain if they exist. The finished file is 6,376 bytes.

GOING DEEPER

What a production asset pipeline adds

Real models come from CAD or Blender, not hand-written boxes. A typical web pipeline then runs tools like gltf-transform or gltfpack to merge duplicate data, **quantize** positions and normals to 16 or 8 bits, compress geometry with Draco or meshopt, convert textures to KTX2/Basis (compressed on the GPU too), and checks the result with the Khronos glTF Validator.

For a browser-based design tool, CAD formats such as STEP describe exact surfaces, not triangles, so parts are **tessellated** into meshes first, at a level of detail chosen per part. Writing a glTF by hand, as here, is the best way to understand what all of those tools are producing.

PART 1.11 · `src/shapes/types.ts`

Shapes in the scene: the data and the reach rule

The arm's floor can hold shapes: boxes, cylinders, spheres and cones, or inventory parts drawn as one of those. They're stored in the inventory database (Chapter 5), so this app is now a client of that API.

The shape type

src/shapes/types.ts · lines 1–38

```
/** Shapes as the inventory API stores them (snack5-inventory/api/src/domain/shapes.ts). */

export type ShapeKind = "box" | "cylinder" | "sphere" | "cone";
export const SHAPE_KINDS: readonly ShapeKind[] = ["box", "cylinder", "sphere", "cone"];

/** Bounding box in meters: width, height, depth. Round shapes have depth = width. */
export interface Size {
  x: number;
  y: number;
  z: number;
}

export interface Shape {
  id: string;
  name: string;
```

```
  kind: ShapeKind;
  size: Size;
  color: string;
  /** Floor position of the center, in meters. */
  position: { x: number; z: number };
  /** Degrees about the vertical axis. */
  rotationY: number;
  /** The inventory part this shape stands for, or null for a plain shape. */
  sku: string | null;
  /** Set once a machine has joined it onto another shape: it then moves with that shape. */
  attachedTo?: string | null;
  /** Where it sits on that shape: its center in that shape's frame (meters), and its turn
relative to it. */
  attach?: { x: number; y: number; z: number; rotationY: number } | null;
  /** Set while it's stored in a bin: off the floor and out of sight. */
  binId?: string | null;
  /** True once its part has left stock (loaded into or caught by a bin, or joined). */
  stockTaken?: boolean;
  createdAt: string;
  updatedAt: string;
}

export type NewShape = Omit<Shape, "id" | "createdAt" | "updatedAt" | "attachedTo" | "attach" |
"binId" | "stockTaken">;
export type ShapePatch = Partial<Pick<Shape, "name" | "color" | "position" | "rotationY">>;
```

This mirrors the API's type field for field, including the rule that `size` is always a bounding box: a cylinder's depth equals its width, a sphere's three sizes are its diameter. One meaning for `size` lets the drawing code and the reach code treat every kind the same way.

Only x and z are stored for the position. Every shape rests on the floor, so its height decides where its center is; storing y as well would allow shapes floating in the air or sunk into the floor.

Can the gripper get there?

src/shapes/reach.ts · lines 1–50

```
/**
 * Can the arm reach a shape? Pure math on the arm's dimensions, no Three.js, so it's
 * unit-tested like the kinematics.
 *
 * The gripper picks from above, so the question is whether the tool center point can get
 * to the middle of the shape's top face. The arm is a chain hinged at the shoulder, so the
```

```
 * points it can reach lie within a sphere of radius maxReach around the shoulder joint.
 * A small margin keeps "reachable" from meaning "only with the arm locked straight".
 */
import { maxReach, type ArmDimensions } from "../arm/kinematics";
import type { Shape } from "./types";

export type Reach = "reachable" | "too-far" | "too-close" | "overlaps-base";

/** Radius of the pedestal at the floor (RobotArm.ts builds it 0.2 m at the bottom). */
export const BASE_RADIUS = 0.2;
/** Closer than this, the arm would have to fold back on its own column. */
export const MIN_REACH_RADIUS = 0.25;
export const REACH_MARGIN = 0.03;

/** Radius of the circle a shape's footprint fits in, whatever its rotation. */
export function footprintRadius(shape: Pick<Shape, "kind" | "size">): number {
  return shape.kind === "box" ? Math.hypot(shape.size.x, shape.size.z) / 2 : shape.size.x / 2;
}

/** baseY is what the shape stands on: 0 on the floor, a belt's height on a conveyor. */
export function reachOf(shape: Pick<Shape, "kind" | "size" | "position">, arm: ArmDimensions,
baseY = 0): Reach {
  const { x, z } = shape.position;
  const radius = Math.hypot(x, z); // horizontal distance from the base axis
  if (radius - footprintRadius(shape) < BASE_RADIUS) return "overlaps-base";
  if (radius < MIN_REACH_RADIUS) return "too-close";
  const shoulderY = arm.baseHeight + arm.shoulderHeight;
  const topY = baseY + shape.size.y;
  const distance = Math.hypot(radius, topY - shoulderY);
  return distance <= maxReach(arm) - REACH_MARGIN ? "reachable" : "too-far";
}

/** How far out, along the floor, the arm can reach a point at height y. */
export function reachRadiusAt(y: number, arm: ArmDimensions): number {
  const r = maxReach(arm) - REACH_MARGIN;
  const dy = y - (arm.baseHeight + arm.shoulderHeight);
  return Math.sqrt(Math.max(0, r * r - dy * dy));
}

export const REACH_LABEL: Record<Reach, string> = {
  reachable: "in reach",
```

```
  "too-far": "out of reach",
  "too-close": "too close",
  "overlaps-base": "hits the base",
};
```

The gripper picks from above, so the target is the middle of the shape's top face. Every point the TCP can reach lies inside a sphere of radius `maxReach` around the shoulder, because the chain is hinged there: that's the whole test, `Math.hypot(radius, topY - shoulderY) <= maxReach - margin`. The 3 cm margin means “in reach” doesn't require the arm locked dead straight.

Two checks come first. A shape whose footprint overlaps the pedestal's 0.2 m radius would collide with the base, and a point closer than 0.25 m would need the arm to fold back on its own column. A box's footprint radius is half its diagonal, so the check holds at any rotation.

`reachRadiusAt(y)` turns the sphere into a circle at one height: Pythagoras on the sphere's radius and the height difference. At floor level it's the blue ring drawn on the floor.

Testing the reach rule

tests/reach.test.ts · lines 1–36

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { ARM } from "../src/arm/jointSpecs";
import { maxReach } from "../src/arm/kinematics";
import { BASE_RADIUS, footprintRadius, reachOf, reachRadiusAt, REACH_MARGIN } from
"../src/shapes/reach";

const cube = (x: number, z: number, cm = 20) => ({ kind: "box" as const, size: { x: cm / 100,
y: cm / 100, z: cm / 100 }, position: { x, z } });

test("a small box in front of the arm is reachable", () => {
  assert.equal(reachOf(cube(0.6, 0), ARM), "reachable");
  assert.equal(reachOf(cube(0, -0.6), ARM), "reachable"); // the base turns, so direction
doesn't matter
});

test("far away is out of reach, and the limit matches the reach sphere", () => {
  assert.equal(reachOf(cube(1.5, 0), ARM), "too-far");
  // A thin 1 cm plate's top is at y = 0.01: the limit is the reach sphere's radius at that
height.
  const r = reachRadiusAt(0.01, ARM);
  const plate = (x: number) => ({ kind: "box" as const, size: { x: 0.05, y: 0.01, z: 0.05 },
position: { x, z: 0 } });
```

```
  assert.equal(reachOf(plate(r - 0.001), ARM), "reachable");
  assert.equal(reachOf(plate(r + 0.001), ARM), "too-far");
});

test("tall shapes reach higher, so they can be reached further out", () => {
  const tall = { kind: "cylinder" as const, size: { x: 0.1, y: 0.35, z: 0.1 }, position: { x:
1.04, z: 0 } };
  const flat = { ...tall, size: { x: 0.1, y: 0.02, z: 0.1 } };
  assert.equal(reachOf(tall, ARM), "reachable"); // top level with the shoulder: full reach
sideways
  assert.equal(reachOf(flat, ARM), "too-far");
  assert.ok(Math.abs(reachRadiusAt(ARM.baseHeight + ARM.shoulderHeight, ARM) - (maxReach(ARM) -
REACH_MARGIN)) < 1e-12);
});

test("shapes overlapping the pedestal, or tucked against it, are flagged", () => {
  assert.equal(reachOf(cube(0.25, 0), ARM), "overlaps-base"); // 20 cm box reaches within 0.2 m
of the axis
  assert.equal(reachOf({ kind: "sphere", size: { x: 0.04, y: 0.04, z: 0.04 }, position: { x:
0.23, z: 0 } }, ARM), "too-close");
  assert.ok(footprintRadius(cube(0, 0)) > 0.1); // a box's footprint circle covers its corners
  assert.equal(BASE_RADIUS, 0.2);
});
```

The boundary test is the interesting one: it computes the exact limit for a 1 cm plate from `reachRadiusAt`, then checks a plate 1 mm inside is reachable and 1 mm outside is not. The tall-shapes test captures something a person might not expect: a shape whose top is level with the shoulder can be reached furthest out, because the arm reaches sideways there at full length.

PART 1.12 · **src/shapes/fromPart.ts**

Inventory parts as shapes

The inventory knows SKUs, names and stock, but not dimensions. This module gives each part a plausible shape from what its SKU and category say.

Reading sizes out of SKUs

src/shapes/fromPart.ts · lines 1–33

```
/**
 * Turns an inventory part into a shape for the scene. The inventory doesn't store
```

```
 * dimensions, so they're read from the SKU where it carries them (BAR-4040 is a 40 x 40 mm
 * profile, PL-200 a 200 mm plate, LA-300 a 300 mm stroke) and otherwise come from a
 * typical size for the category. Pure, so it's unit-tested.
 */
import type { NewShape, ShapeKind, Size } from "./types";

/** The fields of an inventory part this needs (snack5-inventory/api/src/domain/types.ts). */
export interface InventoryPart {
  sku: string;
  name: string;
  category: string;
  unit: string;
  onHand: number;
  available: number;
}

export interface PartLook {
  kind: ShapeKind;
  size: Size;
  color: string;
}

/** A cut piece, not a whole 3 m bar, so it fits on the arm's floor. */
export const EXTRUSION_PIECE_M = 0.5;

const mm = (n: number) => n / 1000;

/** The first run of digits in a SKU, e.g. 4040 in BAR-4040-CLR. */
function digits(sku: string): string | null {
  return /(\d+)/.exec(sku)?.[1] ?? null;
}
```

`InventoryPart` lists only the fields this module uses, a deliberately narrow type. The full part has more, and any object with at least these fields fits (TypeScript's structural typing), so the API's response can be passed straight in.

src/shapes/fromPart.ts · lines 35–66

```
export function lookFor(part: Pick<InventoryPart, "sku" | "category">): PartLook {
  const d = digits(part.sku);
  switch (part.category) {
    case "extrusion": {
```

```
      // BAR-2020 / BAR-4040 / BAR-4080: width and height of the profile in mm.
      const w = d && d.length === 4 ? Number(d.slice(0, 2)) : 40;
      const h = d && d.length === 4 ? Number(d.slice(2)) : 40;
      const black = /-BLK\b/.test(part.sku);
      // Lying on the floor along x, taller side up.
      return { kind: "box", size: { x: EXTRUSION_PIECE_M, y: mm(Math.max(w, h)), z:
mm(Math.min(w, h)) }, color: black ? "#2b2f36" : "#c3c9d1" };
    }
    case "plate": {
      if (/^BR-/.test(part.sku)) return { kind: "box", size: { x: 0.04, y: 0.04, z: 0.04 },
color: "#9aa3af" }; // corner bracket
      const side = d ? Math.min(Math.max(Number(d), 50), 1000) : 150;
      return { kind: "box", size: { x: mm(side), y: 0.01, z: mm(side) }, color: "#9aa3af" };
    }
    case "fastener":
      return { kind: "box", size: { x: 0.08, y: 0.04, z: 0.06 }, color: "#b7791f" }; // a small
box of bolts
    case "motor":
      return { kind: "cylinder", size: { x: 0.09, y: 0.14, z: 0.09 }, color: "#1f2937" };
    case "actuator": {
      const stroke = d ? Math.min(Number(d), 1500) : 300;
      return { kind: "box", size: { x: mm(stroke) + 0.1, y: 0.06, z: 0.06 }, color: "#3b82f6"
};
    }
    case "controller":
      return { kind: "box", size: { x: 0.2, y: 0.08, z: 0.15 }, color: "#334155" };
    case "gripper":
      return { kind: "box", size: { x: 0.12, y: 0.08, z: 0.06 }, color: "#f59e0b" };
    default:
      return { kind: "box", size: { x: 0.1, y: 0.1, z: 0.1 }, color: "#94a3b8" };
  }
}
```

Vention-style SKUs carry dimensions: `BAR-4080-BLK` is a 40 × 80 mm black profile, `PL-300` a 300 mm plate, `LA-300` a 300 mm stroke. `digits` takes the first run of numbers and each category interprets it. Bars are drawn as a 50 cm cut piece, not a full 3 m bar that would dwarf the arm, lying on their narrow side with the tall side up.

Exceptions are written down rather than guessed: `BR-90` is a 90° bracket, not a 90 mm one, so brackets get a fixed 4 cm block. Anything unknown gets a neutral 10 cm box, so a new category never breaks the scene.

Linking a shape to its part

src/shapes/fromPart.ts · lines 68–78

```
/** A new shape for `part` at (x, z), named after it and linked to it by SKU. */
export function shapeForPart(part: InventoryPart, x: number, z: number, look: PartLook =
lookFor(part), rotationY = 0): NewShape {
  return { name: part.name.slice(0, 40), ...look, position: { x, z }, rotationY, sku: part.sku
};
}

/** How many shapes in the scene stand for each SKU. */
export function placedBySku(shapes: readonly { sku: string | null }[]): Map<string, number> {
  const counts = new Map<string, number>();
  for (const s of shapes) if (s.sku) counts.set(s.sku, (counts.get(s.sku) ?? 0) + 1);
  return counts;
}
```

A part's shape carries its SKU. That link is what lets the panel say “2 in scene, 1 on hand”: `placedBySku` counts shapes per SKU, and the panel compares with stock. Placing a part doesn't change stock: the scene is a layout, and holding stock is what reservations are for (Chapter 5).

tests/fromPart.test.ts · lines 1–31

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { EXTRUSION_PIECE_M, lookFor, placedBySku, shapeForPart } from "../src/shapes/fromPart";

test("extrusions: the profile comes from the SKU, black or clear from the suffix", () => {
  const black = lookFor({ sku: "BAR-4080-BLK", category: "extrusion" });
  assert.deepEqual(black.size, { x: EXTRUSION_PIECE_M, y: 0.08, z: 0.04 });
  assert.equal(black.color, "#2b2f36");
  assert.deepEqual(lookFor({ sku: "BAR-2020-CLR", category: "extrusion" }).size, { x: 0.5, y:
0.02, z: 0.02 });
});

test("plates, brackets and actuators take their size from the SKU", () => {
  assert.deepEqual(lookFor({ sku: "PL-300", category: "plate" }).size, { x: 0.3, y: 0.01, z:
0.3 });
  assert.deepEqual(lookFor({ sku: "BR-90", category: "plate" }).size, { x: 0.04, y: 0.04, z:
0.04 }); // 90 is the angle, not a size
  const la = lookFor({ sku: "LA-300", category: "actuator" });
  assert.ok(Math.abs(la.size.x - 0.4) < 1e-12); // 300 mm stroke + 100 mm body
```

```
});

test("every category gets a shape, and unknown ones a neutral box", () => {
  for (const category of ["fastener", "motor", "controller", "gripper", "something-new"]) {
    const look = lookFor({ sku: "X-1", category });
    assert.ok(look.size.x > 0 && look.size.y > 0 && look.size.z > 0, category);
  }
  assert.equal(lookFor({ sku: "SRV-400", category: "motor" }).kind, "cylinder");
});

test("a part's shape is linked by SKU and named after the part", () => {
  const s = shapeForPart({ sku: "MC-1", name: "Motion controller", category: "controller",
unit: "piece", onHand: 4, available: 4 }, 0.5, -0.3);
  assert.deepEqual([s.sku, s.name, s.position], ["MC-1", "Motion controller", { x: 0.5, z: -0.3
}]);
  assert.deepEqual(placedBySku([{ sku: "MC-1" }, { sku: null }, { sku: "MC-1" }, { sku: "PL-
200" }]), new Map([["MC-1", 2], ["PL-200", 1]]));
});
```

PART 1.13 · `src/shapes/ShapeLayer.ts`

Drawing the shapes

One Three.js group holds the stored shapes, a see-through preview for placing, and the reach ring.

The layer

src/shapes/ShapeLayer.ts · lines 1–47

```
import * as THREE from "three";
import { DEG } from "../arm/kinematics";
import type { Shape, ShapeKind, Size } from "./types";
import { poses, rootOf, type Pose } from "../line/joints";

/**
 * Draws the stored shapes in the scene, plus a see-through "ghost" that follows the mouse
 * while placing one, and a faint ring on the floor showing how far the arm reaches.
 *
 * sync() is the only way shapes change: give it the list from the API and it adds, updates
 * and removes meshes to match, keyed by shape id, so polling the API is enough to show
 * changes made from another browser.
```

```
 */
export class ShapeLayer {
  readonly root = new THREE.Group();
  private readonly meshes = new Map<string, THREE.Mesh<THREE.BufferGeometry,
THREE.MeshStandardMaterial>>();
  private readonly ghost: THREE.Mesh<THREE.BufferGeometry, THREE.MeshStandardMaterial>;
  private ghostKey = "";
  private selectedId: string | null = null;

  /**
   * What a shape at (x, z) stands on: the floor (0) unless a conveyor is there. Set by the
   * conveyor controller, so this layer doesn't need to know about belts.
   */
  supportAt: (x: number, z: number) => number = () => 0;

  constructor(reachRadius: number) {
    this.root.name = "Shapes";

    this.ghost = new THREE.Mesh(
      new THREE.BoxGeometry(1, 1, 1),
      new THREE.MeshStandardMaterial({ transparent: true, opacity: 0.45, depthWrite: false }),
    );
    this.ghost.visible = false;
    this.root.add(this.ghost);

    // Floor-level reach: a thin ring, drawn just above the grid to avoid z-fighting.
    const ring = new THREE.Mesh(
      new THREE.RingGeometry(reachRadius - 0.006, reachRadius + 0.006, 128),
      new THREE.MeshBasicMaterial({ color: 0x3b82f6, transparent: true, opacity: 0.5 }),
    );
    ring.rotation.x = -Math.PI / 2;
    ring.position.y = 0.002;
    ring.name = "ReachRing";
    this.root.add(ring);
  }
```

The ghost is a normal mesh with a transparent material. `depthWrite: false` stops it hiding things behind it in the depth buffer, so the floor grid and other shapes stay visible through it, which is what makes it read as a preview.

The reach ring sits 2 mm above the floor, like the grid in stage.ts, to avoid z-fighting with the floor.

Syncing with the database

src/shapes/ShapeLayer.ts · lines 48–116

```
/** Makes the scene match `shapes` exactly. */
sync(shapes: readonly Shape[]): void {
  const keep = new Set(shapes.map((s) => s.id));
  for (const [id, mesh] of this.meshes) {
    if (!keep.has(id)) {
      this.root.remove(mesh);
      mesh.geometry.dispose();
      mesh.material.dispose();
      this.meshes.delete(id);
    }
  }
  for (const s of shapes) {
    let mesh = this.meshes.get(s.id);
    const key = geometryKey(s.kind, s.size);
    if (!mesh) {
      mesh = new THREE.Mesh(geometryFor(s.kind, s.size), new THREE.MeshStandardMaterial({
roughness: 0.55, metalness: 0.05 }));
      mesh.castShadow = true;
      mesh.receiveShadow = true;
      mesh.userData.shapeId = s.id;
      mesh.userData.geometryKey = key;
      this.meshes.set(s.id, mesh);
      this.root.add(mesh);
    } else if (mesh.userData.geometryKey !== key) {
      mesh.geometry.dispose();
      mesh.geometry = geometryFor(s.kind, s.size);
      mesh.userData.geometryKey = key;
    }
    mesh.name = s.name;
    mesh.material.color.set(s.color);
    mesh.userData.halfHeight = s.size.y / 2;
  }
  // Shapes on their own stand on the floor or a belt; joined shapes ride on their base.
  const where = poses(shapes, (s) => ({
    x: s.position.x, y: this.supportAt(s.position.x, s.position.z) + s.size.y / 2, z:
s.position.z, rotationY: s.rotationY,
  }));
  for (const [id, p] of where) this.poseCenter(id, p);
```

```
  // Items stored in a bin (with anything joined onto them) are out of sight.
  const byId = new Map(shapes.map((s) => [s.id, s]));
  for (const s of shapes) this.setVisible(s.id, !byId.get(rootOf(s.id, shapes))?.binId);
  this.select(this.selectedId && keep.has(this.selectedId) ? this.selectedId : null);
}

/**
 * Moves one shape's mesh without touching the database: used every frame while the
 * belts run. y is the height of the shape's base.
 */
pose(id: string, x: number, y: number, z: number): void {
  const mesh = this.meshes.get(id);
  if (mesh) mesh.position.set(x, y + (mesh.userData.halfHeight as number), z);
}

/** Places a shape's center exactly: used for shapes riding on others. */
poseCenter(id: string, p: Pose): void {
  const mesh = this.meshes.get(id);
  if (!mesh) return;
  mesh.position.set(p.x, p.y, p.z);
  mesh.rotation.y = p.rotationY * DEG;
}

setVisible(id: string, visible: boolean): void {
  const mesh = this.meshes.get(id);
  if (mesh) mesh.visible = visible;
}

/** The mesh drawing a shape, so connection point markers can be added to it. */
meshOf(id: string): THREE.Mesh | undefined {
  return this.meshes.get(id);
}
```

`sync` makes the scene match a list from the API exactly: remove meshes whose id is gone (disposing their geometry and material, since GPU memory isn't garbage-collected), add new ones, update the rest. This is the same idea as React's reconciliation, by hand: the API's list is the state, and the scene is redrawn to match it.

A geometry is rebuilt only when kind or size changed, tracked by a `geometryKey` string in `userData`. Moving or recoloring a shape, the common case, touches only its transform and material.

Selecting, picking and the ghost

src/shapes/ShapeLayer.ts · lines 117–145

```
/** Highlights one shape (or none) by making it glow slightly. */
select(id: string | null): void {
  this.selectedId = id;
  for (const [shapeId, mesh] of this.meshes) {
    mesh.material.emissive.set(shapeId === id ? 0x3b3b3b : 0x000000);
  }
}

/** The id of the nearest shape under the ray, if any. */
pick(raycaster: THREE.Raycaster): string | null {
  const hit = raycaster.intersectObjects([...this.meshes.values()], false)[0];
  return (hit?.object.userData.shapeId as string | undefined) ?? null;
}

showGhost(kind: ShapeKind, size: Size, color: string, x: number, z: number, rotationY: number):
void {
  const key = geometryKey(kind, size);
  if (key !== this.ghostKey) {
    this.ghost.geometry.dispose();
    this.ghost.geometry = geometryFor(kind, size);
    this.ghostKey = key;
  }
  this.ghost.material.color.set(color);
  place(this.ghost, size, x, z, rotationY, this.supportAt(x, z));
  this.ghost.visible = true;
}

hideGhost(): void {
  this.ghost.visible = false;
}
```

Selection reuses the emissive glow the joint housings use. Picking raycasts only against shape meshes and reads the id from `userData`, as joint picking reads `jointIndex`.

Geometry for each kind

src/shapes/ShapeLayer.ts · lines 147–169

```
/** Three.js builds every shape centered on its origin; ours rest on the floor or a belt. */
function place(mesh: THREE.Object3D, size: Size, x: number, z: number, rotationY: number,
baseY: number) {
  mesh.position.set(x, baseY + size.y / 2, z);
  mesh.rotation.y = rotationY * DEG;
}

function geometryKey(kind: ShapeKind, s: Size): string {
  return `${kind}:${s.x}:${s.y}:${s.z}`;
}

function geometryFor(kind: ShapeKind, s: Size): THREE.BufferGeometry {
  switch (kind) {
    case "box":
      return new THREE.BoxGeometry(s.x, s.y, s.z);
    case "cylinder":
      return new THREE.CylinderGeometry(s.x / 2, s.x / 2, s.y, 40);
    case "cone":
      return new THREE.ConeGeometry(s.x / 2, s.y, 40);
    case "sphere":
      return new THREE.SphereGeometry(s.x / 2, 40, 20);
  }
}
```

Three.js builds geometry centered on the origin, so `place` lifts every shape by half its height, plus the height of whatever it stands on: 0 for the floor, a belt's top for a shape on a conveyor. `geometryFor` is a `switch` over the union of kinds; because TypeScript knows every case returns, adding a fifth kind without handling it here is a compile error.

PART 1.14 · `src/shapes/controller.ts`

The Shapes panel

The panel and the mouse placement, in one module that main.ts calls with three functions.

State

src/shapes/controller.ts · lines 1–72

```
/**
 * The Shapes panel: create shapes, or pick a part from the inventory, place them on the
 * floor with the mouse, move and delete them. Shapes are saved through the inventory API, so
they're in the database, and the
 * panel polls every few seconds so changes from another browser appear here too.
 *
 * main.ts owns the pointer and keyboard and asks this module first: handleClick and
 * handleKey return true when they used the event.
 */
import * as THREE from "three";
import type { ArmDimensions } from "../arm/kinematics";
import { shapesApi, ShapesApiError } from "./api";
import { reachOf, reachRadiusAt, REACH_LABEL, type Reach } from "./reach";
import { ShapeLayer } from "./ShapeLayer";
import { lookFor, placedBySku, type InventoryPart } from "./fromPart";
import { SHAPE_KINDS, type NewShape, type Shape, type ShapeKind, type Size } from "./types";

const POLL_MS = 4000;
const FLOOR_LIMIT = 3; // meters, same as the API
const DEFAULT_COLORS: Record<ShapeKind, string> = { box: "#f59e0b", cylinder: "#22c55e",
sphere: "#e11d48", cone: "#a855f7" };

type Mode = { kind: "idle" } | { kind: "placing" } | { kind: "moving"; id: string };

interface Draft {
  kind: ShapeKind;
  /** centimeters, as typed */
  w: number; h: number; d: number;
  color: string;
  name: string;
  rotationY: number;
  /** centimeters */
  x: number; z: number;
}

/**
 * What's under the floor plane's way: conveyor belts, provided by the conveyor controller.
 * support(x, z) is the height a shape at (x, z) stands on; pick() is where the mouse ray
```

```
 * hits the top of a belt, so you can place a shape on one.
 */
export interface Surface {
  support(x: number, z: number): number;
  pick(raycaster: THREE.Raycaster): { x: number; z: number } | null;
}

interface Options {
  root: HTMLElement;
  scene: THREE.Scene;
  camera: THREE.Camera;
  canvas: HTMLCanvasElement;
  arm: ArmDimensions;
}

export function createShapes({ root, scene, camera, canvas, arm }: Options) {
  const layer = new ShapeLayer(reachRadiusAt(0, arm));
  scene.add(layer.root);

  let shapes: Shape[] = [];
  let parts: InventoryPart[] = [];
  /** "basic": a plain shape; "part": a shape standing for an inventory part. */
  let source: "basic" | "part" = "basic";
  let partSku: string | null = null;
  let partSearch = "";
  let selected: string | null = null;
  let mode: Mode = { kind: "idle" };
  let online = false;
  let busy = false;
  let message: { text: string; error: boolean } | null = null;
  let hover: { x: number; z: number } | null = null;
  /** True while the belts run: shapes move on their own, so editing waits until Stop. */
  let running = false;
  let surface: Surface = { support: () => 0, pick: () => null };
  const reach = (s: Pick<Shape, "kind" | "size" | "position">) => reachOf(s, arm,
surface.support(s.position.x, s.position.z));
  const draft: Draft = { kind: "box", w: 20, h: 20, d: 20, color: DEFAULT_COLORS.box, name: "",
rotationY: 0, x: 60, z: 0 };
```

The panel's state is a handful of local variables inside `createShapes`, closed over by the functions below. `Mode` is a discriminated union: idle, placing a new shape, or moving the shape with a given id. A moving mode without an id can't be represented, so it can't happen.

Talking to the API

src/shapes/controller.ts · lines 74–149

```
// ---- Talking to the API ----

async function load() {
  try {
    const [list, stock, info] = await Promise.all([shapesApi.list(), shapesApi.parts(), online
? null : shapesApi.info()]);
    if (info) dbLabel.textContent = info.db;
    online = true;
    shapes = list;
    parts = stock;
    layer.sync(shapes);
    if (message?.error && message.text.startsWith("Can't reach")) message = null;
  } catch (err) {
    online = false;
    message = { text: (err as Error).message, error: true };
  }
  render();
}

async function act(work: () => Promise<unknown>, success?: string) {
  busy = true;
  render();
  try {
    await work();
    message = success ? { text: success, error: false } : null;
    await load();
  } catch (err) {
    const e = err as ShapesApiError;
    message = { text: e.problems?.length ? e.problems.join("; ") : e.message, error: true };
  } finally {
    busy = false;
    render();
  }
}
```

```
function sizeOf(d: Draft): Size {
  const cm = (n: number) => Math.round(n) / 100;
  if (d.kind === "sphere") return { x: cm(d.w), y: cm(d.w), z: cm(d.w) };
  if (d.kind === "cylinder" || d.kind === "cone") return { x: cm(d.w), y: cm(d.h), z: cm(d.w)
};
  return { x: cm(d.w), y: cm(d.h), z: cm(d.d) };
}

function newShape(x: number, z: number): NewShape {
  const count = shapes.filter((s) => s.kind === draft.kind).length + 1;
  const sku = source === "part" ? partSku : null;
  return {
    name: draft.name.trim() || `${draft.kind[0].toUpperCase()}${draft.kind.slice(1)} ${count}`,
    kind: draft.kind, size: sizeOf(draft), color: draft.color, position: { x, z }, rotationY:
draft.rotationY, sku,
  };
}

function create(x: number, z: number) {
  if (source === "part" && !partSku) {
    message = { text: "Pick a part from the inventory list first.", error: true };
    render();
    return;
  }
  const shape = newShape(x, z);
  void act(() => shapesApi.create(shape), `Added ${shape.name}${shape.sku ? ` (${shape.sku})` :
""}.`);
  if (source === "basic") draft.name = "";
}

/** Fills the form from a part: its look (kind, size, color) and its name. */
function pickPart(sku: string) {
  const part = parts.find((p) => p.sku === sku);
  if (!part) return;
  partSku = sku;
  const look = lookFor(part);
  const cm = (m: number) => Math.round(m * 1000) / 10;
  Object.assign(draft, { kind: look.kind, w: cm(look.size.x), h: cm(look.size.y), d:
cm(look.size.z), color: look.color, name: part.name.slice(0, 40) });
  render();
```

```
}

// Poll so changes from other browsers (or the API directly) show up. Skipped mid-placement.
setInterval(() => {
  if (mode.kind === "idle" && !busy && !running) void load();
}, POLL_MS);
```

`load` reads shapes and parts together with `Promise.all`. `act` wraps every change: mark busy, call the API, show the API's own problems if it refuses, then reload. Reloading after every change, rather than patching the local list, means the screen always shows what the database holds.

Sizes are typed in centimeters and sent in meters. `pickPart` fills the form from a part's look, so a part can still be resized or recolored before it's placed. The poll every 4 s is skipped mid-placement, so a refresh never moves the ghost under the mouse.

Placing with the mouse

src/shapes/controller.ts · lines 151–241

```
// ---- Pointer: placing and moving on the floor ----

const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
const floorPlane = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
const hitPoint = new THREE.Vector3();

function aim(e: PointerEvent) {
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
}

/**
 * Where the pointer meets the top of a belt, or else the floor, rounded to the centimeter;
 * null if off the floor area.
 */
function floorPoint(e: PointerEvent): { x: number; z: number } | null {
  aim(e);
  let at = surface.pick(raycaster);
  if (!at) {
    if (!raycaster.ray.intersectPlane(floorPlane, hitPoint)) return null;
    at = { x: hitPoint.x, z: hitPoint.z };
```

```
  }
  const x = Math.round(at.x * 100) / 100;
  const z = Math.round(at.z * 100) / 100;
  return Math.abs(x) <= FLOOR_LIMIT && Math.abs(z) <= FLOOR_LIMIT ? { x, z } : null;
}

function ghostSource(): { kind: ShapeKind; size: Size; color: string; rotationY: number } |
null {
  if (mode.kind === "placing") return { kind: draft.kind, size: sizeOf(draft), color:
draft.color, rotationY: draft.rotationY };
  if (mode.kind === "moving") {
    const id = mode.id;
    const s = shapes.find((x) => x.id === id);
    return s ? { kind: s.kind, size: s.size, color: s.color, rotationY: s.rotationY } : null;
  }
  return null;
}

/** Called on every pointer move. Returns true if the cursor is ours to set. */
function handleHover(e: PointerEvent): boolean {
  const src = ghostSource();
  if (src) {
    hover = floorPoint(e);
    if (hover) layer.showGhost(src.kind, src.size, src.color, hover.x, hover.z, src.rotationY);
    else layer.hideGhost();
    canvas.style.cursor = hover ? "crosshair" : "not-allowed";
    renderHint();
    return true;
  }
  aim(e);
  if (layer.pick(raycaster)) {
    canvas.style.cursor = "pointer";
    return true;
  }
  return false;
}

/** Called on a click (not a drag). Returns true if it was used. */
function handleClick(e: PointerEvent): boolean {
  if (mode.kind === "placing" || mode.kind === "moving") {
    const at = floorPoint(e);
```

```
    if (!at) return true;
    if (mode.kind === "placing") create(at.x, at.z);
    else {
      const id = mode.id;
      void act(() => shapesApi.update(id, { position: at }), "Moved.");
    }
    setMode({ kind: "idle" });
    return true;
  }
  aim(e);
  const id = layer.pick(raycaster);
  if (id) {
    setSelected(id === selected ? null : id);
    return true;
  }
  return false;
}

function handleKey(e: KeyboardEvent): boolean {
  if (e.key === "Escape" && mode.kind !== "idle") {
    setMode({ kind: "idle" });
    return true;
  }
  if ((e.key === "Delete" || e.key === "Backspace") && selected && mode.kind === "idle" &&
!running) {
    remove(selected);
    return true;
  }
  return false;
}
```

Placing needs the point where the mouse ray meets the floor. `Ray.intersectPlane` with the plane y = 0 gives it directly, with no mesh to raycast against, and it works even where the floor mesh doesn't reach. The point is rounded to the centimeter so typed and placed positions look alike.

`handleClick` and `handleKey` return true when they used the event, the contract main.ts relies on to decide whether the arm gets it.

The panel's HTML and events

src/shapes/controller.ts · lines 268–311

```
// ---- The panel ----

root.innerHTML = `
  <header>
    <h1>Shapes</h1>
    <p class="sub">Saved in the inventory database · <span class="db">connecting…</span></p>
  </header>
  <section>
    <h2>Add to the scene</h2>
    <div class="sources" role="group" aria-label="What to add">
      <button type="button" data-source="basic">Basic shape</button>
      <button type="button" data-source="part">Inventory part</button>
    </div>
    <div class="part-picker" hidden>
      <input type="search" class="part-search" placeholder="Search SKU or name" aria-
label="Search inventory parts">
      <ul class="part-list" role="listbox" aria-label="Inventory parts"></ul>
    </div>
    <div class="kinds" role="group" aria-label="Kind">
      ${SHAPE_KINDS.map((k) => `<button type="button" data-kind="${k}">${k}
</button>`).join("")}
    </div>
    <div class="dims"></div>
    <div class="row">
      <label>Color <input type="color" data-field="color"></label>
      <label>Turn ° <input type="number" data-field="rotationY" min="-360" max="360" step="5">
</label>
    </div>
    <label class="wide">Name <input type="text" data-field="name" maxlength="40"
placeholder="Automatic"></label>
    <div class="row">
      <label>X cm <input type="number" data-field="x" min="-300" max="300" step="5"></label>
      <label>Z cm <input type="number" data-field="z" min="-300" max="300" step="5"></label>
    </div>
    <p class="reach-line">At X, Z: <span class="reach"></span></p>
    <div class="actions">
      <button type="button" data-action="place" class="primary">Place with mouse</button>
      <button type="button" data-action="add">Add at X, Z</button>
```

```
    </div>
    <p class="hint" hidden></p>
  </section>
  <section>
    <h2>In the scene <span class="count"></span></h2>
    <ul class="shape-list"></ul>
  </section>
  <section class="points-host" hidden></section>
  <p class="shapes-status" role="status" hidden></p>
`;
```

The panel is one template string, built once. Events are attached to the panel root and dispatched by `data-` attributes (**event delegation**), so the list can be re-rendered freely without re-attaching handlers to new buttons.

Rendering

src/shapes/controller.ts · lines 381–458

```
function renderParts() {
  const placed = placedBySku(shapes);
  const shown = parts.filter((p) => !partSearch || p.sku.toLowerCase().includes(partSearch) ||
p.name.toLowerCase().includes(partSearch));
  partList.innerHTML = shown.length === 0
    ? `<li class="empty">${parts.length ? "No parts match." : "No parts in the inventory yet."}
</li>`
    : shown.map((p) => {
        const inScene = placed.get(p.sku) ?? 0;
        return `<li role="option" data-sku="${p.sku}" aria-selected="${p.sku === partSku}"
class="${p.sku === partSku ? "on" : ""}">
          <span class="sku">${p.sku}</span>
          <span class="pname">${escapeHtml(p.name)}</span>
          <span class="stock ${inScene > p.onHand ? "over" : ""}">${p.available} avail${inScene
? ` · ${inScene} in scene` : ""}</span>
        </li>`;
      }).join("");
}

function render() {
  root.querySelectorAll<HTMLButtonElement>("[data-source]").forEach((b) =>
b.setAttribute("aria-pressed", String(b.dataset.source === source)));
  picker.hidden = source !== "part";
```

```
  kinds.hidden = source === "part"; // a part's kind comes from its category
  if (source === "part") renderParts();
  root.querySelectorAll<HTMLButtonElement>("[data-kind]").forEach((b) => b.setAttribute("aria-
pressed", String(b.dataset.kind === draft.kind)));
  if (renderedKind !== draft.kind || dims.dataset.for !== `${draft.w}:${draft.h}:${draft.d}` &&
!dims.contains(document.activeElement)) {
    // Rebuild only when the kind changes, so typing in a field doesn't lose focus.
    dims.innerHTML =
      draft.kind === "box" ? dimInput("Width cm", "w") + dimInput("Height cm", "h") +
dimInput("Depth cm", "d")
      : draft.kind === "sphere" ? dimInput("Diameter cm", "w")
      : dimInput("Diameter cm", "w") + dimInput("Height cm", "h");
    renderedKind = draft.kind;
    dims.dataset.for = `${draft.w}:${draft.h}:${draft.d}`;
  }
  for (const key of ["color", "rotationY", "name", "x", "z"] as const) {
    const input = field(key);
    if (document.activeElement !== input) input.value = String(draft[key]);
  }

  placeBtn.textContent = mode.kind === "placing" ? "Cancel placing" : "Place with mouse";
  placeBtn.setAttribute("aria-pressed", String(mode.kind === "placing"));
  placeBtn.disabled = addBtn.disabled = !online || busy || running || (source === "part" &&
!partSku);
  if (!online) dbLabel.textContent = "offline";

  // Items stored in bins are listed with their bin (Bins tab), not here.
  const stored = new Set(shapes.filter((s) => s.binId).map((s) => s.id));
  const isStored = (s: Shape): boolean => stored.has(s.id) || (!!s.attachedTo &&
!!shapes.find((x) => x.id === s.attachedTo && isStored(x)));
  const visible = shapes.filter((s) => !isStored(s));
  list.innerHTML = visible.length === 0
    ? `<li class="empty">${online ? "No shapes yet. Place one with the mouse." : "Start the
inventory API to add shapes."}</li>`
    : visible.map((s) => {
        const where = reach(s);
        const part = s.sku ? parts.find((p) => p.sku === s.sku) : undefined;
        const placed = s.sku ? placedBySku(shapes).get(s.sku) ?? 0 : 0;
        const stockNote = part
          ? `<small class="${placed > part.onHand ? "over" : ""}">${s.sku} · ${placed} in
scene, ${part.onHand} on hand</small>`
```

```
          : s.sku ? `<small class="over">${s.sku} · no longer in the inventory</small>` : "";
        const moving = mode.kind === "moving" && mode.id === s.id;
        const base = s.attachedTo ? shapes.find((x) => x.id === s.attachedTo) : undefined;
        const joined = base ? `<small class="joined">joined onto ${escapeHtml(base.name)}
</small>` : "";
        // A joined shape moves with its base, so it has no Move of its own.
        const moveBtn = base ? "" : `<button type="button" data-id="${s.id}" data-do="move"
aria-pressed="${moving}" ${running ? "disabled" : ""} title="Click the floor to move
it">${moving ? "Cancel" : "Move"}</button>`;
        return `<li class="${s.id === selected ? "selected" : ""}">
          <button type="button" class="pick" data-id="${s.id}" aria-pressed="${s.id ===
selected}">
            <span class="swatch" style="background:${s.color}"></span>
            <span class="label"><strong>${escapeHtml(s.name)}</strong><small>${s.kind} ·
${sizeText(s)}</small>${stockNote}${joined}
              <span class="pill ${where}">${REACH_LABEL[where]}</span></span>
          </button>
          <span class="row-actions">
            ${moveBtn}
            <button type="button" data-id="${s.id}" data-do="delete" ${running ? "disabled" :
""} title="Delete (or press Delete)">x</button>
          </span>
        </li>`;
      }).join("");
  $("span.count").textContent = visible.length ? `(${visible.length})` : "";

  status.hidden = !message;
  status.textContent = message?.text ?? "";
  status.classList.toggle("error", !!message?.error);
  renderHint();
}
```

`render` updates everything from state, with one subtlety: inputs aren't overwritten while focused, and the size fields are rebuilt only when the kind or the values change from outside, so typing never loses focus or the caret. User text (a shape's name) goes through `escapeHtml` before entering `innerHTML`; a name like `<img onerror=…>` stays text.

Each shape row shows its reach, and for parts the count in the scene against stock on hand, in orange when the scene uses more than the shelf holds.

The API client

src/shapes/api.ts · lines 1–47

```
/**
 * Talks to the inventory API (Snack 5), which stores the shapes in PostgreSQL or MongoDB.
 * The API allows calls from pages on localhost (CORS), so this dev server can use it directly.
 */
import type { InventoryPart } from "./fromPart";
import type { NewShape, Shape, ShapePatch } from "./types";
import type { Conveyor, ConveyorPatch, NewConveyor } from "../conveyors/types";
import type { NewSplitter, Splitter, SplitterPatch } from "../splitters/types";
import type { Part } from "../splitters/rules";
import type { Bin, BinPatch, ItemLook, NewBin } from "../line/types";
import type { Assembly, ConnectionPoint, Connectors, Machine, MachinePatch, Merger,
MergerPatch, NewAssembly, NewMachine, NewMerger, Owner } from "../line/types";

/** Override with VITE_INVENTORY_API, e.g. http://localhost:8080 for the API without Docker. */
export const API_BASE: string = (import.meta.env.VITE_INVENTORY_API as string | undefined) ??
"http://localhost:8081";

export class ShapesApiError extends Error {
  readonly problems: string[];
  constructor(message: string, problems: string[] = []) {
    super(message);
    this.problems = problems;
  }
}

async function call<T>(method: string, path: string, body?: unknown): Promise<T> {
  let res: Response;
  try {
    res = await fetch(`${API_BASE}/api${path}`, {
      method,
      headers: body === undefined ? undefined : { "content-type": "application/json" },
      body: body === undefined ? undefined : JSON.stringify(body),
    });
  } catch {
    throw new ShapesApiError(`Can't reach the inventory API at ${API_BASE}. Is Snack 5 running?
`);
  }
  const data = (await res.json().catch(() => ({}))) as { error?: string; problems?: string[] };
```

```
  if (!res.ok) throw new ShapesApiError(data.error ?? res.statusText, data.problems ?? []);
  return data as T;
}

export const shapesApi = {
  info: () => call<{ db: string; host: string }>("GET", "/info"),
  list: () => call<Shape[]>("GET", "/shapes"),
  parts: () => call<InventoryPart[]>("GET", "/parts"),
  create: (shape: NewShape) => call<Shape>("POST", "/shapes", shape),
  update: (id: string, patch: ShapePatch) => call<Shape>("PATCH", `/shapes/${id}`, patch),
  remove: (id: string) => call<{ deleted: string }>("DELETE", `/shapes/${id}`),
};
```

The base URL defaults to the Docker setup (nginx on 8081) and can be overridden with a Vite environment variable. A failed `fetch` (nothing listening) becomes a readable message, and the API's list of problems is kept for display. This works across origins because the API sends CORS headers for localhost pages (Chapter 5).

PART 1.15 · `src/conveyors/transport.ts`

Conveyor belts: how a belt carries things

You can build conveyor belts in the scene, and shapes resting on one ride along when the line runs. This part is the motion: plain math with no Three.js, so it's unit-tested like the kinematics.

What a belt is

src/conveyors/types.ts · lines 1–21

```
/** Conveyor belts as the inventory API stores them (snack5-
inventory/api/src/domain/conveyors.ts). */
export interface Conveyor {
  id: string;
  name: string;
  /** Meters along the direction of travel. */
  length: number;
  /** Meters across. */
  width: number;
  /** Height of the belt surface, meters. */
  height: number;
  /** Meters per second along the belt's own +x; negative runs it backwards. */
  speed: number;
  position: { x: number; z: number };
  /** Degrees about the vertical axis. */
  rotationY: number;
  createdAt: string;
  updatedAt: string;
}

export type NewConveyor = Omit<Conveyor, "id" | "createdAt" | "updatedAt">;
export type ConveyorPatch = Partial<Pick<Conveyor, "name" | "position" | "rotationY" |
"speed">>;
```

A belt is a box on the floor. `height` is its top surface, which is what a shape stands on. `speed` is along the belt's own length, so turning a belt changes where things go without changing its speed. The type matches the API's field for field (Chapter 5 stores it). `ConveyorPatch` leaves out the size: a belt of a different size is a different belt, as with shapes.

The belt's own frame

src/conveyors/transport.ts · lines 1–48

```
/**
 * How belts carry things. Pure math with no Three.js, so it's unit-tested.
 *
 * The model is deliberately simple, a "kinematic" conveyor like the arm's own gripper:
 * - a shape rests on whatever is under its center: the highest belt top at or below it, or the
floor;
```

```
 * - resting on a running belt, it moves at the belt's speed along the belt;
 * - with nothing under it any more (it rode off the end), it falls with gravity, keeping the
 *   belt's forward speed like a thrown ball, until it lands on the floor (and stops) or on a
 *   lower belt (which carries it on).
 * - passing a splitter's gate, it's sent left, right or straight on by the splitter's rules
 *   (decided by the caller); sent sideways, it slides off the belt's side at an angle.
 * Shapes don't collide with each other or with a belt's sides. Snack 3 is where real contact
 * physics lives; here the point is layout and flow.
 */
import type { Conveyor } from "./types";

type Belt = Pick<Conveyor, "length" | "width" | "height" | "speed" | "position" | "rotationY">
& { id?: string };

/** A splitter's gate: on which belt, and how far from that belt's start (its -u end). */
export interface Gate {
  id: string;
  conveyorId: string;
  along: number;
}
export type Exit = "left" | "straight" | "right";

/** A diverted item leaves at this angle to the belt's direction of travel. */
export const DIVERT_ANGLE = 60 * (Math.PI / 180);

export const GRAVITY = 9.81; // m/s²
const DEG = Math.PI / 180;
const EPS = 1e-6;

/**
 * A world point in the belt's own frame: u along its length (direction of travel), v across.
 * Three.js turns local +x to (cos θ, 0, -sin θ) for a rotation θ about +y, so this is the
inverse.
 */
export function toBelt(b: Belt, x: number, z: number): { u: number; v: number } {
  const t = b.rotationY * DEG;
  const dx = x - b.position.x;
  const dz = z - b.position.z;
  return { u: dx * Math.cos(t) - dz * Math.sin(t), v: dx * Math.sin(t) + dz * Math.cos(t) };
}
```

```
export function isOver(b: Belt, x: number, z: number): boolean {
  const { u, v } = toBelt(b, x, z);
  return Math.abs(u) <= b.length / 2 + EPS && Math.abs(v) <= b.width / 2 + EPS;
}
```

A turned belt is awkward in world coordinates and trivial in its own. `toBelt` subtracts the belt's position and rotates by −θ, giving `u` along the belt and `v` across it (Figure 1.6). In that frame every belt is an axis-aligned rectangle, so “is this point on the belt?” is two comparisons.

The sign convention is the one subtle part. Three.js turns an object's +x to (cos θ, 0, −sin θ) when `rotation.y = θ`, because +y points up and positive angles turn counter-clockwise seen from above. The math copies that exactly, so the belt drawn by Three.js and the belt the math moves things along are the same belt. The first test pins it down: a belt turned 90° runs toward −z.

`EPS` lets a point exactly on an edge count as on the belt. Without it, rounding (0.15 computed as 0.15000000000000002) would make edge cases depend on luck.

What holds a shape up

src/conveyors/transport.ts · lines 50–68

```
/** The highest belt under (x, z) whose top is at or below maxY, if any. */
export function beltUnder<B extends Belt>(belts: readonly B[], x: number, z: number, maxY =
Infinity): B | null {
  let best: B | null = null;
  for (const b of belts) {
    if (b.height <= maxY + EPS && isOver(b, x, z) && (!best || b.height > best.height)) best =
b;
  }
  return best;
}

/** What a shape at (x, z) rests on: the top of the highest belt at or below maxY, or the
floor. */
export function supportHeight(belts: readonly Belt[], x: number, z: number, maxY = Infinity):
number {
  return beltUnder(belts, x, z, maxY)?.height ?? 0;
}

/** World velocity of a belt's surface. */
export function beltVelocity(b: Belt): { vx: number; vz: number } {
  const t = b.rotationY * DEG;
  return { vx: b.speed * Math.cos(t), vz: -b.speed * Math.sin(t) };
}
```

`beltUnder` returns the highest belt under a point, but only among belts whose top is at or below `maxY`. That limit is what makes belts at different heights behave: a shape falling past a high belt's end shouldn't be caught by it, and a flat plate on the floor under a raised belt shouldn't jump up onto it. The generic `<B extends Belt>` returns the caller's own type, so the controller gets a full `Conveyor` back, not just the fields this module reads.

`beltVelocity` is the speed along `u`, turned into world x and z with the same cos/−sin pair as `toBelt`.

One step of motion

src/conveyors/transport.ts · lines 70–91

```
/** A shape while the belts run: where it is, and how fast it's moving. */
export interface Body {
  /** The shape's id, passed back to the caller when the body reaches a gate. */
  id?: string;
  /** Set when a gate sent it sideways: which belt, and which side of it (+1 is the belt's +v
side). */
  divert?: { beltId: string | undefined; side: 1 | -1 };
  x: number;
  z: number;
  /** Height of the shape's base. */
  y: number;
  /** Velocity, m/s: vx, vz while falling off a belt, vy negative while falling. */
  vx: number;
  vy: number;
  vz: number;
  /** Its length along a belt, meters, for spacing in a queue. Defaults to 0.2. */
  size?: number;
  /** Set while it's held at a stop line (that line's id) or queued behind something held
("queue"). */
  heldAt?: string;
  /** Set when a station slides it somewhere, such as onto a main belt at a merge: target and
speed. */
  guide?: { x: number; z: number; y: number; speed: number };
}
```

src/conveyors/transport.ts · lines 141–257

```
/** Advances every body by dt seconds. Bodies are changed in place. */
export function step(bodies: Iterable<Body>, belts: readonly Belt[], dt: number, options:
StepOptions | number = {}): void {
  const { gates = [], route, stops = [], hold, floorLimit = 2.95 } = typeof options ===
"number" ? { floorLimit: options } : options;
  const all = [...bodies];
  for (const body of all) {
    if (body.guide) {
      // Slid by a station: straight to the target at the guide's speed, then let go on
whatever is there.
      const g = body.guide;
      const dx = g.x - body.x;
```

```
      const dz = g.z - body.z;
      const d = Math.hypot(dx, dz);
      body.y = g.y;
      body.vx = body.vy = body.vz = 0;
      if (d <= g.speed * dt) {
        body.x = g.x;
        body.z = g.z;
        body.guide = undefined;
      } else {
        body.x += (dx / d) * g.speed * dt;
        body.z += (dz / d) * g.speed * dt;
      }
      continue;
    }
    // Only what's at or below the base can hold it up: a belt above it doesn't lift it.
    const support = supportHeight(belts, body.x, body.z, body.y);
    if (body.y > support + 1e-4) {
      // Semi-implicit Euler, as in Snack 3: velocity first, then position.
      body.vy -= GRAVITY * dt;
      body.y += body.vy * dt;
      move(body, body.vx, body.vz, dt, floorLimit);
      if (body.y <= support) {
        // Landed. What it landed on is found again next step, since the fall moved it.
        body.y = supportHeight(belts, body.x, body.z, support);
        body.vx = body.vy = body.vz = 0;
        body.divert = undefined;
      }
      continue;
    }
    body.y = support;
    body.vy = 0;
    const belt = beltUnder(belts, body.x, body.z, body.y);
    if (!belt || Math.abs(belt.height - body.y) > 1e-4) {
      body.vx = body.vz = 0; // resting on the floor
      body.divert = undefined;
      continue;
    }
    // Sent sideways by a gate on this belt: keep sliding off. On another belt, ride it
normally.
    const diverted = body.divert && body.divert.beltId === belt.id ? body.divert : undefined;
    if (!diverted) body.divert = undefined;
```

```
    const v = diverted ? divertVelocity(belt, diverted.side) : beltVelocity(belt);
    const start = toBelt(belt, body.x, body.z);
    const before = start.u;
    body.vx = v.vx;
    body.vz = v.vz;
    move(body, v.vx, v.vz, dt, floorLimit);
    if (diverted || belt.id === undefined) continue;
    let after = toBelt(belt, body.x, body.z).u;
    const dir = belt.speed >= 0 ? 1 : -1;

    // Stop lines: a station may hold the item at the line (it's put back exactly on it).
    let held: string | undefined;
    if (hold) {
      for (const stop of stops) {
        if (stop.conveyorId !== belt.id) continue;
        const g = stop.along - belt.length / 2;
        if (Math.min(before, after) <= g && g <= Math.max(before, after) && before !== after &&
hold(body, stop)) {
          after = g;
          held = stop.id;
          break;
        }
      }
    }
    // Queues: don't pass through something held at a queueing line, or queued behind one.
    if (!held) {
      const queueing = new Set(stops.filter((s) => s.queue && s.conveyorId === belt.id).map((s)
=> s.id));
      for (const other of all) {
        if (other === body || !other.heldAt || (other.heldAt !== "queue" &&
!queueing.has(other.heldAt))) continue;
        if (!isOver(belt, other.x, other.z) || Math.abs(other.y - body.y) > 1e-3) continue;
        const ou = toBelt(belt, other.x, other.z).u;
        const limit = ou - dir * ((body.size ?? 0.2) + (other.size ?? 0.2)) / 2 - dir *
QUEUE_GAP;
        // Only what's ahead of it counts, and only if this step would take it past the limit.
        if ((ou - before) * dir > 0 && (after - limit) * dir > 0) {
          after = (before - limit) * dir > 0 ? before : limit;
          held = "queue";
        }
      }
```

```
    }
    body.heldAt = held;
    if (held) {
      const p = fromBelt(belt, after, start.v);
      body.x = p.x;
      body.z = p.z;
      body.vx = body.vz = 0;
      continue;
    }
    if (!route) continue;

    // Did this step carry it past a gate on this belt? Gates are measured from the belt's -u
end.
    for (const gate of gates) {
      if (gate.conveyorId !== belt.id) continue;
      const g = gate.along - belt.length / 2;
      const crossed = (before < g && after >= g) || (before > g && after <= g);
      if (!crossed) continue;
      const exit = route(body, gate);
      if (exit !== "straight") {
        body.divert = { beltId: belt.id, side: sideOf(exit, belt.speed) };
        break;
      }
    }
  }
}

/** Stop at the edge of the stored floor area rather than leaving it. */
function move(body: Body, vx: number, vz: number, dt: number, limit: number): void {
  body.x = Math.max(-limit, Math.min(limit, body.x + vx * dt));
  body.z = Math.max(-limit, Math.min(limit, body.z + vz * dt));
}
```

Every shape is a `Body`: where its base is, and its velocity. Each step does one of three things. (The `divert` field and the gate check at the end of the loop belong to splitters; `heldAt`, `guide`, the stop lines and the queues belong to mergers and machines. Each is covered in its own part below.)

1. **In the air** (above whatever is under it): fall. Velocity first, then position, the same semi-implicit Euler as Snack 3's conveyor physics. Forward speed is kept, so an item leaving a belt flies on like a thrown ball (Figure 1.7). When it reaches what's below, it lands and stops.
2. **On the floor:** nothing. The floor doesn't move things.

3. **On a belt:** move at the belt's surface velocity, and remember it as the body's velocity in case the next step finds nothing underneath.

This is a **kinematic** model: belts set velocities directly, with no forces, friction or collisions. Shapes pass through each other and through side rails. That's a deliberate trade: the point here is layout and flow (where does each item end up?), and it keeps the step to forty lines that can be tested exactly. Snack 3 is where contact physics lives.

`move` clamps to ±2.95 m, inside the API's ±3 m floor limit, so saving a position after a run can never be refused.

Testing the belts

tests/transport.test.ts · lines 1–52

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { beltUnder, beltVelocity, step, supportHeight, toBelt, type Body } from
"../src/conveyors/transport";

const belt = (over: Partial<{ length: number; width: number; height: number; speed: number; x:
number; z: number; rotationY: number }> = {}) => {
  const b = { length: 1, width: 0.3, height: 0.2, speed: 0.5, x: 0, z: 0, rotationY: 0, ...over
};
  return { length: b.length, width: b.width, height: b.height, speed: b.speed, position: { x:
b.x, z: b.z }, rotationY: b.rotationY };
};
const close = (a: number, b: number, tol = 1e-9) => assert.ok(Math.abs(a - b) <= tol, `${a} vs
${b}`);

function run(bodies: Body[], belts: ReturnType<typeof belt>[], seconds: number, dt = 1 / 120) {
  for (let t = 0; t < seconds - 1e-9; t += dt) step(bodies, belts, dt);
}

test("a belt's frame: u along its length, v across, following its rotation", () => {
  const b = belt({ x: 1, z: 1, rotationY: 90 }); // turned 90°: travels toward -z
  const p = toBelt(b, 1, 0.6); // 0.4 m in the direction of travel
  close(p.u, 0.4);
  close(p.v, 0);
  const v = beltVelocity(b);
  close(v.vx, 0);
  close(v.vz, -0.5);
});
```

```
test("support: the highest belt under a point, ignoring belts above a given height", () => {
  const low = belt({ height: 0.2 });
  const high = belt({ height: 0.5, width: 0.1 });
  assert.equal(supportHeight([low, high], 0, 0), 0.5);
  assert.equal(supportHeight([low, high], 0, 0.1), 0.2); // outside the narrow high belt
  assert.equal(supportHeight([low, high], 0, 0, 0.3), 0.2); // a body at 0.3 m sits on the low
one
  assert.equal(supportHeight([low, high], 2, 0), 0); // floor
  assert.equal(beltUnder([low], 0.5, 0.15), low); // edges count
});

test("a shape on a running belt rides at the belt's speed", () => {
  const b = belt({ length: 2 }); // 0.5 m/s along +x, from x = -1 to 1
  const body: Body = { x: -0.8, z: 0, y: 0.2, vx: 0, vy: 0, vz: 0 };
  run([body], [b], 1);
  close(body.x, -0.3, 1e-6);
  close(body.y, 0.2);
});

test("it falls off the end, keeping its forward speed, and stops on the floor", () => {
  const b = belt({ length: 1, height: 0.5, speed: 0.5 });
  const body: Body = { x: 0.3, z: 0, y: 0.5, vx: 0, vy: 0, vz: 0 };
  run([body], [b], 3);
  assert.equal(body.y, 0);
  // A 0.5 m fall takes sqrt(2·0.5/9.81) = 0.32 s, at 0.5 m/s forward: lands about 0.16 m past
the end.
  close(body.x, 0.5 + 0.5 * Math.sqrt(1 / 9.81), 0.01);
  assert.deepEqual([body.vx, body.vz], [0, 0]);
});
```

Each test sets up belts, steps the simulation at 120 Hz, and checks where things end up. The fall-off test checks the landing point against the projectile formula, `end + speed · √(2h/g)`, to within a centimeter: that's the test that caught the first version of `step`, which dropped items straight down and so could never hand them to another belt.

tests/transport.test.ts · lines 53–92

```
test("falling takes the time gravity says", () => {
  const body: Body = { x: 5, z: 0, y: 0.4905, vx: 0, vy: 0, vz: 0 }; // h = g t²/2 with t =
0.316 s
  let t = 0;
```

```
  while (body.y > 0) {
    step([body], [], 1 / 1000);
    t += 1 / 1000;
  }
  close(t, Math.sqrt((2 * 0.4905) / 9.81), 0.003);
});

test("a higher belt hands over to a lower one, which carries it on", () => {
  const upper = belt({ x: 0, length: 1, height: 0.4, speed: 0.4 });
  const lower = belt({ x: 0.6, length: 1, height: 0.15, speed: 0.4, rotationY: 90 }); // runs
toward -z
  const body: Body = { x: 0, z: 0, y: 0.4, vx: 0, vy: 0, vz: 0 };
  run([body], [upper, lower], 2.5);
  close(body.y, 0.15);
  assert.ok(body.z < -0.3, `carried along the lower belt to z = ${body.z}`);
});

test("shapes on the floor, or under a belt that's above them, don't move", () => {
  const b = belt({ height: 0.5 });
  const onFloor: Body = { x: 2, z: 0, y: 0, vx: 0, vy: 0, vz: 0 };
  const under: Body = { x: 0, z: 0, y: 0, vx: 0, vy: 0, vz: 0 }; // e.g. a flat plate beneath a
high belt
  run([onFloor, under], [b], 1);
  assert.deepEqual([onFloor.x, under.x, under.y], [2, 0, 0]);
});

test("a stopped belt holds things in place, and a reversed one runs backwards", () => {
  const body: Body = { x: 0, z: 0, y: 0.2, vx: 0, vy: 0, vz: 0 };
  run([body], [belt({ speed: 0 })], 1);
  assert.equal(body.x, 0);
  run([body], [belt({ speed: -0.2 })], 1);
  close(body.x, -0.2, 1e-6);
});

test("carried shapes stop at the edge of the floor area", () => {
  const body: Body = { x: 2.8, z: 0, y: 0.2, vx: 0, vy: 0, vz: 0 };
  run([body], [belt({ x: 2.5, length: 1.5, speed: 1 })], 1);
  assert.equal(body.x, 2.95);
});
```

The gravity test steps at 1 ms and compares the fall time with √(2h/g). The hand-over test is the reason the model exists: a high belt feeding a lower belt turned 90°, with the item arriving on the second belt and travelling along it toward −z.

PART 1.16 · `src/conveyors/ConveyorLayer.ts`

Drawing the belts

Belt models built from boxes, a moving chevron texture, and a see-through preview for placing.

A model in the belt's frame

src/conveyors/ConveyorLayer.ts · lines 1–33

```
import * as THREE from "three";
import { DEG } from "../arm/kinematics";
import type { Conveyor } from "./types";

/** One chevron per this many meters of belt. */
const CHEVRON_PITCH = 0.25;
const FRAME_DEPTH = 0.06; // the belt bed under the surface
const RAIL = 0.015; // side rail thickness and height above the belt

type Belt = Pick<Conveyor, "length" | "width" | "height">;

/**
 * Draws the conveyors. Each belt is a group in its own frame: x along its length (the way
 * it runs at a positive speed), z across, y up from the floor. The group's rotation about y
 * is the stored rotationY, which is the same convention transport.ts uses, so what you see
 * and what moves the shapes agree.
 *
 * Like ShapeLayer, sync() makes the scene match the API's list, keyed by id.
 */
export class ConveyorLayer {
  readonly root = new THREE.Group();
  private readonly belts = new Map<string, { group: THREE.Group; key: string; surface:
THREE.Mesh; texture: THREE.CanvasTexture; frame: THREE.MeshStandardMaterial; speed: number }>
();
  private readonly ghost = new THREE.Group();
  private ghostKey = "";
  private readonly chevrons = chevronCanvas();
```

```
  private selectedId: string | null = null;

  constructor() {
    this.root.name = "Conveyors";
    this.ghost.visible = false;
    this.root.add(this.ghost);
  }
```

Each belt is a `Group` whose children are built in the belt's own frame: x along, z across, y up from the floor. The group's `rotation.y` is the stored angle, so a belt's model and its motion share one convention (previous part).

src/conveyors/ConveyorLayer.ts · lines 120–158

```
/** The model: a bed, the moving surface on top, side rails, end rollers and legs. */
function buildBelt(b: Belt, frame: THREE.Material, belt: THREE.Material): { group: THREE.Group;
surface: THREE.Mesh } {
  const group = new THREE.Group();
  const add = (geometry: THREE.BufferGeometry, material: THREE.Material, x: number, y: number,
z: number) => {
    const mesh = new THREE.Mesh(geometry, material);
    mesh.position.set(x, y, z);
    mesh.castShadow = true;
    mesh.receiveShadow = true;
    group.add(mesh);
    return mesh;
  };

  const bed = Math.min(FRAME_DEPTH, b.height);
  add(new THREE.BoxGeometry(b.length, bed, b.width), frame, 0, b.height - bed / 2, 0);

  // The surface is a flat plane a hair above the bed, facing up. Its UVs run along x, so
  // the chevron texture repeats along the belt's length.
  const surface = add(new THREE.PlaneGeometry(b.length, b.width).rotateX(-Math.PI / 2), belt,
0, b.height + 0.001, 0);
  surface.castShadow = false;

  for (const side of [-1, 1]) {
    add(new THREE.BoxGeometry(b.length, RAIL * 2, RAIL), frame, 0, b.height + RAIL / 2, side *
(b.width / 2 + RAIL / 2));
  }
  for (const end of [-1, 1]) {
```

```
    const roller = add(new THREE.CylinderGeometry(bed / 2, bed / 2, b.width,
20).rotateX(Math.PI / 2), frame, end * (b.length / 2), b.height - bed / 2, 0);
    roller.castShadow = false;
  }

  const legHeight = b.height - bed;
  if (legHeight > 0.01) {
    const inset = Math.min(0.1, b.length / 4);
    for (const end of [-1, 1]) {
      for (const side of [-1, 1]) {
        add(new THREE.BoxGeometry(0.03, legHeight, 0.03), frame, end * (b.length / 2 - inset),
legHeight / 2, side * (b.width / 2 - 0.02));
      }
    }
  }
  return { group, surface };
}
```

The bed is at most 6 cm deep, hanging below the surface. Below that, legs run to the floor if there's room, so a 5 cm belt is a flat slab and a 1 m belt stands on four legs. The moving surface is a separate plane a millimeter above the bed: it carries the texture, and it's the mesh the mouse ray tests when you place a shape on a belt.

Sync, and the moving chevrons

src/conveyors/ConveyorLayer.ts · lines 34–79

```
sync(list: readonly Conveyor[]): void {
  const keep = new Set(list.map((b) => b.id));
  for (const [id, belt] of this.belts) {
    if (!keep.has(id)) {
      this.root.remove(belt.group);
      disposeTree(belt.group);
      belt.texture.dispose();
      this.belts.delete(id);
    }
  }
  for (const b of list) {
    const key = `${b.length}:${b.width}:${b.height}`;
    let belt = this.belts.get(b.id);
    if (belt && belt.key !== key) {
      // A different size is a different model; rebuild it (sizes don't change from this app
```

```
today).
      this.root.remove(belt.group);
      disposeTree(belt.group);
      belt.texture.dispose();
      belt = undefined;
    }
    if (!belt) {
      const texture = new THREE.CanvasTexture(this.chevrons);
      texture.wrapS = THREE.RepeatWrapping;
      texture.repeat.set(b.length / CHEVRON_PITCH, 1);
      texture.colorSpace = THREE.SRGBColorSpace;
      const frame = new THREE.MeshStandardMaterial({ color: 0x4b5563, roughness: 0.6,
metalness: 0.4 });
      const { group, surface } = buildBelt(b, frame, new THREE.MeshStandardMaterial({ map:
texture, roughness: 0.9 }));
      group.traverse((o) => (o.userData.beltId = b.id));
      belt = { group, key, surface, texture, frame, speed: b.speed };
      this.belts.set(b.id, belt);
      this.root.add(group);
    }
    belt.speed = b.speed;
    belt.group.name = b.name;
    belt.group.position.set(b.position.x, 0, b.position.z);
    belt.group.rotation.y = b.rotationY * DEG;
  }
  this.select(this.selectedId && keep.has(this.selectedId) ? this.selectedId : null);
}

/** Slides each belt's chevrons along at its speed. Call every frame while running. */
animate(dt: number): void {
  // With the texture repeated over the belt, moving the texture's offset by -d moves
  // the pattern +d·pitch along the belt.
  for (const belt of this.belts.values()) belt.texture.offset.x -= (belt.speed * dt) /
CHEVRON_PITCH;
}
```

`sync` reconciles meshes with the API's list, as ShapeLayer does. Changing a belt's angle or speed only touches the group's transform; a new size rebuilds the model.

The chevron texture is drawn once on a 64 × 64 canvas, and each belt gets a `CanvasTexture` on it with `RepeatWrapping` and `repeat.x = length / 0.25`: one chevron every 25 cm, whatever the belt's length. `animate` slides the texture's offset. A texture coordinate is `uv · repeat + offset`, so

decreasing the offset by d moves the pattern forward d / repeat of the belt, which is d · 0.25 m. Setting d = speed · dt / 0.25 makes the chevrons move at exactly the belt's speed, matching the shapes riding on it.

src/conveyors/ConveyorLayer.ts · lines 160–187

```
/** A dark belt with one light chevron pointing along +x (the direction of travel). */
function chevronCanvas(): HTMLCanvasElement {
  const c = document.createElement("canvas");
  c.width = 64;
  c.height = 64;
  const g = c.getContext("2d")!;
  g.fillStyle = "#1f2937";
  g.fillRect(0, 0, 64, 64);
  g.strokeStyle = "#9ca3af";
  g.lineWidth = 7;
  g.lineCap = "round";
  g.beginPath();
  g.moveTo(22, 10);
  g.lineTo(42, 32);
  g.lineTo(22, 54);
  g.stroke();
  return c;
}

function disposeTree(object: THREE.Object3D): void {
  object.traverse((o) => {
    if (o instanceof THREE.Mesh) {
      o.geometry.dispose();
      // The texture is disposed by the caller; materials are per belt.
      (Array.isArray(o.material) ? o.material : [o.material]).forEach((m: THREE.Material) =>
m.dispose());
    }
  });
}
```

A canvas texture costs nothing to download and scales to any belt. `disposeTree` releases each mesh's geometry and material when a belt is deleted, since GPU memory isn't garbage-collected.

Picking

src/conveyors/ConveyorLayer.ts · lines 80–118

```
  select(id: string | null): void {
    this.selectedId = id;
    for (const [beltId, belt] of this.belts) belt.frame.emissive.set(beltId === id ? 0x1e3a8a :
0x000000);
  }

  /** The belt under the ray, if any. */
  pick(raycaster: THREE.Raycaster): string | null {
    const hit = raycaster.intersectObjects([...this.belts.values()].map((b) => b.group), true)
[0];
    return (hit?.object.userData.beltId as string | undefined) ?? null;
  }

  /** Where the ray first hits a belt's top surface, for placing shapes on belts. */
  surfacePoint(raycaster: THREE.Raycaster): { x: number; z: number } | null {
    const hit = raycaster.intersectObjects([...this.belts.values()].map((b) => b.surface),
false)[0];
    return hit ? { x: hit.point.x, z: hit.point.z } : null;
  }

  showGhost(b: Belt, x: number, z: number, rotationY: number): void {
    const key = `${b.length}:${b.width}:${b.height}`;
    if (key !== this.ghostKey) {
      for (const child of [...this.ghost.children]) {
        this.ghost.remove(child);
        disposeTree(child);
      }
      const see = () => new THREE.MeshStandardMaterial({ color: 0x60a5fa, transparent: true,
opacity: 0.4, depthWrite: false });
      const { group } = buildBelt(b, see(), see());
      this.ghost.add(group);
      this.ghostKey = key;
    }
    this.ghost.position.set(x, 0, z);
    this.ghost.rotation.y = rotationY * DEG;
    this.ghost.visible = true;
  }
```

```
  hideGhost(): void {
    this.ghost.visible = false;
  }
}
```

`pick` raycasts against whole groups (the `true` argument recurses into children) and reads the belt id every child was given in `userData`. `surfacePoint` tests only the top planes, which is what the Shapes panel asks for: the point on a belt's top under the mouse, so a shape dropped there lands where the cursor is, not where the ray would have met the floor beneath.

PART 1.17 · `src/conveyors/controller.ts`

The Conveyors tab, and running the line

The panel that builds belts, and the Run / Stop loop that moves the shapes and saves where they end up.

State, and telling the shapes about belts

src/conveyors/controller.ts · lines 1–90

```
/**
 * The Conveyors tab: build belts, place them with the mouse, turn, reverse and delete them,
 * and Run / Stop the line. Belts are saved through the inventory API like the shapes.
 *
 * While running, transport.ts moves the shapes every frame. Nothing is saved until Stop,
 * when each shape's new position is written back with one PATCH, so a run is just a
 * preview until you stop it.
 */
import * as THREE from "three";
import { conveyorsApi, ShapesApiError } from "../shapes/api";
import type { createShapes } from "../shapes/controller";
import { ConveyorLayer } from "./ConveyorLayer";
import { step, supportHeight, type Body, type Exit, type Gate } from "./transport";
import { poses, rootOf } from "../line/joints";
import type { Join, Stations } from "../line/stations";
import type { Attachment } from "../line/types";
import type { Conveyor, NewConveyor } from "./types";

type Shapes = ReturnType<typeof createShapes>;
type Mode = { kind: "idle" } | { kind: "placing" } | { kind: "moving"; id: string };
```

```
const POLL_MS = 4000;
const FLOOR_LIMIT = 3;
/** Physics steps are at most this long, however slow the frame. */
const MAX_STEP = 1 / 120;

/** What the splitters tell the running line: where their gates are, and where each item goes.
*/
export interface Gates {
  list(): readonly Gate[];
  /** Asked once when a shape reaches a gate. */
  route(shapeId: string, gate: Gate): Exit;
}

/** What the mergers and machines plug in: their stations, and how to save a run's joins. */
export interface Line {
  stations: Stations;
  /** Called on Stop, after positions are saved, with the joins machines made during the run.
*/
  saveJoins(joins: readonly Join[]): Promise<void>;
  /** Called on Stop: items that left bins (first) and items that ended up in bins (last). */
  saveReleases(items: readonly { binId: string; shapeId: string; x: number; z: number }[]):
Promise<void>;
  saveCatches(items: readonly { binId: string; shapeId: string }[]): Promise<void>;
  /** Called every frame while running, after the step, to update the stations' 3D models. */
  frame(dt: number): void;
}

interface Draft {
  /** centimeters (speed in cm/s), as typed */
  length: number; width: number; height: number; speed: number;
  rotationY: number; name: string; x: number; z: number;
}

interface Options {
  root: HTMLElement;
  runButton: HTMLButtonElement;
  scene: THREE.Scene;
  camera: THREE.Camera;
  canvas: HTMLCanvasElement;
  shapes: Shapes;
```

```
}

export function createConveyors({ root, runButton, scene, camera, canvas, shapes }: Options) {
  const layer = new ConveyorLayer();
  scene.add(layer.root);

  let belts: Conveyor[] = [];
  let selected: string | null = null;
  let mode: Mode = { kind: "idle" };
  let online = false;
  let busy = false;
  let message: { text: string; error: boolean } | null = null;
  let hover: { x: number; z: number } | null = null;
  const draft: Draft = { length: 150, width: 30, height: 30, speed: 20, rotationY: 0, name: "",
x: 70, z: -60 };

  /** Set while running: one body per shape, keyed by shape id. */
  let bodies: Map<string, Body> | null = null;
  let started: Map<string, { x: number; z: number }> = new Map();
  let gates: Gates = { list: () => [], route: () => "straight" };
  let line: Line | null = null;
  /** Joins made during this run: which shape now rides on which. */
  let links = new Map<string, { parentId: string; attach: Attachment }>();
  let joins: Join[] = [];
  /** Where each stored item was when the run started: shape id -> bin id. */
  let storedAtStart = new Map<string, string>();
  const listeners: ((belts: readonly Conveyor[]) => void)[] = [];

  // Shapes stand on whichever belt is under them, and can be placed onto one.
  shapes.setSurface({
    support: (x, z) => supportHeight(belts, x, z),
    pick: (raycaster) => layer.surfacePoint(raycaster),
  });
```

The panel mirrors the Shapes panel: a few closed-over variables, a `Mode` union, a draft in centimeters. Two variables are new: `bodies` is non-null only while the line runs, which makes “running” a fact about the data rather than a separate flag that could disagree with it, and `started` remembers where each shape was when Run was pressed.

`shapes.setSurface` is how the two panels cooperate without importing each other's internals. The Shapes panel asks a `Surface` two questions, what height is under (x, z) and where the mouse ray hits a belt's top, and this module answers with the transport math and the layer. Shapes now sit on

belts, show reach from the top of the belt, and can be placed onto one.

src/shapes/controller.ts · lines 485–537

```
  // ---- For the conveyor controller ----

  /** Belts changed: shapes may now stand on (or fall off) one. */
  function setSurface(next: Surface) {
    surface = next;
    layer.supportAt = (x, z) => surface.support(x, z);
    refresh();
  }

  function refresh() {
    layer.sync(shapes);
    render();
  }

  function setRunning(on: boolean) {
    running = on;
    if (on) setMode({ kind: "idle" });
    else render();
  }

  /** After a run: keep where the belts left each shape, here and in the database. */
  function saveMoves(moves: readonly { id: string; x: number; z: number }[]) {
    const byId = new Map(moves.map((m) => [m.id, m]));
    shapes = shapes.map((s) => {
      const m = byId.get(s.id);
      return m ? { ...s, position: { x: m.x, z: m.z } } : s;
    });
    refresh();
    if (moves.length === 0) return Promise.resolve();
    return act(async () => {
      // One request per shape; a few dozen at most, so a bulk endpoint isn't worth it.
      for (const m of moves) await shapesApi.update(m.id, { position: { x: m.x, z: m.z } });
    }, `Saved where ${moves.length} shape${moves.length === 1 ? "" : "s"} ended up.`);
  }

  return {
    layer, handleClick, handleHover, handleKey, setSurface, setRunning, saveMoves, refresh,
act,
```

```
    /** Reload from the API now (after a change made elsewhere, such as an assembly). */
    reload: load,
    /** Where the connection point editor goes. */
    pointsHost: $("section.points-host"),
    get selectedId(): string | null {
      return selected;
    },
    onSelect(fn: (id: string | null) => void) {
      selectListeners.push(fn);
    },
    /** The shapes as last loaded (or saved). */
    get shapes(): readonly Shape[] {
      return shapes;
    },
  };
}
```

Run and Stop

src/conveyors/controller.ts · lines 149–253

```
// ---- Running ----

function start() {
  if (bodies) return;
  setMode({ kind: "idle" });
  bodies = new Map();
  started = new Map();
  links = new Map();
  joins = [];
  line?.stations.reset();
  storedAtStart = new Map(shapes.shapes.filter((s) => s.binId && !s.attachedTo).map((s) =>
[s.id, s.binId!]));
  // Shapes joined onto another ride on it, and stored ones are in their bins: only the ones
  // standing on their own get a body.
  for (const s of shapes.shapes) {
    if (s.attachedTo || s.binId) continue;
    bodies.set(s.id, {
      id: s.id, x: s.position.x, z: s.position.z, y: supportHeight(belts, s.position.x,
s.position.z),
      vx: 0, vy: 0, vz: 0, size: Math.max(s.size.x, s.size.z),
    });
```

```
    started.set(s.id, { ...s.position });
  }
  shapes.setRunning(true);
  message = null;
  render();
}

async function stop() {
  if (!bodies) return;
  const moves: { id: string; x: number; z: number }[] = [];
  // Shapes joined during the run are saved where they ended up too, so if the assembly is
  // refused (not enough stock), they're left at the machine rather than back at the start.
  const where = currentPoses();
  const storedNow = new Map(line?.stations.stored() ?? []);
  for (const [id, from] of started) {
    const p = where.get(id);
    if (!p || storedNow.has(id)) continue; // ended up in a bin: its position doesn't matter
    // Millimeters are plenty, and the API keeps whatever we send.
    const x = Math.round(p.x * 1000) / 1000;
    const z = Math.round(p.z * 1000) / 1000;
    if (Math.hypot(x - from.x, z - from.z) >= 0.001) moves.push({ id, x, z });
  }
  // What changed in the bins: items that left one (saved with where they ended up), and items
now in one.
  const round = (v: number) => Math.round(v * 1000) / 1000;
  const releases = [...storedAtStart].filter(([id, bin]) => storedNow.get(id) !== bin &&
where.has(id))
    .map(([shapeId, binId]) => ({ binId, shapeId, x: round(where.get(shapeId)!.x), z:
round(where.get(shapeId)!.z) }));
  const catches = [...storedNow].filter(([id, bin]) => storedAtStart.get(id) !==
bin).map(([shapeId, binId]) => ({ binId, shapeId }));
  const made = joins;
  bodies = null;
  joins = [];
  line?.stations.reset();
  line?.frame(0); // lights back to idle
  shapes.setRunning(false);
  if (moves.length === 0 && made.length === 0 && !releases.length && !catches.length) message =
{ text: "Nothing moved. Shapes ride a belt when placed on top of it.", error: false };
  render();
  // Releases first (they put items back on the floor), joins next, catches last (an item can
```

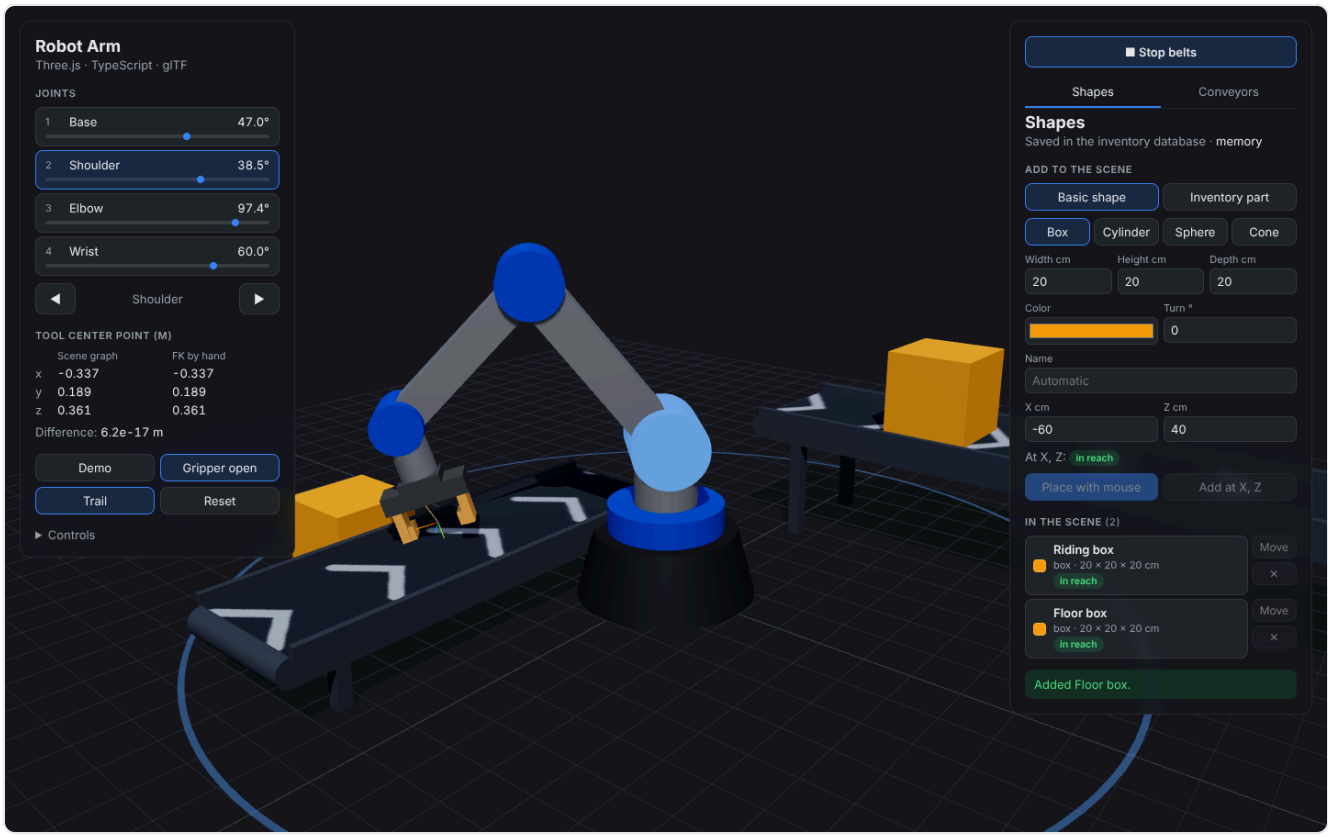
```
  // be joined and then caught as one assembly in the same run).
  if (releases.length && line) await line.saveReleases(releases);
  await shapes.saveMoves(moves);
  if (made.length && line) await line.saveJoins(made);
  if (catches.length && line) await line.saveCatches(catches);
  if (line && (releases.length || made.length || catches.length)) await shapes.reload();
}

/** Where every shape is now: bodies for the free ones, and joined ones riding on them. */
function currentPoses() {
  return poses(shapes.shapes, (s) => {
    const b = bodies?.get(s.id);
    return b
      ? { x: b.x, y: b.y + s.size.y / 2, z: b.z, rotationY: s.rotationY }
      : { x: s.position.x, y: supportHeight(belts, s.position.x, s.position.z) + s.size.y / 2,
z: s.position.z, rotationY: s.rotationY };
  }, links);
}

/** Called every frame by main.ts. */
function update(dt: number) {
  if (!bodies) return;
  layer.animate(dt);
  const st = line?.stations;
  const options = {
    gates: gates.list(), route: (b: Body, g: Gate) => gates.route(b.id!, g),
    stops: st?.stops() ?? [], hold: st?.hold,
  };
  for (let left = dt; left > 1e-9; left -= MAX_STEP) {
    const h = Math.min(MAX_STEP, left);
    st?.prepare(bodies);
    step(bodies.values(), belts, h, options);
    for (const j of st?.afterStep(h) ?? []) {
      // The attached shape stops being a free body: from now on it rides on the base.
      bodies.delete(j.attachedId);
      links.set(j.attachedId, { parentId: j.baseId, attach: j.attach });
      joins.push(j);
    }
    const bins = st?.binStep(h, bodies);
    for (const c of bins?.caught ?? []) bodies.delete(c.shapeId);
    for (const r of bins?.released ?? []) bodies.set(r.body.id!, r.body);
```

```
  }
  const stored = st?.stored() ?? new Map();
  for (const [id, p] of currentPoses()) {
    shapes.layer.poseCenter(id, p);
    shapes.layer.setVisible(id, !stored.has(rootOf(id, shapes.shapes, links)));
  }
  line?.frame(dt);
}

runButton.addEventListener("click", () => (bodies ? void stop() : start()));
```



The line running: the box on the back belt rides toward its far end, the Run button has become Stop, and the Shapes panel's edit buttons are disabled until the run ends.

Start makes a body for every shape, standing on whatever is under it, and freezes the Shapes panel: its poll stops and its edit buttons disable, because a refresh mid-run would snap shapes back to their saved positions.

update runs every frame from main.ts. A frame can take 16 ms or, on a slow machine, 50 ms; the physics always steps at most 1/120 s, as many times as needed. Big steps would make falls land late and could carry a fast item past a short belt in one jump. Then each shape's mesh is posed directly, with no database involved.

Stop compares each body with where it started and saves only the shapes that moved, rounded to the millimeter. Until Stop, a run is a preview: reload the page mid-run and everything is back where it was.

main.ts caps each frame's dt at 50 ms, so a tab that was hidden for a minute doesn't fast-forward. The same cap means a very slow machine runs the belts in slow motion rather than skipping: the test browser, drawing 3 frames a second, moved shapes at about a third of real speed.

Placing, moving and picking belts

src/conveyors/controller.ts · lines 255–357

```
// ---- Pointer ----

const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();
const floorPlane = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
const hitPoint = new THREE.Vector3();

function aim(e: PointerEvent) {
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
}

function floorPoint(e: PointerEvent): { x: number; z: number } | null {
  aim(e);
  if (!raycaster.ray.intersectPlane(floorPlane, hitPoint)) return null;
  const x = Math.round(hitPoint.x * 100) / 100;
  const z = Math.round(hitPoint.z * 100) / 100;
  return Math.abs(x) <= FLOOR_LIMIT && Math.abs(z) <= FLOOR_LIMIT ? { x, z } : null;
}

function ghostBelt(): { length: number; width: number; height: number; rotationY: number } |
null {
  if (mode.kind === "placing") return { length: draft.length / 100, width: draft.width / 100,
height: draft.height / 100, rotationY: draft.rotationY };
  if (mode.kind === "moving") {
    const id = mode.id;
    return belts.find((b) => b.id === id) ?? null;
  }
  return null;
```

```
}

/** True while placing or moving a belt: main.ts gives this module the pointer first then. */
const isActive = () => mode.kind !== "idle";

function handleHover(e: PointerEvent): boolean {
  const g = ghostBelt();
  if (!g) return false;
  hover = floorPoint(e);
  if (hover) layer.showGhost(g, hover.x, hover.z, g.rotationY);
  else layer.hideGhost();
  canvas.style.cursor = hover ? "crosshair" : "not-allowed";
  renderHint();
  return true;
}

/** Placing or moving: the click puts the belt down. */
function handleClick(e: PointerEvent): boolean {
  if (mode.kind === "idle") return false;
  const at = floorPoint(e);
  if (!at) return true;
  if (mode.kind === "placing") create(at.x, at.z);
  else {
    const id = mode.id;
    void act(() => conveyorsApi.update(id, { position: at }), "Moved.");
  }
  setMode({ kind: "idle" });
  return true;
}

/** Idle: a click on a belt selects it. Asked after the shapes, which sit on top. */
function handlePick(e: PointerEvent): boolean {
  aim(e);
  const id = layer.pick(raycaster);
  if (!id) return false;
  setSelected(id === selected ? null : id);
  showTab();
  return true;
}

function hoversBelt(e: PointerEvent): boolean {
```

```
  aim(e);
  return layer.pick(raycaster) !== null;
}

function handleKey(e: KeyboardEvent): boolean {
  if (e.key === "Escape" && mode.kind !== "idle") {
    setMode({ kind: "idle" });
    return true;
  }
  if ((e.key === "Delete" || e.key === "Backspace") && selected && mode.kind === "idle" &&
!bodies) {
    remove(selected);
    return true;
  }
  return false;
}

function setMode(next: Mode) {
  mode = next;
  hover = null;
  if (mode.kind === "idle") {
    layer.hideGhost();
    canvas.style.cursor = "";
  }
  render();
}

function setSelected(id: string | null) {
  selected = id;
  layer.select(id);
  render();
}

/** Clicking a belt in the scene opens its tab; main.ts wires the tabs. */
let showTab = () => {};
```

Belts are placed on the floor plane like shapes. While a belt (or a splitter) is being placed, main.ts gives that module the pointer first. Otherwise shapes are asked first, then splitter gates, then belts: whatever sits on top wins the click.

src/main.ts · lines 200–218

```
canvas.addEventListener("pointerup", (e) => {
  if (Math.hypot(e.clientX - pressX, e.clientY - pressY) > 4) return; // that was an orbit drag
  // Whoever is placing something gets the click; then shapes (they sit on top of belts), then
  // splitter gates (they stand over belts), then belts.
  if (line.isActive() && line.handleClick(e)) return; // putting a connection point on an
object, or placing a bin
  if (splitters.isActive() && splitters.handleClick(e)) return;
  if (belts.isActive() && belts.handleClick(e)) return;
  if (shapes.handleClick(e)) return; // placing, moving or selecting a shape
  if (splitters.handlePick(e)) return;
  if (belts.handlePick(e)) return;
  const hit = housingAt(e);
  if (hit) select(hit.userData.jointIndex as number);
});
canvas.addEventListener("pointermove", (e) => {
  if (e.buttons !== 0) return;
  if (line.isActive() ? line.handleHover(e) : splitters.isActive() ? splitters.handleHover(e) :
belts.isActive() ? belts.handleHover(e) : shapes.handleHover(e)) return;
  canvas.style.cursor = housingAt(e) || splitters.hoversGate(e) || belts.hoversBelt(e) ?
"pointer" : "";
});
```

The panel

src/conveyors/controller.ts · lines 359–428

```
// ---- The panel ----

root.innerHTML = `
  <section>
    <h2>Build a belt</h2>
    <div class="dims">
      <label>Length cm <input type="number" data-field="length" min="30" max="400" step="10">
</label>
      <label>Width cm <input type="number" data-field="width" min="10" max="120" step="5">
</label>
      <label>Height cm <input type="number" data-field="height" min="5" max="120" step="5">
</label>
    </div>
    <div class="row">
```

```
      <label>Speed cm/s <input type="number" data-field="speed" min="-100" max="100" step="5">
</label>
      <label>Turn ° <input type="number" data-field="rotationY" min="-360" max="360" step="15">
</label>
    </div>
    <label class="wide">Name <input type="text" data-field="name" maxlength="40"
placeholder="Automatic"></label>
    <div class="row">
      <label>X cm <input type="number" data-field="x" min="-300" max="300" step="5"></label>
      <label>Z cm <input type="number" data-field="z" min="-300" max="300" step="5"></label>
    </div>
    <div class="actions">
      <button type="button" data-action="place" class="primary">Place with mouse</button>
      <button type="button" data-action="add">Add at X, Z</button>
    </div>
    <p class="hint" hidden></p>
  </section>
  <section>
    <h2>Belts <span class="count"></span></h2>
    <ul class="shape-list belt-list"></ul>
  </section>
  <p class="shapes-status" role="status" hidden></p>
`;
const $ = <T extends HTMLElement>(sel: string) => root.querySelector(sel) as T;
const list = $("ul.belt-list");
const status = $("p.shapes-status");
const hint = $("p.hint");
const placeBtn = $<HTMLButtonElement>('[data-action="place"]');
const addBtn = $<HTMLButtonElement>('[data-action="add"]');
const field = (name: keyof Draft) => root.querySelector(`[data-field="${name}"]`) as
HTMLInputElement;

root.addEventListener("input", (e) => {
  const input = e.target as HTMLInputElement;
  const key = input.dataset.field as keyof Draft | undefined;
  if (!key) return;
  if (key === "name") draft.name = input.value;
  else if (input.value !== "") draft[key] = Number(input.value);
  if (mode.kind === "placing" && hover) layer.showGhost(ghostBelt()!, hover.x, hover.z,
draft.rotationY);
});
```

```
placeBtn.addEventListener("click", () => setMode(mode.kind === "placing" ? { kind: "idle" } : {
kind: "placing" }));
addBtn.addEventListener("click", () => create(draft.x / 100, draft.z / 100));
list.addEventListener("click", (e) => {
  const btn = (e.target as HTMLElement).closest<HTMLElement>("[data-id]");
  if (!btn) return;
  const id = btn.dataset.id!;
  const belt = belts.find((b) => b.id === id);
  if (!belt) return;
  switch (btn.dataset.do) {
    case "delete":
      return remove(id);
    case "move":
      setSelected(id);
      return setMode(mode.kind === "moving" && mode.id === id ? { kind: "idle" } : { kind:
"moving", id });
    case "turn":
      return void act(() => conveyorsApi.update(id, { rotationY: (belt.rotationY + 90) % 360
}));
    case "reverse":
      return void act(() => conveyorsApi.update(id, { speed: -belt.speed }));
    default:
      setSelected(id === selected ? null : id);
  }
});
```

Turn and reverse are one PATCH each: `rotationY + 90` modulo 360, and `-speed`. Every button that changes something is disabled while the line runs, in the list's HTML (`${dis}`) as well as in the click handler's state, so the screen never offers what the code would refuse.

src/conveyors/controller.ts · lines 429–489

```
function render() {
  for (const key of ["length", "width", "height", "speed", "rotationY", "name", "x", "z"] as
const) {
    const input = field(key);
    if (document.activeElement !== input) input.value = String(draft[key]);
  }
  const running = !!bodies;
  const locked = !online || busy || running;
  placeBtn.textContent = mode.kind === "placing" ? "Cancel placing" : "Place with mouse";
  placeBtn.setAttribute("aria-pressed", String(mode.kind === "placing"));
```

```
  placeBtn.disabled = addBtn.disabled = locked;

  runButton.textContent = running ? "■ Stop belts" : "▶ Run belts";
  runButton.setAttribute("aria-pressed", String(running));
  runButton.disabled = !online || busy || (!running && belts.length === 0);
  runButton.title = belts.length === 0 ? "Add a conveyor first" : running ? "Stop and save
where the shapes ended up" : "Start the belts";

  list.innerHTML = belts.length === 0
    ? `<li class="empty">${online ? "No belts yet. Place one with the mouse." : "Start the
inventory API to add belts."}</li>`
    : belts.map((b) => {
        const moving = mode.kind === "moving" && mode.id === b.id;
        const dis = running ? "disabled" : "";
        const dir = b.speed === 0 ? "stopped" : `${Math.abs(Math.round(b.speed * 100))} cm/s
${b.speed > 0 ? "forward" : "reversed"}`;
        return `<li class="${b.id === selected ? "selected" : ""}">
          <button type="button" class="pick" data-id="${b.id}" aria-pressed="${b.id ===
selected}">
            <span class="swatch belt-swatch"></span>
            <span class="label"><strong>${escapeHtml(b.name)}</strong>
              <small>${cm(b.length)} × ${cm(b.width)} cm, ${cm(b.height)} cm high</small>
              <small>${dir} · turned ${Math.round(b.rotationY)}°</small></span>
          </button>
          <span class="row-actions">
            <button type="button" data-id="${b.id}" data-do="move" aria-pressed="${moving}"
${dis}>${moving ? "Cancel" : "Move"}</button>
            <button type="button" data-id="${b.id}" data-do="turn" ${dis} title="Turn 90°">↻
90°</button>
            <button type="button" data-id="${b.id}" data-do="reverse" ${dis} title="Run the
other way">⇄</button>
            <button type="button" data-id="${b.id}" data-do="delete" ${dis} title="Delete (or
press Delete)">×</button>
          </span>
        </li>`;
      }).join("");
  $("span.count").textContent = belts.length ? `(${belts.length})` : "";

  status.hidden = !message;
  status.textContent = message?.text ?? "";
  status.classList.toggle("error", !!message?.error);
```

```
  renderHint();
}

function renderHint() {
  if (mode.kind === "idle") {
    hint.hidden = !bodies;
    hint.textContent = "Running. Shapes on a belt ride along and drop off the end. Stop saves
where they are.";
    return;
  }
  hint.hidden = false;
  const verb = mode.kind === "placing" ? "place it" : "move it there";
  hint.textContent = hover
    ? `x ${(hover.x * 100).toFixed(0)} cm, z ${(hover.z * 100).toFixed(0)} cm. Click to
${verb}.`
    : `Click the floor to ${verb}. Esc cancels.`;
}

render();
void load();
```

Wiring: tabs and the frame loop

src/main.ts · lines 28–67

```
// Shapes on the floor, stored in the inventory database (Snack 5's API).
const shapes = createShapes({ root: document.getElementById("shapes-panel")!, scene, camera,
canvas, arm: ARM });

// Conveyor belts, also stored there. Shapes on a running belt ride along.
const belts = createConveyors({
  root: document.getElementById("conveyors-panel")!,
  runButton: document.getElementById("btn-run") as HTMLButtonElement,
  scene, camera, canvas, shapes,
});

// Splitters: gates on the belts that send each item left, right or straight on, by your rules.
const splitters = createSplitters({ root: document.getElementById("splitters-panel")!, scene,
camera, canvas, shapes, conveyors: belts });

// Mergers and joining machines, plus the connection point editor in the Shapes tab.
const line = createLine({
```

```
  root: document.getElementById("machines-panel")!, binsRoot: document.getElementById("bins-
panel")!,
  scene, camera, canvas, shapes, conveyors: belts,
});

// The right-hand panel's tabs.
const tabs = {
  shapes: document.getElementById("tab-shapes")!,
  conveyors: document.getElementById("tab-conveyors")!,
  splitters: document.getElementById("tab-splitters")!,
  machines: document.getElementById("tab-machines")!,
  bins: document.getElementById("tab-bins")!,
};
function showTab(name: keyof typeof tabs) {
  for (const [key, tab] of Object.entries(tabs)) {
    const on = key === name;
    tab.setAttribute("aria-selected", String(on));
    document.getElementById(tab.getAttribute("aria-controls")!)!.hidden = !on;
  }
}
tabs.shapes.addEventListener("click", () => showTab("shapes"));
tabs.conveyors.addEventListener("click", () => showTab("conveyors"));
tabs.splitters.addEventListener("click", () => showTab("splitters"));
tabs.machines.addEventListener("click", () => showTab("machines"));
tabs.bins.addEventListener("click", () => showTab("bins"));
belts.onShowTab(() => showTab("conveyors"));
```

The right-hand panel became two tabs. Showing one hides the other's panel through the `aria-controls` link, which is also what a screen reader uses to announce them. Clicking a belt in the scene switches to the Conveyors tab.

`src/main.ts · lines 250–251`

```
gripper?.update(dt);
belts.update(dt); // moves shapes riding the belts while running
```

One line in the frame loop drives the whole line: `belts.update(dt)`, which does nothing unless it's running.

PART 1.18 · `src/splitters/rules.ts`

Splitters: rules that decide where items go

A splitter is a gate on a belt that sends each item left, right or straight on. What it separates is up to you: an ordered list of rules, stored in the database as data. This part is the decision itself, pure functions with no Three.js.

A rule list as data

src/splitters/types.ts · lines 1–58

```
/** Splitters as the inventory API stores them (snack5-inventory/api/src/domain/splitters.ts).
*/
export const EXITS = ["left", "straight", "right"] as const;
export type Exit = (typeof EXITS)[number];

export type FieldType = "text" | "number" | "boolean";

/** What a condition can test. Sizes are meters in the API; the panel shows centimeters. */
export const FIELDS = {
  kind: { type: "text", label: "Shape kind" },
  color: { type: "text", label: "Color" },
  name: { type: "text", label: "Shape name" },
  sku: { type: "text", label: "SKU" },
  width: { type: "number", label: "Width (cm)" },
  height: { type: "number", label: "Height (cm)" },
  depth: { type: "number", label: "Depth (cm)" },
  isPart: { type: "boolean", label: "Is an inventory part" },
  "part.category": { type: "text", label: "Part category" },
  "part.location": { type: "text", label: "Part location" },
  "part.name": { type: "text", label: "Part name" },
  "part.onHand": { type: "number", label: "Part on hand" },
  "part.available": { type: "number", label: "Part available" },
  "part.low": { type: "boolean", label: "Part low on stock" },
} as const satisfies Record<string, { type: FieldType; label: string }>;
export type Field = keyof typeof FIELDS;

export type Op = "eq" | "ne" | "in" | "contains" | "lt" | "lte" | "gt" | "gte";
export const OPS: Record<FieldType, readonly Op[]> = {
  text: ["eq", "ne", "in", "contains"],
  number: ["eq", "ne", "lt", "lte", "gt", "gte"],
```

```
  boolean: ["eq"],
};
export const OP_LABEL: Record<Op, string> = { eq: "is", ne: "is not", in: "is one of",
contains: "contains", lt: "<", lte: "≤", gt: ">", gte: "≥" };

export interface Condition {
  field: Field;
  op: Op;
  value: string | number | boolean | string[];
}

export interface Rule {
  conditions: Condition[];
  exit: Exit;
}

export interface Splitter {
  id: string;
  name: string;
  conveyorId: string;
  /** Meters from the start of the belt (its -x end before turning). */
  along: number;
  rules: Rule[];
  otherwise: Exit;
  createdAt: string;
  updatedAt: string;
}

export type NewSplitter = Omit<Splitter, "id" | "createdAt" | "updatedAt">;
export type SplitterPatch = Partial<Pick<Splitter, "name" | "along" | "rules" | "otherwise">>;
```

A rule is a list of conditions that must all hold, and an exit. A condition is three values: a **field**, an **operator** and a **value**. Keeping rules this plain is what lets them live in the database and be edited from anywhere: the panel, the API, or SQL. Nothing about a rule is code.

`FIELDS` gives each field a type, and `OPS` says which operators each type allows: text can be equal, not equal, one of a list, or contain something; numbers compare; true/false fields can only be equal. The editor builds its dropdowns from these two tables, so a rule like “kind > box” can't even be expressed. `satisfies` checks the table's shape without widening its type, so `FIELDS[f].type` stays the exact literal text, number or boolean.

Fields starting with `part.` describe the **inventory part** a shape stands for: its category, location and stock.

Reading a field, and matching one condition

src/splitters/rules.ts · lines 1–62

```
/**
 * Which way does a splitter send an item? Pure functions over a shape and the inventory
 * part it stands for, so they're unit-tested and the panel can preview every decision.
 *
 * Part fields are looked up from the latest inventory list when the item reaches the gate,
 * so changing a part in the database (its category, location or stock) changes where its
 * items go on the next pass, without touching the splitter.
 */
import type { Shape } from "../shapes/types";
import type { InventoryPart } from "../shapes/fromPart";
import type { Condition, Exit, Field, Splitter } from "./types";

export type Part = InventoryPart & { location?: string; onHand: number; available: number;
low?: boolean; reorderLevel?: number };

/** The value of one field for a shape, or undefined when it doesn't apply (part fields for a
plain shape). */
export function fieldValue(field: Field, shape: Shape, part: Part | undefined): string | number
| boolean | undefined {
  switch (field) {
    case "kind": return shape.kind;
    case "color": return shape.color;
    case "name": return shape.name;
    case "sku": return shape.sku ?? undefined;
    case "width": return shape.size.x;
    case "height": return shape.size.y;
    case "depth": return shape.size.z;
    case "isPart": return shape.sku !== null;
    case "part.category": return part?.category;
    case "part.location": return part?.location;
    case "part.name": return part?.name;
    case "part.onHand": return part?.onHand;
    case "part.available": return part?.available;
    case "part.low": return part ? (part.low ?? part.available <= (part.reorderLevel ?? 0)) :
undefined;
  }
}
```

```
/** Sizes are stored in meters but typed in centimeters; compare to the nearest millimeter so
20 cm equals 0.2 m. */
const close = (a: number, b: number) => Math.abs(a - b) < 0.0005;

export function matches(c: Condition, shape: Shape, part: Part | undefined): boolean {
  const v = fieldValue(c.field, shape, part);
  // A condition about a part never matches a shape that isn't one, even "is not".
  if (v === undefined) return false;
  if (typeof v === "number") {
    const x = Number(c.value);
    switch (c.op) {
      case "eq": return close(v, x);
      case "ne": return !close(v, x);
      case "lt": return v < x && !close(v, x);
      case "lte": return v < x || close(v, x);
      case "gt": return v > x && !close(v, x);
      case "gte": return v > x || close(v, x);
      default: return false;
    }
  }
  if (typeof v === "boolean") return c.op === "eq" && v === c.value;
  const text = v.toLowerCase();
  switch (c.op) {
    case "eq": return text === String(c.value).toLowerCase();
    case "ne": return text !== String(c.value).toLowerCase();
    case "contains": return text.includes(String(c.value).toLowerCase());
    case "in": return Array.isArray(c.value) && c.value.some((s) => s.toLowerCase() === text);
    default: return false;
  }
```

`fieldValue` is one `switch` that maps a field name to a value from the shape or its part. It returns `undefined` when the field doesn't apply, a part field for a plain shape. `matches` then treats undefined as “no match” for every operator, including “is not”. Otherwise “part category is not plate” would be true for a plain red box, which has no category at all: true by a technicality, and surprising.

Sizes are compared to the millimeter. The panel takes centimeters and the API stores meters, and `20 / 100` is exactly 0.2, but a size computed elsewhere might be 0.20000000000000004. Exact equality on floating-point numbers is a classic bug, so `close` allows half a millimeter.

Text comparisons ignore case, so “Plate”, “plate” and “PLATE” are the same category. The API also lowercases text values when it saves a rule.

First match wins

src/splitters/rules.ts · lines 64–69

```

/** The first rule whose conditions all hold decides; otherwise the splitter's default. rule is
-1 for the default. */
export function route(splitter: Pick<Splitter, "rules" | "otherwise">, shape: Shape, part: Part
| undefined): { exit: Exit; rule: number } {
  const i = splitter.rules.findIndex((r) => r.conditions.length > 0 && r.conditions.every((c)
=> matches(c, shape, part)));
  return i >= 0 ? { exit: splitter.rules[i].exit, rule: i } : { exit: splitter.otherwise, rule:
-1 };
}
```

`route` is the whole decision: the first rule whose conditions all hold picks the exit; if none does, the splitter's `otherwise` exit. It returns the rule's index too, so the panel can say *why* an item went where it did.

Order matters, as in a firewall or a mail filter. A rule “plates → left” above “anything taller than 10 cm → right” sends a tall plate left; swap them and it goes right. That's why the editor has ↑ ↓ buttons, and why the API stores the rules as one ordered list rather than as separate rows.

Testing the rules

tests/splitters.test.ts · lines 1–51

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { matches, route, type Part } from "../src/splitters/rules";
import { step, sideOf, type Body, type Gate } from "../src/conveyors/transport";
import type { Shape } from "../src/shapes/types";
import type { Condition, Rule } from "../src/splitters/types";

const shape = (over: Partial<Shape> = {}): Shape => ({
  id: "s1", name: "Crate", kind: "box", size: { x: 0.2, y: 0.15, z: 0.2 }, color: "#e11d48",
  position: { x: 0, z: 0 }, rotationY: 0, sku: null, createdAt: "", updatedAt: "", ...over,
});
const plate: Part = { sku: "PL-200", name: "Mounting plate, 200 mm", category: "plate", unit:
"plate", onHand: 45, available: 41, location: "B-01-2", reorderLevel: 10 };
const c = (field: Condition["field"], op: Condition["op"], value: Condition["value"]):
Condition => ({ field, op, value });

test("text conditions ignore case; 'is one of' takes a list", () => {
```

```
  assert.ok(matches(c("color", "eq", "#E11D48"), shape(), undefined));
  assert.ok(matches(c("kind", "in", ["sphere", "BOX"]), shape(), undefined));
  assert.ok(matches(c("name", "contains", "rat"), shape(), undefined));
  assert.ok(!matches(c("kind", "ne", "box"), shape(), undefined));
});

test("sizes compare in meters, to the millimeter", () => {
  assert.ok(matches(c("height", "eq", 0.15), shape({ size: { x: 0.2, y: 0.15000000000000002, z:
0.2 } }), undefined));
  assert.ok(matches(c("height", "gte", 0.15), shape(), undefined));
  assert.ok(!matches(c("height", "gt", 0.15), shape(), undefined));
  assert.ok(matches(c("width", "lt", 0.25), shape(), undefined));
});

test("part conditions read the inventory part, and never match a plain shape", () => {
  const p = shape({ sku: "PL-200" });
  assert.ok(matches(c("part.category", "eq", "plate"), p, plate));
  assert.ok(matches(c("part.available", "gt", 40), p, plate));
  assert.ok(matches(c("isPart", "eq", true), p, plate));
  assert.ok(matches(c("isPart", "eq", false), shape(), undefined));
  assert.ok(!matches(c("part.category", "ne", "plate"), shape(), undefined)); // not a part: no
part.category at all
  assert.ok(matches(c("part.low", "eq", false), p, plate));
  assert.ok(matches(c("part.low", "eq", true), p, { ...plate, available: 10 }));
});

test("the first matching rule wins; otherwise the default", () => {
  const rules: Rule[] = [
    { conditions: [c("part.category", "eq", "plate")], exit: "left" },
    { conditions: [c("color", "eq", "#e11d48"), c("height", "gt", 0.1)], exit: "right" },
  ];
  const s = { rules, otherwise: "straight" as const };
  assert.deepEqual(route(s, shape({ sku: "PL-200" }), plate), { exit: "left", rule: 0 });
  assert.deepEqual(route(s, shape(), undefined), { exit: "right", rule: 1 });
  assert.deepEqual(route(s, shape({ size: { x: 0.2, y: 0.05, z: 0.2 } }), undefined), { exit:
"straight", rule: -1 });
  // Changing the part in the database changes the decision, with the same rules.
  assert.deepEqual(route(s, shape({ sku: "PL-200", color: "#000000" }), { ...plate, category:
"gripper" }), { exit: "straight", rule: -1 });
});
```

The last test is the one that matters for “I want to change it by changing the database”: the same rules give a different answer when only the part's category is different. The arm app doesn't cache decisions; it asks `route` with the latest inventory each time an item reaches a gate.

PART 1.19 · `src/conveyors/transport.ts`

Gates on the running line

How the belts' motion code asks a splitter what to do, and what “send it left” means physically.

Gates, sides and the push-off velocity

src/conveyors/transport.ts · lines 19–28

```
/** A splitter's gate: on which belt, and how far from that belt's start (its -u end). */
export interface Gate {
  id: string;
  conveyorId: string;
  along: number;
}
export type Exit = "left" | "straight" | "right";

/** A diverted item leaves at this angle to the belt's direction of travel. */
export const DIVERT_ANGLE = 60 * (Math.PI / 180);
```

To the transport code, a gate is just a belt id and a distance along the belt. It doesn't know about rules: it asks a `route` callback, so the same motion code works whatever decides the exits.

src/conveyors/transport.ts · lines 122–140

```
/**
 * Which side of a belt is "left" depends on which way it runs. Seen from above, facing the
 * direction of travel, right is the belt's +v side when it runs forward (+u), and -v when
 * it runs backwards.
 */
export function sideOf(exit: "left" | "right", speed: number): 1 | -1 {
  const forward = speed >= 0 ? 1 : -1;
  return (exit === "right" ? forward : -forward) as 1 | -1;
}

/** World velocity of an item pushed off a belt's side at DIVERT_ANGLE to its travel. */
export function divertVelocity(b: Belt, side: 1 | -1): { vx: number; vz: number } {
  const t = b.rotationY * DEG;
  const du = b.speed * Math.cos(DIVERT_ANGLE);
  const dv = side * Math.abs(b.speed) * Math.sin(DIVERT_ANGLE);
  // u points along (cos θ, -sin θ) in world x, z, and v along (sin θ, cos θ): see toBelt.
  return { vx: du * Math.cos(t) + dv * Math.sin(t), vz: -du * Math.sin(t) + dv * Math.cos(t) };
}
```

Left and right are taken facing the way the belt runs, as a person standing on the line would say them. Seen from above with y up, facing +x, your right hand points to +z, which is the belt's +v. Running backwards you face −u, and right becomes −v. `sideOf` is that rule, and a test checks both directions (Figure 1.8).

A diverted item leaves at 60° to the travel direction at the belt's speed: `cos 60°` of it still along the belt, `sin 60°` sideways. The result is turned into world x and z with the same (cos θ, −sin θ) and (sin θ, cos θ) axes as `toBelt`. Once it's past the side rail nothing holds it up, and the existing falling code takes over: it drops to the floor, or onto a belt running alongside.

Crossing a gate

src/conveyors/transport.ts · lines 236–252

```
    if (!route) continue;

    // Did this step carry it past a gate on this belt? Gates are measured from the belt's -u
end.
    for (const gate of gates) {
      if (gate.conveyorId !== belt.id) continue;
      const g = gate.along - belt.length / 2;
      const crossed = (before < g && after >= g) || (before > g && after <= g);
      if (!crossed) continue;
      const exit = route(body, gate);
      if (exit !== "straight") {
        body.divert = { beltId: belt.id, side: sideOf(exit, belt.speed) };
        break;
      }
    }
  }
}
```

A gate is a line across the belt, so “reached the gate” means **crossed** it during this step: the item's `u` was before the gate and is now at or past it (or the reverse on a backwards belt). Checking for “exactly at the gate” would almost never be true, because items move a few millimeters per step.

Each item is asked about once per gate. A diverted item is skipped (it's leaving), and an item sent straight has passed the line and won't cross it again. The gate is measured from the belt's start (its −u end), so `along - length / 2` converts it to the belt-centered `u` that `toBelt` returns.

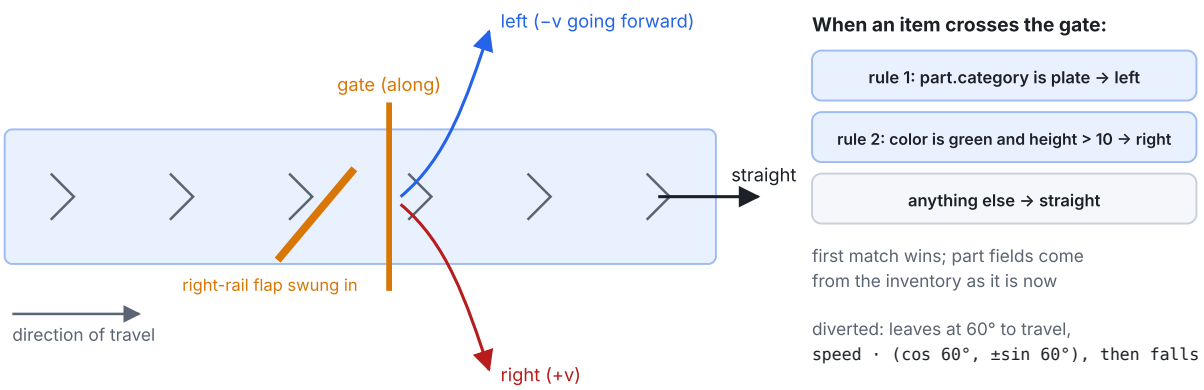


Figure 1.8. A splitter seen from above. Left and right are taken facing the way the belt runs, so they swap when the belt is reversed.

Testing the gates

tests/splitters.test.ts · lines 53–100

```
// ---- Gates on a running belt ----

const belt = { id: "b1", length: 2, width: 0.4, height: 0.3, speed: 0.5, position: { x: 0, z: 0
}, rotationY: 0 };
const gate: Gate = { id: "g1", conveyorId: "b1", along: 1 }; // the middle of the belt
const body = (id: string): Body => ({ id, x: -0.8, z: 0, y: 0.3, vx: 0, vy: 0, vz: 0 });
function run(bodies: Body[], belts: typeof belt[], exits: Record<string, "left" | "right" |
"straight">, seconds: number, asked: string[] = []) {
  const route = (b: Body, g: Gate) => (asked.push(`${b.id}@${g.id}`), exits[b.id!]);
  for (let t = 0; t < seconds; t += 1 / 120) step(bodies, belts, 1 / 120, { gates: [gate],
route });
}

test("facing the way the belt runs, right is +v going forward and -v going backward", () => {
  assert.equal(sideOf("right", 0.5), 1);
  assert.equal(sideOf("left", 0.5), -1);
  assert.equal(sideOf("right", -0.5), -1);
});

test("a gate sends items left, right or straight, and asks once per item", () => {
  const [l, r, s] = [body("l"), body("r"), body("s")];
  const asked: string[] = [];
  run([l, r, s], [belt], { l: "left", r: "right", s: "straight" }, 6, asked);
```

```
  assert.deepEqual(asked, ["l@g1", "r@g1", "s@g1"]);
  // Belt runs +x; seen from above facing +x, right is +z.
  assert.equal(l.y, 0);
  assert.ok(l.z < -0.2, `left went to -z: ${l.z}`);
  assert.equal(r.y, 0);
  assert.ok(r.z > 0.2, `right went to +z: ${r.z}`);
  assert.ok(Math.abs(s.z) < 1e-9 && s.x > 1, `straight rode off the end: ${s.x}`);
  // Diverted items leave near the gate, not at the end.
  assert.ok(l.x > 0 && l.x < 0.6 && r.x > 0 && r.x < 0.6, `${l.x}, ${r.x}`);
});

test("on a reversed belt, left and right follow the direction of travel", () => {
  const b = { ...belt, speed: -0.5 };
  const it: Body = { id: "x", x: 0.8, z: 0, y: 0.3, vx: 0, vy: 0, vz: 0 };
  for (let t = 0; t < 4; t += 1 / 120) step([it], [b], 1 / 120, { gates: [gate], route: () =>
"right" });
  assert.ok(it.z < -0.2, `facing -x, right is -z: ${it.z}`);
});

test("gates only act on their own belt, and a diverted item lands on a belt beside it", () => {
  const side = { id: "b2", length: 2, width: 0.4, height: 0.15, speed: 0.5, position: { x: 0.2,
z: 0.45 }, rotationY: 0 };
  const it = body("p");
  for (let t = 0; t < 2.5; t += 1 / 120) step([it], [belt, side], 1 / 120, { gates: [gate, {
id: "g2", conveyorId: "b2", along: 0.1 }], route: () => "right" });
  assert.equal(it.y, 0.15); // caught by the lower belt on the right
  assert.ok(it.divert === undefined, "riding normally again");
  const x = it.x;
  for (let t = 0; t < 0.5; t += 1 / 120) step([it], [belt, side], 1 / 120, { gates: [gate],
route: () => "right" });
  assert.ok(it.x > x + 0.2, "carried along the second belt");
});
```

Three items cross one gate: the route callback is asked exactly three times, in order, and each ends up on the right side of the belt, or off its end. The diverted ones leave near the gate rather than at the end, which shows the push happened at the right place. The last test chains belts: an item pushed right lands on a lower belt alongside and is carried on, with its `divert` cleared.

PART 1.20 · `src/splitters/controller.ts`

Drawing gates, and the Splitters tab

The gate model with its swinging flaps, and the panel where rules are written.

A gate in its own frame

src/splitters/SplitterLayer.ts · lines 1–29

```
import * as THREE from "three";
import { DEG } from "../arm/kinematics";
import type { Conveyor } from "../conveyors/types";
import type { Exit, Splitter } from "./types";

const FLAP_ANGLE = 50 * DEG; // how far a flap swings into the belt
const SWING_RATE = 10; // 1/s: flaps ease toward their target, fast but visibly
const HOLD = 0.9; // seconds a flap stays out after an item was sent its way

interface Gate {
  group: THREE.Group;
  /** Hinged on the +z (right, going forward) and -z (left) rails of the gate's own frame. */
  flaps: { right: THREE.Object3D; left: THREE.Object3D };
  /** Current and target swing, 0..1, and when each flap may close again. */
  swing: { right: number; left: number };
  until: { right: number; left: number };
  beam: THREE.MeshStandardMaterial;
  key: string;
}

/**
 * Draws each splitter as a diverter gate on its belt: an amber gantry across the belt at the
 * gate's position, and two flaps hinged on the side rails. When an item is sent left, the flap
 * on the right swings across the belt and guides it off the left side, and the other way
round.
 *
 * Each gate is built in its own frame: x along the direction of travel (so the gate is turned
 * round on a reversed belt), z to the right, y up. Then "right" is always +z, whichever way
 * the belt is turned or running.
 */
```

src/splitters/SplitterLayer.ts · lines 133–166

```
function placeOnBelt(group: THREE.Object3D, belt: Conveyor, along: number): void {
  const t = belt.rotationY * DEG;
  const u = along - belt.length / 2;
  group.position.set(belt.position.x + u * Math.cos(t), 0, belt.position.z - u * Math.sin(t));
  group.rotation.y = t + (belt.speed < 0 ? Math.PI : 0);
}

function buildGate(belt: Pick<Conveyor, "width" | "height">, beam: THREE.Material, flap:
THREE.Material) {
  const group = new THREE.Group();
  const w = belt.width;
  const top = belt.height + 0.22;
  const add = (geo: THREE.BufferGeometry, mat: THREE.Material, x: number, y: number, z: number,
parent: THREE.Object3D = group) => {
    const m = new THREE.Mesh(geo, mat);
    m.position.set(x, y, z);
    m.castShadow = true;
    parent.add(m);
    return m;
  };
  // Gantry: two posts outside the rails and a beam across, so the gate is easy to spot and
click.
  for (const side of [-1, 1]) add(new THREE.BoxGeometry(0.03, top, 0.03), beam, 0, top / 2,
side * (w / 2 + 0.04));
  add(new THREE.BoxGeometry(0.04, 0.04, w + 0.11), beam, 0, top, 0);

  // Flaps: hinged at the rail just upstream of the gate, lying along the rail when idle.
  const length = w * 0.9;
  const hingeX = -length * Math.cos(FLAP_ANGLE) * 0.5;
  const make = (side: 1 | -1) => {
    const hinge = new THREE.Group();
    hinge.position.set(hingeX, belt.height + 0.04, side * (w / 2 - 0.01));
    add(new THREE.BoxGeometry(length, 0.06, 0.012), flap, length / 2, 0, 0, hinge);
    add(new THREE.CylinderGeometry(0.012, 0.012, 0.08, 12), beam, 0, 0, 0, hinge);
    group.add(hinge);
    return hinge;
  };
  return { group, flaps: { right: make(1), left: make(-1) } };
```

Each gate is a group built in a frame where +x is the direction of travel and +z is the right-hand side. `placeOnBelt` turns the group by the belt's angle, plus 180° when the belt runs backwards. After that, “the right-hand flap” is always the one at +z, whatever the belt is doing, and the swing code never needs to think about the belt's direction.

Each flap sits inside its own hinge group, positioned at the rail. Rotating the hinge swings the flap about its end, the same trick as the arm's joints: put the pivot in a parent object and rotate that.

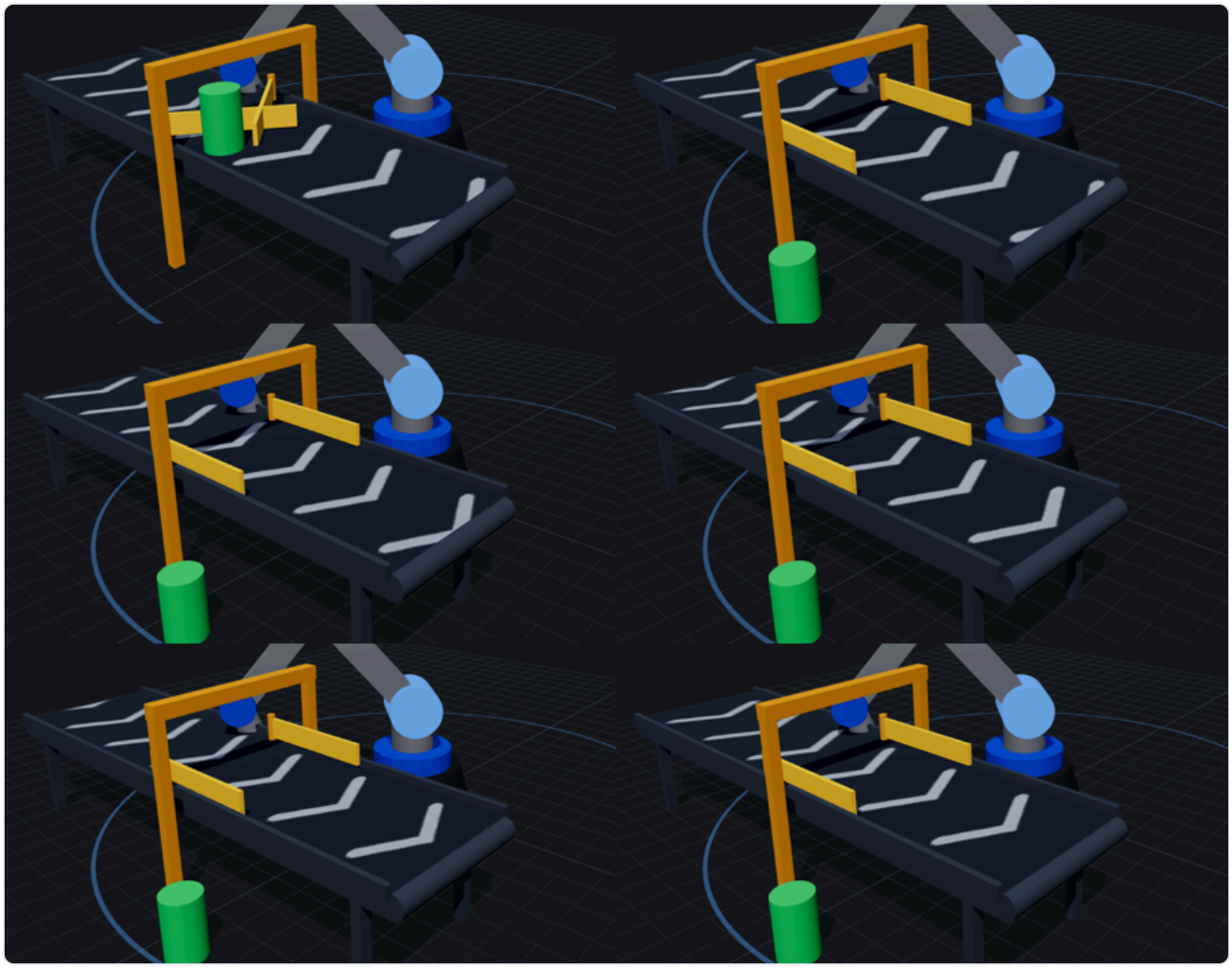
Swinging the flaps

src/splitters/SplitterLayer.ts · lines 80–100

```
/** An item was routed at this gate: swing the flap that guides it out. */
trigger(id: string, exit: Exit): void {
  const g = this.gates.get(id);
  if (!g || exit === "straight") return;
  // Sending left needs the right-hand flap across the belt, and the other way round.
  const flap = exit === "left" ? "right" : "left";
  g.until[flap] = this.time + HOLD;
}

update(dt: number): void {
  this.time += dt;
  const k = 1 - Math.exp(-SWING_RATE * dt);
  for (const g of this.gates.values()) {
    for (const side of ["right", "left"] as const) {
      const target = this.time < g.until[side] ? 1 : 0;
      g.swing[side] += (target - g.swing[side]) * k;
      // Positive rotation about y turns +x toward -z: the right flap swings left, into the
belt.
      g.flaps[side].rotation.y = (side === "right" ? 1 : -1) * FLAP_ANGLE * g.swing[side];
    }
  }
}
```

To send an item **left**, the flap on the **right** rail swings across and guides it out: `trigger` records until when that flap should stay out. `update` eases each flap toward out or in with the same frame-rate-independent factor as the demo motion, `1 − e^(−rate·dt)`. A positive rotation about y turns +x toward −z, so the right flap swings positive and the left flap negative. The flaps are only drawn: the motion code has already diverted the item, so the picture follows the decision rather than causing it.



A run filmed from in front, panels hidden. The can matches “kind is cylinder → right”, so the flap on the far (left) rail swings across and the can drops off the near side.

The splitter's state, and plugging into the line

src/splitters/controller.ts · lines 1–71

```
/**
 * The Splitters tab: put diverter gates on belts and write the rules that decide where each
 * item goes. Rules are saved through the inventory API, so they can also be changed from the
 * API or the database directly; the tab re-reads them, and the inventory, every few seconds,
 * including while the line runs. A change takes effect for the next item that reaches the
gate.
 */
import * as THREE from "three";
import { splittersApi, ShapesApiError } from "../shapes/api";
import type { createShapes } from "../shapes/controller";
import type { createConveyors } from "../conveyors/controller";
```

```
import { toBelt } from "../conveyors/transport";
import type { Conveyor } from "../conveyors/types";
import { SHAPE_KINDS, type Shape } from "../shapes/types";
import { route, type Part } from "./rules";
import { SplitterLayer } from "./SplitterLayer";
import { EXITS, FIELDS, OP_LABEL, OPS, type Condition, type Exit, type Field, type Rule, type
Splitter } from "./types";

const POLL_MS = 3000;
const EXIT_LABEL: Record<Exit, string> = { left: "← left", straight: "↑ straight", right:
"right →" };
/** Size fields are meters in the API and centimeters on screen. */
const IN_CM = new Set<Field>(["width", "height", "depth"]);

interface Options {
  root: HTMLElement;
  scene: THREE.Scene;
  camera: THREE.Camera;
  canvas: HTMLCanvasElement;
  shapes: ReturnType<typeof createShapes>;
  conveyors: ReturnType<typeof createConveyors>;
}

export function createSplitters({ root, scene, camera, canvas, shapes, conveyors }: Options) {
  const layer = new SplitterLayer();
  scene.add(layer.root);

  let splitters: Splitter[] = [];
  let parts: Part[] = [];
  let selected: string | null = null;
  let placing = false;
  let online = false;
  let busy = false;
  let message: { text: string; error: boolean } | null = null;
  const draft = { name: "", beltId: "", along: 50 };
  /** The selected splitter's rules as being edited, and whether they differ from what's saved.
*/
  let edit: { rules: Rule[]; otherwise: Exit } | null = null;
  let dirty = false;
  /** The latest decision at each gate, shown in the panel while the line runs. */
  const lastRouted = new Map<string, { shape: string; exit: Exit; rule: number }>();
```

```
  const belts = () => conveyors.belts;
  const partFor = (s: Shape) => (s.sku ? parts.find((p) => p.sku === s.sku) : undefined);

  // The running line asks here when an item reaches a gate. Parts are the latest loaded,
  // so a change in the inventory applies from the next item on.
  conveyors.setGates({
    list: () => splitters.filter((s) => belts().some((b) => b.id === s.conveyorId)),
    route: (shapeId, gate) => {
      const s = splitters.find((x) => x.id === gate.id);
      const shape = shapes.shapes.find((x) => x.id === shapeId);
      if (!s || !shape) return "straight";
      const decision = route(s, shape, partFor(shape));
      lastRouted.set(s.id, { shape: shape.name, ...decision });
      layer.trigger(s.id, decision.exit);
      if (s.id === selected) renderLive();
      return decision.exit;
    },
  });
  conveyors.onBeltsChange(() => {
    layer.sync(splitters, belts());
    render();
  });
```

`conveyors.setGates` is how the splitters reach the running line without the line knowing about rules. The line asks for the gates and, when an item crosses one, calls `route` with the shape's id. This module finds the shape, looks up its part in the latest inventory list, applies the rules, starts the flap and returns the exit.

Here is where “change the database, change the sorting” happens. Splitters and parts are re-read every 3 s, **including while the belts run**, unlike shapes and belts, which are frozen during a run. The next item through a gate is judged on the inventory as it is now.

Loading, saving, and edits that aren't lost

src/splitters/controller.ts · lines 73–144

```
// ---- API ----

async function load() {
  try {
    const [list, stock] = await Promise.all([splittersApi.list(), splittersApi.parts()]);
    online = true;
```

```
    splitters = list;
    parts = stock;
    layer.sync(splitters, belts());
    if (selected && !splitters.some((s) => s.id === selected)) select(null);
    // Someone else changed the rules (API, database, another browser): show theirs unless
we're mid-edit.
    if (selected && !dirty) edit = editable(splitters.find((s) => s.id === selected)!);
    if (message?.error && message.text.startsWith("Can't reach")) message = null;
  } catch (err) {
    online = false;
    message = { text: (err as Error).message, error: true };
  }
  render();
}

async function act(work: () => Promise<unknown>, success?: string) {
  busy = true;
  render();
  try {
    await work();
    message = success ? { text: success, error: false } : null;
    await load();
  } catch (err) {
    const e = err as ShapesApiError;
    message = { text: e.problems?.length ? e.problems.join("; ") : e.message, error: true };
  } finally {
    busy = false;
    render();
  }
}

setInterval(() => {
  if (!busy && !placing) void load();
}, POLL_MS);

function create(beltId: string, along: number) {
  const belt = belts().find((b) => b.id === beltId);
  if (!belt) {
    message = { text: "Pick a belt first.", error: true };
    return render();
  }
```

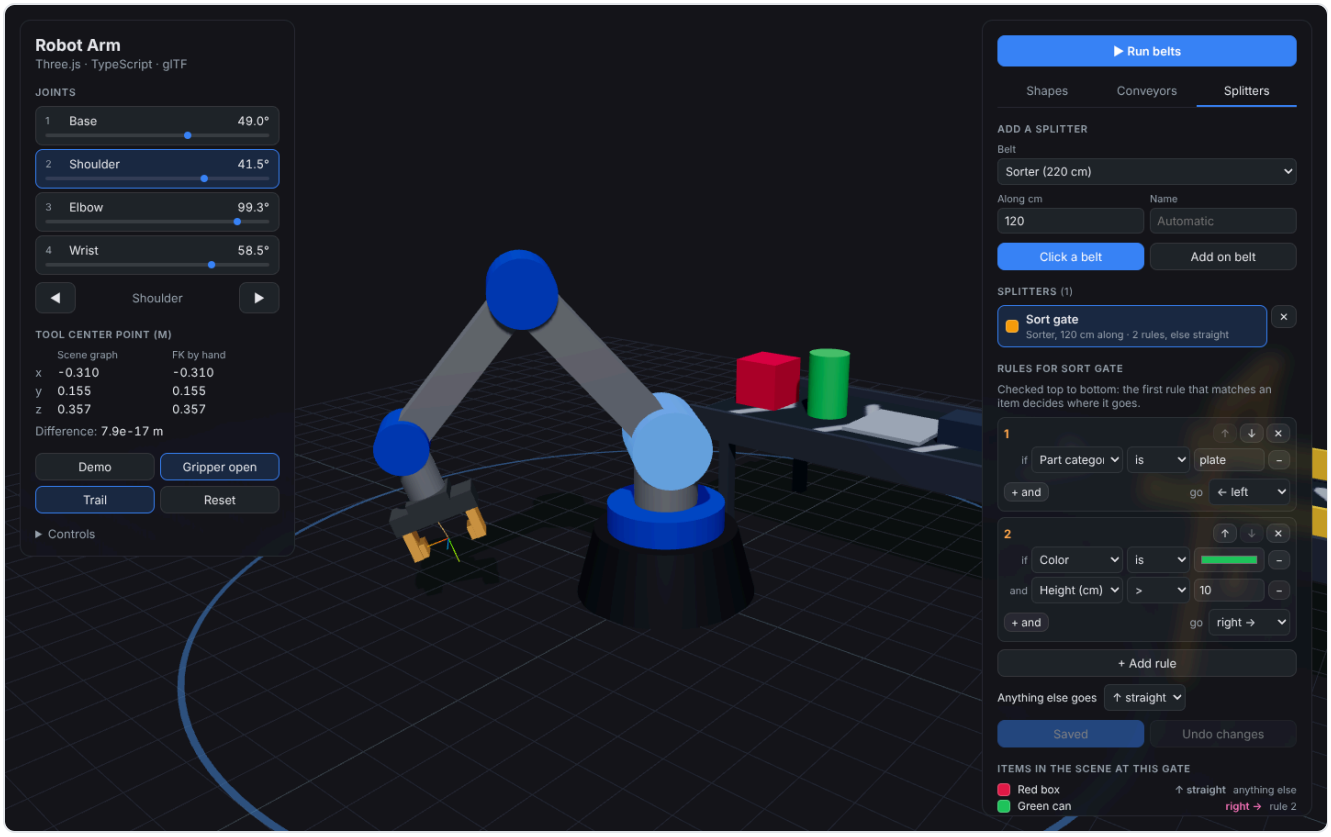
```
  const name = draft.name.trim() || `Splitter ${splitters.length + 1}`;
  void act(async () => {
    const s = await splittersApi.create({ name, conveyorId: beltId, along:
Math.min(belt.length, Math.max(0, along)), rules: [], otherwise: "straight" });
    select(s.id);
  }, `Added ${name}. Now add rules to say what goes where.`);
  draft.name = "";
}

function select(id: string | null) {
  selected = id;
  dirty = false;
  const s = splitters.find((x) => x.id === id);
  edit = s ? editable(s) : null;
  layer.select(id);
  render();
}

function save() {
  if (!selected || !edit) return;
  const id = selected;
  const body = { rules: edit.rules.map(toApi), otherwise: edit.otherwise };
  void act(async () => {
    await splittersApi.update(id, body);
    dirty = false;
  }, "Rules saved.");
}
```

The rule editor works on a copy, `edit`, with a `dirty` flag. A poll replaces the copy with the saved rules only when nothing is being edited, so changes made elsewhere (another browser, the API, SQL) appear without wiping what you're typing. Save sends the whole ordered list; Undo goes back to what's saved. Switching to another splitter with unsaved changes asks first.

The rule editor



Two rules being edited. The list at the bottom previews where each item in the scene would go at this gate, before saving.

src/splitters/controller.ts · lines 285–378

```
root.addEventListener("input", (e) => {
  const el = e.target as HTMLInputElement | HTMLSelectElement;
  const key = el.dataset.draft as keyof typeof draft | undefined;
  if (key === "along") draft.along = Number(el.value || 0);
  else if (key === "name" || key === "beltId") draft[key] = el.value;
  if (key) return;
  if (!edit) return;
  // Rule editor: data-r is the rule, data-c the condition, data-k what's being changed.
  if (el.dataset.otherwise !== undefined) {
    edit.otherwise = el.value as Exit;
    return changed();
  }
  const r = Number(el.dataset.r);
  const rule = edit.rules[r];
  if (!rule) return;
  if (el.dataset.k === "exit") rule.exit = el.value as Exit;
  else {
```

```
    const cond = rule.conditions[Number(el.dataset.c)];
    if (!cond) return;
    if (el.dataset.k === "field") {
      cond.field = el.value as Field;
      const type = FIELDS[cond.field].type;
      if (!OPS[type].includes(cond.op)) cond.op = OPS[type][0];
      cond.value = defaultValue(cond.field);
      return changed(true);
    }
    if (el.dataset.k === "op") {
      cond.op = el.value as Condition["op"];
      if (cond.op === "in" && !Array.isArray(cond.value)) cond.value = String(cond.value) ?
[String(cond.value)] : [];
      if (cond.op !== "in" && Array.isArray(cond.value)) cond.value = cond.value.join(", ");
      return changed(true);
    }
    if (el.dataset.k === "value") cond.value = readValue(cond, el.value);
  }
  changed();
});

/** Re-draws the preview at once, and the editor's rows only when their layout changed (so
typing keeps focus). */
function changed(relayout = false) {
  dirty = true;
  if (relayout) renderRules();
  renderPreview();
  renderButtons();
}

root.addEventListener("click", (e) => {
  const btn = (e.target as HTMLElement).closest<HTMLButtonElement>("button");
  if (!btn || !root.contains(btn)) return;
  if (btn === placeBtn) return setPlacing(!placing);
  if (btn === addBtn) return create(draft.beltId, draft.along / 100);
  const id = btn.dataset.id;
  if (id) {
    if (btn.dataset.do === "delete") {
      const s = splitters.find((x) => x.id === id);
      if (id === selected) select(null);
      return void act(() => splittersApi.remove(id), s ? `Deleted ${s.name}.` : undefined);
```

```
    }
    if (id !== selected && !confirmDiscard()) return;
    return select(id === selected ? null : id);
  }
  if (!edit) return;
  const r = Number(btn.dataset.r);
  switch (btn.dataset.do) {
    case "add-rule":
      edit.rules.push({ conditions: [{ field: "kind", op: "eq", value: "box" }], exit: "left"
});
      return changed(true);
    case "add-cond":
      if (edit.rules[r].conditions.length < 5) edit.rules[r].conditions.push({ field: "color",
op: "eq", value: "#e11d48" });
      return changed(true);
    case "del-cond": {
      const cs = edit.rules[r].conditions;
      cs.splice(Number(btn.dataset.c), 1);
      if (cs.length === 0) edit.rules.splice(r, 1);
      return changed(true);
    }
    case "del-rule":
      edit.rules.splice(r, 1);
      return changed(true);
    case "up":
    case "down": {
      const to = btn.dataset.do === "up" ? r - 1 : r + 1;
      if (to < 0 || to >= edit.rules.length) return;
      [edit.rules[r], edit.rules[to]] = [edit.rules[to], edit.rules[r]];
      return changed(true);
    }
    case "save":
      return save();
    case "revert":
      dirty = false;
      edit = editable(splitters.find((s) => s.id === selected)!);
      return render();
  }
});
```

One `input` listener and one `click` listener handle the whole editor, by event delegation. Each control carries its rule index (`data-r`), condition index (`data-c`) and what it changes (`data-k`). Changing a condition's field resets its operator and value to ones that fit the new field's type. Changing a value only updates the data, without re-rendering the rows, so typing never loses the cursor.

src/splitters/controller.ts · lines 462–498

```
function conditionRow(c: Condition, r: number, i: number): string {
  const type = FIELDS[c.field].type;
  const fieldSel = `<select data-r="${r}" data-c="${i}" data-k="field" aria-
label="Field">${(Object.keys(FIELDS) as Field[])
    .map((f) => `<option value="${f}" ${f === c.field ? "selected" : ""}>${FIELDS[f].label}
</option>`).join("")}</select>`;
  const opSel = `<select data-r="${r}" data-c="${i}" data-k="op" aria-
label="Comparison">${OPS[type]
    .map((o) => `<option value="${o}" ${o === c.op ? "selected" : ""}>${OP_LABEL[o]}
</option>`).join("")}</select>`;
  const attrs = `data-r="${r}" data-c="${i}" data-k="value" aria-label="Value"`;
  let input: string;
  if (type === "boolean") {
    input = `<select ${attrs}><option value="true" ${c.value === true ? "selected" :
""}>yes</option><option value="false" ${c.value === false ? "selected" : ""}>no</option>
</select>`;
  } else if (type === "number") {
    const v = IN_CM.has(c.field) ? Math.round(Number(c.value) * 1000) / 10 : Number(c.value);
    input = `<input type="number" step="any" value="${v}" ${attrs}>`;
  } else if (c.field === "color" && (c.op === "eq" || c.op === "ne")) {
    input = `<input type="color" value="${esc(String(c.value))}" ${attrs}>`;
  } else {
    const list = { kind: "dl-kind", "part.category": "dl-category", sku: "dl-sku",
"part.location": "dl-location" }[c.field as string];
    const v = Array.isArray(c.value) ? c.value.join(", ") : String(c.value);
    input = `<input type="text" value="${esc(v)}" ${list ? `list="${list}"` : ""}
placeholder="${c.op === "in" ? "a, b, c" : ""}" ${attrs}>`;
  }
  return `<div class="cond">${i > 0 ? `<span class="and">and</span>` : `<span
class="and">if</span>`}${fieldSel}${opSel}${input}
    <button type="button" data-do="del-cond" data-r="${r}" data-c="${i}" title="Remove
condition" aria-label="Remove condition">–</button></div>`;
}
```

```
/** Where every shape would go at this gate under the rules being edited, saved or not. */
function renderPreview() {
  if (!edit) return;
  const items = shapes.shapes;
  preview.innerHTML = items.length === 0
    ? `<li class="empty">No shapes in the scene.</li>`
    : items.map((s) => {
        const d = route(edit!, s, partFor(s));
        return `<li><span class="swatch" style="background:${esc(s.color)}"></span><span
class="pname">${esc(s.name)}</span>
          <span class="exit ${d.exit}">${EXIT_LABEL[d.exit]}</span><small>${d.rule >= 0 ? `rule
${d.rule + 1}` : "anything else"}</small></li>`;
      }).join("");
}
```

The value control follows the field: yes/no for true/false fields, a number box in centimeters for sizes, a color picker for “color is”, and a text box with suggestions for the rest. The suggestions (`<datalist>`) come from the inventory as it is now, such as the categories and locations that exist.

`renderPreview` runs `route` on every shape with the rules **as edited**, saved or not. That makes the editor a tool for thinking: change a rule and immediately see which items would go where.

PART 1.21 · `src/line/joints.ts`

Connection points and joining geometry

Objects can now be joined to each other at named connection points: a can on a plate's hole, an actuator on a bracket. This part is the geometry: where a point is on an object, where to put one object so its point meets another's, and where everything in an assembly ends up.

Points per kind of object

src/line/types.ts · lines 1–23

```
/** Line equipment as the inventory API stores it (snack5-
inventory/api/src/domain/assembly.ts). */
import type { ShapeKind, Size } from "../shapes/types";
import type { Condition } from "../splitters/types";

/** A kind of object: an inventory part, or plain shapes of one kind. */
export type Owner = `part:${string}` | `kind:${ShapeKind}`;

/** A named spot on an object, as fractions of its size from its center (-0.5..0.5; y = 0.5 is
the top). */
export interface ConnectionPoint {
  name: string;
  x: number;
  y: number;
  z: number;
}

export interface Connectors {
  owner: Owner;
  points: ConnectionPoint[];
  updatedAt: string;
}

export type MergerKind = "side" | "y";
export type Priority = "main" | "feeders" | "alternate";
```

src/line/joints.ts · lines 1–40

```
/**
 * Joining geometry: where a connection point is on an object, where to put one object so its
 * point meets another's, and where everything in an assembly is in the world. Pure math, no
 * Three.js, so it's unit-tested.
 *
 * Rotations are about the vertical axis in degrees, with Three.js's sign: turning a vector
 * (x, z) by θ gives (x cos θ + z sin θ, −x sin θ + z cos θ).
 */
import type { Shape } from "../shapes/types";
import type { Attachment, ConnectionPoint, Connectors, Owner } from "./types";
```

```
export const DEFAULT_POINTS: readonly ConnectionPoint[] = [
  { name: "top", x: 0, y: 0.5, z: 0 },
  { name: "bottom", x: 0, y: -0.5, z: 0 },
];

const DEG = Math.PI / 180;

export function ownerOf(shape: Pick<Shape, "sku" | "kind">): Owner {
  return shape.sku ? `part:${shape.sku}` : `kind:${shape.kind}`;
}

/** The points for a kind of object: what's stored, or the defaults. */
export function pointsFor(owner: Owner, stored: readonly Connectors[]): readonly
ConnectionPoint[] {
  return stored.find((c) => c.owner === owner)?.points ?? DEFAULT_POINTS;
}

export function findPoint(owner: Owner, name: string, stored: readonly Connectors[]):
ConnectionPoint | undefined {
  return pointsFor(owner, stored).find((p) => p.name.toLowerCase() === name.toLowerCase());
}

export function turn(x: number, z: number, degrees: number): { x: number; z: number } {
  const t = degrees * DEG;
  return { x: x * Math.cos(t) + z * Math.sin(t), z: -x * Math.sin(t) + z * Math.cos(t) };
}

/** A connection point in meters, in the object's own frame (from its center, before its turn).
*/
export function localPoint(shape: Pick<Shape, "size">, p: ConnectionPoint): { x: number; y:
number; z: number } {
  return { x: p.x * shape.size.x, y: p.y * shape.size.y, z: p.z * shape.size.z };
}
```

A connection point belongs to a **kind** of object, not to one object: every PL-200 plate has the same points, and so does every plain cylinder. `ownerOf` names the kind: `part:PL-200` for a shape that stands for a part, `kind:cylinder` for a plain one. Until someone defines points for a kind it gets **top** and **bottom**, so every object can be joined from the start.

A point's position is stored as **fractions of the object's size**, −0.5 to 0.5 from its center. That one decision makes the points fit every size: a “hole 25% left of center on top” is in the right place on a 20 cm plate and on a 40 cm one. `localPoint` turns the fractions into meters for a particular object.

`turn` is the y rotation used everywhere: the same convention as Three.js's `rotation.y`, so math done here matches what's drawn.

Point to point

src/line/joints.ts · lines 41–56

```
/**
 * Where to put `attached` so its point meets `base`'s point, keeping the attached object's
 * current heading in the world. Returned in the base's frame, which is how it's stored: then
 * the pair moves and turns together.
 */
export function attachmentFor(
  base: Pick<Shape, "size" | "rotationY">, basePoint: ConnectionPoint,
  attached: Pick<Shape, "size" | "rotationY">, attachedPoint: ConnectionPoint,
): Attachment {
  const relative = attached.rotationY - base.rotationY;
  const b = localPoint(base, basePoint);
  const a = localPoint(attached, attachedPoint);
  const at = turn(a.x, a.z, relative); // the attached point, turned into the base's frame
  return { x: b.x - at.x, y: b.y - a.y, z: b.z - at.z, rotationY: relative };
}
```

To join A onto B at their points, A goes where its point lands exactly on B's point, keeping its own heading. In B's frame: B's point, minus A's point turned by A's heading relative to B. The result is stored **in B's frame**, which is what makes the pair move as one: turn or carry B, and A comes along, because its offset is relative to B.

This is the scene-graph idea from the arm (a child's transform is relative to its parent) used for data instead of meshes. The database stores the parent's id and the offset, exactly like a Three.js object's parent and local position.

Poses through any number of joins

src/line/joints.ts · lines 57–109

```
export interface Pose {
  /** Center of the object in the world. */
  x: number;
  y: number;
  z: number;
  rotationY: number;
```

```
}

/**
 * The world pose of every shape. Shapes on their own come from `rootPose` (where they stand,
 * or where the belts have carried them); joined shapes follow their base, through any number
 * of joins.
 */
export function poses(
  shapes: readonly Shape[],
  rootPose: (s: Shape) => Pose,
  links: ReadonlyMap<string, { parentId: string; attach: Attachment }> = new Map(),
): Map<string, Pose> {
  const byId = new Map(shapes.map((s) => [s.id, s]));
  const out = new Map<string, Pose>();
  const linkOf = (s: Shape) => links.get(s.id) ?? (s.attachedTo && s.attach ? { parentId:
s.attachedTo, attach: s.attach } : undefined);
  const visit = (s: Shape, depth: number): Pose => {
    const done = out.get(s.id);
    if (done) return done;
    const link = linkOf(s);
    const parent = link && byId.get(link.parentId);
    let pose: Pose;
    if (!link || !parent || depth > 50) pose = rootPose(s);
    else {
      const p = visit(parent, depth + 1);
      const o = turn(link.attach.x, link.attach.z, p.rotationY);
      pose = { x: p.x + o.x, y: p.y + link.attach.y, z: p.z + o.z, rotationY: p.rotationY +
link.attach.rotationY };
    }
    out.set(s.id, pose);
    return pose;
  };
  for (const s of shapes) visit(s, 0);
  return out;
}

/** The shape at the bottom of a chain of joins: the one that stands on the floor or a belt. */
export function rootOf(id: string, shapes: readonly Shape[], links: ReadonlyMap<string, {
parentId: string }> = new Map()): string {
  const byId = new Map(shapes.map((s) => [s.id, s]));
  let cur = id;
```

```
  for (let i = 0; i < 50; i++) {
    const s = byId.get(cur);
    const parent = links.get(cur)?.parentId ?? s?.attachedTo ?? null;
    if (!parent || !byId.has(parent)) return cur;
    cur = parent;
  }
  return cur;
}
```

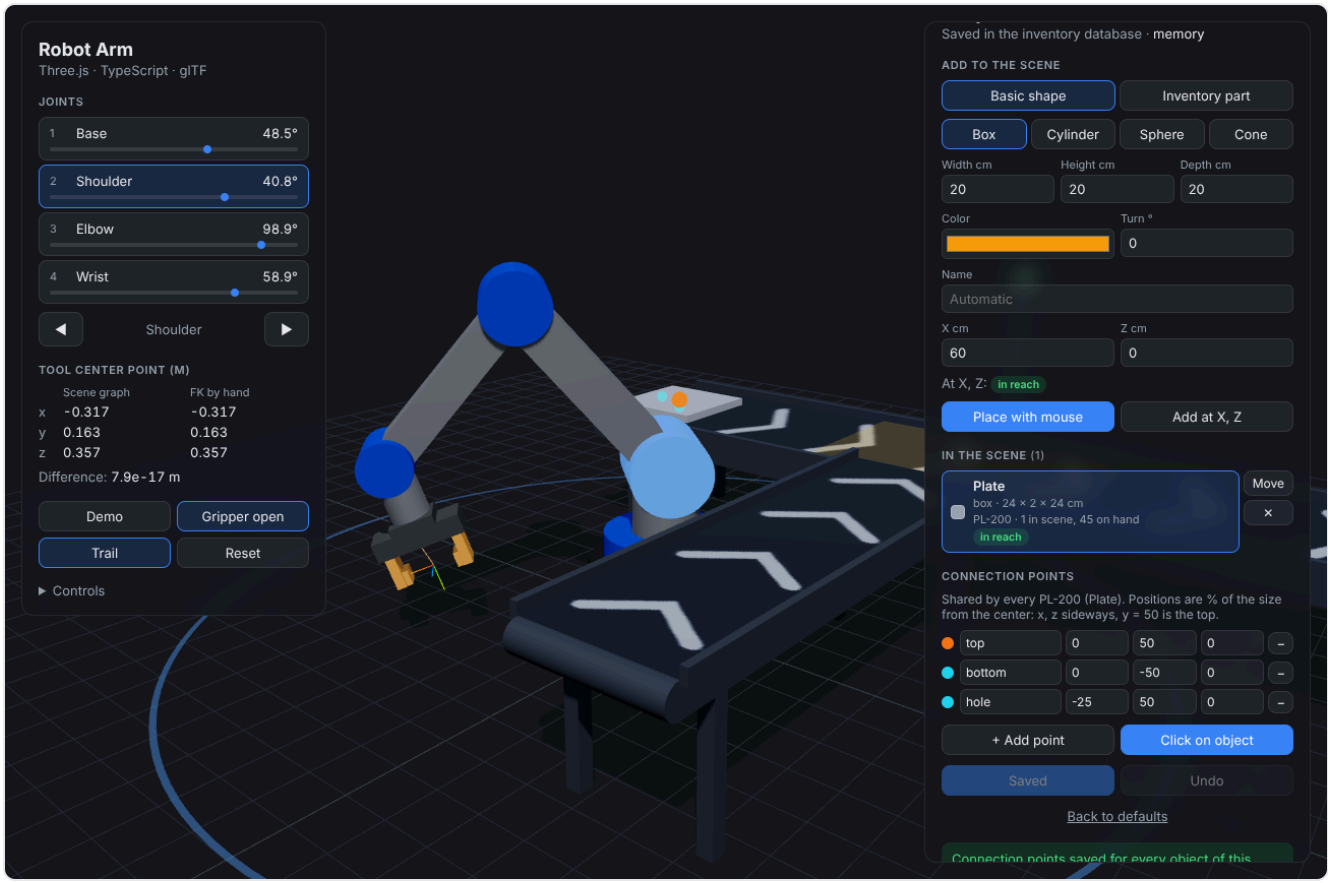
`poses` works out every shape's world position. A free shape gets its pose from `rootPose` (where it stands, or where the belts have carried it); a joined shape takes its parent's pose and adds its offset, turned by the parent's heading. Parents are visited first by recursion, with memoization so each shape is computed once, and a depth limit so a corrupt loop in the data can't hang the page. Joins made during a run come in as `links`, overriding the stored ones.

tests/stations.test.ts · lines 130–150

```
test("joining geometry: the attached point lands on the base point, whatever the turns", () =>
{
  const base = { size: { x: 0.4, y: 0.02, z: 0.2 }, rotationY: 30 };
  const attached = { size: { x: 0.1, y: 0.3, z: 0.1 }, rotationY: 75 };
  const bp = { name: "corner", x: 0.5, y: 0.5, z: -0.5 };
  const ap = { name: "side", x: -0.5, y: -0.5, z: 0 };
  const a = attachmentFor(base, bp, attached, ap);
  // Put the base anywhere and work out both points in the world: they must coincide.
  const shapes = [shape("b", { ...base }), shape("a", { ...attached })];
  const links = new Map([["a", { parentId: "b", attach: a }]]);
  const pose = poses(shapes, () => ({ x: 0.7, y: 0.31, z: -0.2, rotationY: 30 }), links);
  const world = (id: string, p: typeof bp, sz: typeof base.size) => {
    const q = pose.get(id)!;
    const l = localPoint({ size: sz }, p);
    const o = turn(l.x, l.z, q.rotationY);
    return [q.x + o.x, q.y + l.y, q.z + o.z];
  };
  const [w1, w2] = [world("b", bp, base.size), world("a", ap, attached.size)];
  w1.forEach((v, i) => assert.ok(Math.abs(v - w2[i]) < 1e-9, `${w1} vs ${w2}`));
  assert.equal(pose.get("a")!.rotationY, 75, "the attached object keeps its heading");
});
```

The test doesn't check the formula: it checks the promise. It joins an object with an awkward point (a corner) onto another turned 30°, then computes both points in the world, separately, from the poses, and requires them to coincide.

Editing points



Editing the plate kind's points: top, bottom and a new “hole” 25% left of center. The dots on the plate are the points; orange is the highlighted one.

`src/line/PointEditor.ts · lines 85–133`

```
// ---- Markers on every object of this kind ----

function refreshMarkers() {
  for (const m of markers) m.parent?.remove(m);
  markers.length = 0;
  if (!owner) return;
  for (const s of shapes.shapes) {
    if (ownerOf(s) !== owner) continue;
    const mesh = shapes.layer.meshOf(s.id);
    if (!mesh) continue;
    const r = Math.max(0.008, Math.min(0.02, Math.max(s.size.x, s.size.y, s.size.z) * 0.06));
    draft.forEach((p, i) => {
      const on = i === active;
      const marker = new THREE.Mesh(MARKER, new THREE.MeshBasicMaterial({ color: on ? 0xf97316
: 0x22d3ee, depthTest: false, transparent: true }));
```

```
      // Mesh geometry is centered on the object, in meters, so a point is just its fractions
times the size.
      marker.position.set(p.x * s.size.x, p.y * s.size.y, p.z * s.size.z);
      marker.scale.setScalar(on ? r * 1.5 : r);
      marker.renderOrder = 10; // drawn over the object, so points on the far side show too
      mesh.add(marker);
      markers.push(marker);
    });
  }
}

// ---- Clicking on the object to place a point ----

const raycaster = new THREE.Raycaster();
const pointer = new THREE.Vector2();

function handleClick(e: PointerEvent): boolean {
  if (!picking || !owner) return false;
  const rect = canvas.getBoundingClientRect();
  pointer.set(((e.clientX - rect.left) / rect.width) * 2 - 1, -((e.clientY - rect.top) /
rect.height) * 2 + 1);
  raycaster.setFromCamera(pointer, camera);
  const candidates = shapes.shapes.filter((s) => ownerOf(s) === owner).map((s) => ({ s, mesh:
shapes.layer.meshOf(s.id) })).filter((c) => c.mesh);
  const hit = raycaster.intersectObjects(candidates.map((c) => c.mesh!), false)[0];
  if (!hit) return true; // keep picking until the object is clicked
  const s = candidates.find((c) => c.mesh === hit.object)!.s;
  // The hit in the object's own frame, as fractions of its size.
  const local = (hit.object as THREE.Mesh).worldToLocal(hit.point.clone());
  const frac = (v: number, size: number) => Math.round(Math.max(-0.5, Math.min(0.5, v / size))
* 100) / 100;
  draft.push({ name: nextName(), x: frac(local.x, s.size.x), y: frac(local.y, s.size.y), z:
frac(local.z, s.size.z) });
  active = draft.length - 1;
  dirty = true;
  picking = false;
  canvas.style.cursor = "";
  render();
  return true;
}
```

Markers are small spheres added as **children of each object's mesh**. The mesh's geometry is centered on the object and in meters, so a marker's position is just the point's fractions times the size, and it follows the object wherever it goes. `depthTest: false` and a high `renderOrder` draw markers over the object, so points on its far side show too.

Click on object raycasts against the shapes of the kind being edited. `mesh.worldToLocal(hit.point)` undoes the mesh's position and turn, giving the hit in the object's own frame; dividing by the size gives the fractions to store. The same three numbers then work for every object of that kind.

PART 1.22 · `src/line/stations.ts`

Mergers and machines: stations on the line

Mergers bring belts together; joining machines join two items into one. Both are stations: they hold items at lines on a belt and decide when to let them go. This part is the motion they need, and their decisions.

What transport gained: stop lines, queues and guided moves

src/conveyors/transport.ts · lines 92–120

```
/**
 * A line across a belt where a station (a merger, a machine) may hold items. With `queue`,
 * items arriving behind a held item stop behind it, spaced by their sizes, instead of
 * passing through it: that's how a feeder belt backs up while it waits to merge.
 */
export interface StopLine {
  id: string;
  conveyorId: string;
  along: number;
  queue?: boolean;
}

export interface StepOptions {
  gates?: readonly Gate[];
  /** Asked once when a body crosses a gate: where should it go? */
  route?: (body: Body, gate: Gate) => Exit;
  stops?: readonly StopLine[];
  /** Asked every step while a body is at or crossing a stop line: hold it there? */
  hold?: (body: Body, stop: StopLine) => boolean;
  floorLimit?: number;
```

```
}

/** World position of a point given in a belt's frame: the inverse of toBelt. */
export function fromBelt(b: Belt, u: number, v: number): { x: number; z: number } {
  const t = b.rotationY * DEG;
  return { x: b.position.x + u * Math.cos(t) + v * Math.sin(t), z: b.position.z - u *
Math.sin(t) + v * Math.cos(t) };
}

const QUEUE_GAP = 0.02; // meters between queued items
```

src/conveyors/transport.ts · lines 200–235

```
// Stop lines: a station may hold the item at the line (it's put back exactly on it).
let held: string | undefined;
if (hold) {
  for (const stop of stops) {
    if (stop.conveyorId !== belt.id) continue;
    const g = stop.along - belt.length / 2;
    if (Math.min(before, after) <= g && g <= Math.max(before, after) && before !== after &&
hold(body, stop)) {
      after = g;
      held = stop.id;
      break;
    }
  }
}
// Queues: don't pass through something held at a queueing line, or queued behind one.
if (!held) {
  const queueing = new Set(stops.filter((s) => s.queue && s.conveyorId === belt.id).map((s) =>
s.id));
  for (const other of all) {
    if (other === body || !other.heldAt || (other.heldAt !== "queue" &&
!queueing.has(other.heldAt))) continue;
    if (!isOver(belt, other.x, other.z) || Math.abs(other.y - body.y) > 1e-3) continue;
    const ou = toBelt(belt, other.x, other.z).u;
    const limit = ou - dir * ((body.size ?? 0.2) + (other.size ?? 0.2)) / 2 - dir * QUEUE_GAP;
    // Only what's ahead of it counts, and only if this step would take it past the limit.
    if ((ou - before) * dir > 0 && (after - limit) * dir > 0) {
      after = (before - limit) * dir > 0 ? before : limit;
      held = "queue";
    }
```

```
  }
}
body.heldAt = held;
if (held) {
  const p = fromBelt(belt, after, start.v);
  body.x = p.x;
  body.z = p.z;
  body.vx = body.vz = 0;
  continue;
}
```

A **stop line** is a line across a belt where a station may hold an item. As with gates, “at the line” means *crossed it this step*; the station's `hold` callback is asked, and if it says yes the item is put back exactly on the line and its speed set to zero. It's asked again every step while the item sits there, which is how a station releases it later: by answering no.

Stop lines can **queue**. An item arriving behind one held at a queueing line, or behind one already queued, stops a spacing short of it (half of each item's size plus 2 cm) instead of passing through. That's the only collision in the model, kept to one belt and one direction so it stays exact and cheap. It's what lets a feeder back up neatly while it waits to merge.

src/conveyors/transport.ts · lines 146–163

```
if (body.guide) {
  // Slid by a station: straight to the target at the guide's speed, then let go on whatever is
there.
  const g = body.guide;
  const dx = g.x - body.x;
  const dz = g.z - body.z;
  const d = Math.hypot(dx, dz);
  body.y = g.y;
  body.vx = body.vy = body.vz = 0;
  if (d <= g.speed * dt) {
    body.x = g.x;
    body.z = g.z;
    body.guide = undefined;
  } else {
    body.x += (dx / d) * g.speed * dt;
    body.z += (dz / d) * g.speed * dt;
  }
  continue;
}
```

A **guided move** slides an item straight to a target at a set speed, ignoring belts on the way, then lets go. Mergers use it to bring a feeder item onto the main belt's centerline, like the guide rails of a real merge.

Mergers: where they hold, and when the way is clear

src/line/stations.ts · lines 1–30

```
/**
 * What mergers and joining machines do while the belts run. Pure logic over the transport's
 * bodies: no Three.js, no API, so it's unit-tested.
 *
 * Each station puts stop lines on belts (transport.ts holds items there when asked) and keeps
a
 * little state: which item is waiting where, who merged last, what a machine is holding.
 *
 * Mergers
 *   Feeder items stop at the end of their feeder and wait for a gap at the merge point: no
 *   other item within `gap` of it on the main belt, and nothing else being merged. Then
they're
 *   slid onto the main belt's centerline. Priority decides who waits when both sides want to
go:
 *   - main: feeder items wait for a gap; the main belt never stops.
 *   - feeders: main-belt items stop short of the merge point while a feeder item is waiting.
 *   - alternate: they take turns. For a y merger, the two feeders take turns.
 *   Feeder stops queue, so a busy feeder backs up instead of piling items on one spot.
 *
 * Machines
 *   A machine holds the first item matching its base and the first matching its attach slot
 *   (by kind of object). With both, it works for `cycleSeconds`, then joins the attach item
onto
 *   the base at their connection points and lets the base, now carrying it, go on. Items it
 *   doesn't need, or doesn't need yet, pass straight through.
 *
 * Bins
 *   An item that comes to rest on the floor inside a bin is caught if it's one of the bin's
kinds, meets
 *   its accept conditions and the bin has room; otherwise it's left lying there. A bin with a
feed
 *   puts its items back on the line: onto a belt whenever the spot is clear ("steady") or only
 *   when a machine further along that belt needs one ("demand"), or straight into a machine's
slot.
```

```
 *   A bin holding several kinds releases its oldest item that's wanted: for a machine, the
oldest
 *   one whose kind the machine has an empty slot for.
 */
```

src/line/stations.ts · lines 85–138

```
// ---- Geometry ----

/** Where items merge: on the main belt's centerline. For a y merger, just inside the output's
start. */
function mergePoint(m: Merger): { x: number; z: number; y: number; u: number } | null {
  const main = beltById(m.conveyorId);
  if (!main) return null;
  const u = m.kind === "y" ? -dirOf(main) * (main.length / 2 - 0.1) : m.along - main.length /
2;
  return { ...fromBelt(main, u, 0), y: main.height, u };
}

function feederStop(m: Merger, i: number): StopLine | null {
  const f = beltById(m.feederIds[i]);
  if (!f) return null;
  const along = dirOf(f) > 0 ? f.length - FEEDER_STOP_INSET : FEEDER_STOP_INSET;
  return { id: `feed:${m.id}:${i}`, conveyorId: f.id, along, queue: true };
}

/** For "feeders" and "alternate", main-belt items may have to stop a gap short of a side
merge. */
function mainStop(m: Merger): StopLine | null {
  const main = beltById(m.conveyorId);
  if (!main || m.kind === "y" || m.priority === "main") return null;
  const along = m.along - dirOf(main) * m.gap;
  return along >= 0 && along <= main.length ? { id: `main:${m.id}`, conveyorId: main.id, along,
queue: true } : null;
}

function stops(): StopLine[] {
  const { mergers, machines } = world();
  const out: StopLine[] = [];
  for (const m of mergers) {
    m.feederIds.forEach((_, i) => {
      const s = feederStop(m, i);
```

```
      if (s) out.push(s);
    });
    const s = mainStop(m);
    if (s) out.push(s);
  }
  for (const mc of machines) if (beltById(mc.conveyorId)) out.push({ id: `machine:${mc.id}`,
conveyorId: mc.conveyorId, along: mc.along });
  return out;
}

/** Is the merge point clear for one more item? */
function clear(m: Merger, except: Body): boolean {
  const main = beltById(m.conveyorId);
  const at = mergePoint(m);
  if (!main || !at) return false;
  for (const b of bodies.values()) {
    if (b === except) continue;
    if (b.guide && Math.hypot(b.guide.x - at.x, b.guide.z - at.z) < 1e-6) return false; // one
is being merged now
    if (b.heldAt === `main:${m.id}`) continue; // waiting short of the merge point, by design
    if (!isOver(main, b.x, b.z) || Math.abs(b.y - main.height) > 1e-3 ||
beltUnder(world().belts, b.x, b.z, b.y) !== main) continue;
    if (Math.abs(toBelt(main, b.x, b.z).u - at.u) < m.gap) return false;
  }
  return true;
}
```

A merger puts a queueing stop near the end of each feeder, and, for the “feeders first” and “take turns” priorities, one on the main belt a gap short of the merge point. The merge point is on the main belt's centerline: `along` for a side merger, just inside the start for a Y.

`clear` is the rule of the road: nothing being guided into this merge point, and no item on the main belt within `gap` of it, ahead or behind. Items waiting at the main belt's own stop don't count: they're waiting on purpose, a gap away.

Who goes first

src/line/stations.ts · lines 140–197

```
// ---- Holding ----

function holdFeeder(m: Merger, i: number, body: Body, stopId: string): boolean {
  const others = m.feederIds.map((_, j) => `feed:${m.id}:${j}`).filter((id) => id !== stopId &&
```

```
waitingFeed.has(id));
  let wait = !clear(m, body);
  if (!wait && m.kind === "side" && m.priority === "alternate" && waitingMain.has(m.id) &&
lastFrom.get(m.id) !== "main") wait = true;
  if (!wait && m.kind === "y" && others.length && lastFrom.get(m.id) === stopId) wait = true;
// the other feeder's turn
  if (wait) {
    waitingFeed.set(stopId, body.id!);
    return true;
  }
  const at = mergePoint(m)!;
  const f = beltById(m.feederIds[i])!;
  body.guide = { x: at.x, z: at.z, y: at.y, speed: Math.max(0.2, Math.abs(f.speed)) };
  waitingFeed.delete(stopId);
  lastFrom.set(m.id, stopId);
  return false;
}

function holdMain(m: Merger, body: Body): boolean {
  const feederWaiting = m.feederIds.some((_, j) => waitingFeed.has(`feed:${m.id}:${j}`));
  const at = mergePoint(m);
  const merging = at && [...bodies.values()].some((b) => b.guide && Math.hypot(b.guide.x -
at.x, b.guide.z - at.z) < 1e-6);
  const wait = (feederWaiting || !!merging) && (m.priority === "feeders" || lastFrom.get(m.id)
=== "main" || merging);
  if (wait) {
    waitingMain.set(m.id, body.id!);
    return true;
  }
  waitingMain.delete(m.id);
  if (feederWaiting) lastFrom.set(m.id, "main");
  return false;
}

function holdMachine(mc: Machine, body: Body): boolean {
  const shape = world().shapes.find((s) => s.id === body.id);
  if (!shape) return false;
  const finished = done.get(mc.id) ?? new Set<string>();
  if (finished.has(body.id!)) return false;
  const slot = slots.get(mc.id) ?? { progress: 0 };
  slots.set(mc.id, slot);
```

```
  if (slot.base === body.id || slot.attach === body.id) return true;
  const owner = ownerOf(shape);
  if (owner === mc.base.owner && !slot.base) slot.base = body.id;
  else if (owner === mc.attach.owner && !slot.attach) slot.attach = body.id;
  else return false;
  return true;
}

function hold(body: Body, stop: StopLine): boolean {
  const [kind, id, index] = stop.id.split(":");
  if (kind === "machine") {
    const mc = world().machines.find((x) => x.id === id);
    return mc ? holdMachine(mc, body) : false;
  }
  const m = world().mergers.find((x) => x.id === id);
  if (!m) return false;
  return kind === "feed" ? holdFeeder(m, Number(index), body, stop.id) : holdMain(m, body);
}
```

Main belt first: a feeder item goes only when it's clear; the main belt never stops. **Feeders first:** main-belt items stop at their line while any feeder item waits or is merging. **Take turns:** `lastFrom` remembers who went last, and each side yields when the other is waiting and went less recently. A Y merger's two feeders take turns the same way.

Releasing a feeder item sets its guide to the merge point and records the turn. Releasing a main item while a feeder waits records that the main belt went. Every decision is a few comparisons on the current step's bodies: no timers, so it behaves the same at any frame rate.

Machines

src/line/stations.ts · lines 173–187

```
function holdMachine(mc: Machine, body: Body): boolean {
  const shape = world().shapes.find((s) => s.id === body.id);
  if (!shape) return false;
  const finished = done.get(mc.id) ?? new Set<string>();
  if (finished.has(body.id!)) return false;
  const slot = slots.get(mc.id) ?? { progress: 0 };
  slots.set(mc.id, slot);
  if (slot.base === body.id || slot.attach === body.id) return true;
  const owner = ownerOf(shape);
  if (owner === mc.base.owner && !slot.base) slot.base = body.id;
  else if (owner === mc.attach.owner && !slot.attach) slot.attach = body.id;
  else return false;
  return true;
}
```

src/line/stations.ts · lines 209–248

```
function afterStep(dt: number): Join[] {
  const joins: Join[] = [];
  const { machines, shapes, connectors } = world();
  for (const mc of machines) {
    const slot = slots.get(mc.id);
    if (!slot || (!slot.base && !slot.attach)) {
      status.set(mc.id, { state: "idle" });
      continue;
    }
    if (!slot.base || !slot.attach) {
      status.set(mc.id, { state: "waiting", has: slot.base ? "base" : "attach" });
      continue;
    }
    slot.progress += dt;
    if (slot.progress < mc.cycleSeconds) {
      status.set(mc.id, { state: "joining", progress: slot.progress / mc.cycleSeconds });
      continue;
    }
    const base = shapes.find((s) => s.id === slot.base)!;
    const attached = shapes.find((s) => s.id === slot.attach)!;
    const bp = findPoint(mc.base.owner, mc.base.point, connectors);
    const ap = findPoint(mc.attach.owner, mc.attach.point, connectors);
```

```
    const finished = done.get(mc.id) ?? new Set<string>();
    done.set(mc.id, finished);
    finished.add(base.id).add(attached.id);
    slots.set(mc.id, { progress: 0 });
    if (!bp || !ap) {
      // A point was renamed or removed since the machine was set up: let both go, unjoined.
      status.set(mc.id, { state: "error", message: `No connection point "${!bp ? mc.base.point
: mc.attach.point}" any more` });
      continue;
    }
    joins.push({
      machineId: mc.id, machineName: mc.name, baseId: base.id, attachedId: attached.id,
      basePoint: bp.name, attachedPoint: ap.name, attach: attachmentFor(base, bp, attached,
ap),
    });
    status.set(mc.id, { state: "idle" });
  }
  return joins;
}
```

A machine holds the first item of its base kind and the first of its attach kind; anything else, including a second item of a kind it already holds, passes under it. (Queueing those would be more realistic, but on a single belt it can deadlock: a second base waiting in front of the attach item the machine needs.) With both, it counts up `cycleSeconds`, then joins them with `attachmentFor` and remembers both in `done` so they aren't caught again.

If a point named in the machine no longer exists (renamed in the editor since), it lets both go unjoined and shows the reason, rather than guessing a point.

tests/stations.test.ts · lines 44–98

```
test("a feeder item waits at the end of its belt, then slides onto the main belt's centerline",
() => {
  const f = body("f", 0, 1.1, 0.35);
  run(world({}), new Map([["f", f]]), 3);
  assert.ok(onMainCenter(f), `on the main belt's centerline: z ${f.z}, y ${f.y}`);
  assert.ok(f.x > 0.3, `then carried along it: x ${f.x}`);
});

test("main priority: a feeder item waits for a gap, and the main belt never stops", () => {
  const passing = body("p", -0.6, 0, 0.3); // reaches the merge point just as the feeder item
arrives
  const f = body("f", 0, 0.75, 0.35);
```

```
  let heldWhilePassing = false;
  let mainStopped = false;
  let last = passing.x;
  run(world({}), new Map([["p", passing], ["f", f]]), 3, () => {
    const u = passing.x; // main is unturned: u = x
    if (Math.abs(u) < 0.3 && f.heldAt) heldWhilePassing = true;
    if (passing.x === last && passing.x < 0.9) mainStopped = true;
    last = passing.x;
  });
  assert.ok(heldWhilePassing, "the feeder item was held while a main item was within the gap");
  assert.ok(!mainStopped, "the main item never stopped");
  assert.ok(onMainCenter(f) && passing.x - f.x >= 0.3 - 1e-6, `merged behind it with the gap:
${passing.x} vs ${f.x}`);
});

test("a busy feeder queues, and its items merge one at a time, spaced by the gap", () => {
  const items = ["a", "b", "c"].map((id, i) => body(id, 0, 0.75 + i * 0.12, 0.35));
  const bodies = new Map(items.map((b) => [b.id!, b] as const));
  let queued = false;
  const longMain = belt("main", { length: 4, position: { x: 1, z: 0 } }); // starts at x = -1
too, so the merge point is still x = 0
  run(world({ belts: [longMain, feeder] }), bodies, 5, () => {
    // While waiting, no two items on the feeder overlap.
    const onFeeder = items.filter((b) => Math.abs(b.y - 0.35) < 1e-6).map((b) => toBelt(feeder,
b.x, b.z).u).sort((x, y) => x - y);
    for (let i = 1; i < onFeeder.length; i++) assert.ok(onFeeder[i] - onFeeder[i - 1] >= 0.1 -
1e-6, "spaced at least one item apart");
    if (items.some((b) => b.heldAt === "queue")) queued = true;
  });
  assert.ok(queued, "items queued behind the waiting one");
  const xs = items.map((b) => b.x).sort((x, y) => y - x);
  assert.ok(items.every(onMainCenter), "all three merged");
  assert.ok(xs[0] - xs[1] >= 0.3 - 1e-6 && xs[1] - xs[2] >= 0.3 - 1e-6, `spaced by the gap on
the main belt: ${xs}`);
});

test("feeders priority: main-belt items stop short of the merge point while a feeder item
waits", () => {
  const m = body("m", -0.9, 0, 0.3);
  const f = body("f", 0, 0.75, 0.35);
  let mainHeld = false;
```

```
  run(world({ mergers: [merger({ priority: "feeders" })] }), new Map([["m", m], ["f", f]]), 4,
() => {
    if (m.heldAt === "main:m1") {
      mainHeld = true;
      assert.ok(Math.abs(m.x - -0.3) < 1e-6, `held a gap short of the merge point: ${m.x}`);
    }
  });
  assert.ok(mainHeld);
  assert.ok(f.x > m.x, "the feeder item went first");
});
```

The tests run real scenarios through `step`: a feeder item held while a main item passes within the gap, and the main item never stopping; three feeder items queuing without overlapping and merging spaced by the gap; main items stopping exactly a gap short under “feeders first”; and a Y merger taking items in the order A, B, A, B.

PART 1.23 · **src/line/controller.ts**

Running the line with machines, and the Machines tab

How joins made during a run are drawn, saved and turned into assemblies, and the tab where mergers and machines are set up.

Joins during a run

src/conveyors/controller.ts · lines 151–253

```
function start() {
  if (bodies) return;
  setMode({ kind: "idle" });
  bodies = new Map();
  started = new Map();
  links = new Map();
  joins = [];
  line?.stations.reset();
  storedAtStart = new Map(shapes.shapes.filter((s) => s.binId && !s.attachedTo).map((s) =>
[s.id, s.binId!]));
  // Shapes joined onto another ride on it, and stored ones are in their bins: only the ones
  // standing on their own get a body.
  for (const s of shapes.shapes) {
    if (s.attachedTo || s.binId) continue;
    bodies.set(s.id, {
```

```
      id: s.id, x: s.position.x, z: s.position.z, y: supportHeight(belts, s.position.x,
s.position.z),
      vx: 0, vy: 0, vz: 0, size: Math.max(s.size.x, s.size.z),
    });
    started.set(s.id, { ...s.position });
  }
  shapes.setRunning(true);
  message = null;
  render();
}

async function stop() {
  if (!bodies) return;
  const moves: { id: string; x: number; z: number }[] = [];
  // Shapes joined during the run are saved where they ended up too, so if the assembly is
  // refused (not enough stock), they're left at the machine rather than back at the start.
  const where = currentPoses();
  const storedNow = new Map(line?.stations.stored() ?? []);
  for (const [id, from] of started) {
    const p = where.get(id);
    if (!p || storedNow.has(id)) continue; // ended up in a bin: its position doesn't matter
    // Millimeters are plenty, and the API keeps whatever we send.
    const x = Math.round(p.x * 1000) / 1000;
    const z = Math.round(p.z * 1000) / 1000;
    if (Math.hypot(x - from.x, z - from.z) >= 0.001) moves.push({ id, x, z });
  }
  // What changed in the bins: items that left one (saved with where they ended up), and items
now in one.
  const round = (v: number) => Math.round(v * 1000) / 1000;
  const releases = [...storedAtStart].filter(([id, bin]) => storedNow.get(id) !== bin &&
where.has(id))
    .map(([shapeId, binId]) => ({ binId, shapeId, x: round(where.get(shapeId)!.x), z:
round(where.get(shapeId)!.z) }));
  const catches = [...storedNow].filter(([id, bin]) => storedAtStart.get(id) !==
bin).map(([shapeId, binId]) => ({ binId, shapeId }));
  const made = joins;
  bodies = null;
  joins = [];
  line?.stations.reset();
  line?.frame(0); // lights back to idle
  shapes.setRunning(false);
```

```
  if (moves.length === 0 && made.length === 0 && !releases.length && !catches.length) message =
{ text: "Nothing moved. Shapes ride a belt when placed on top of it.", error: false };
  render();
  // Releases first (they put items back on the floor), joins next, catches last (an item can
  // be joined and then caught as one assembly in the same run).
  if (releases.length && line) await line.saveReleases(releases);
  await shapes.saveMoves(moves);
  if (made.length && line) await line.saveJoins(made);
  if (catches.length && line) await line.saveCatches(catches);
  if (line && (releases.length || made.length || catches.length)) await shapes.reload();
}

/** Where every shape is now: bodies for the free ones, and joined ones riding on them. */
function currentPoses() {
  return poses(shapes.shapes, (s) => {
    const b = bodies?.get(s.id);
    return b
      ? { x: b.x, y: b.y + s.size.y / 2, z: b.z, rotationY: s.rotationY }
      : { x: s.position.x, y: supportHeight(belts, s.position.x, s.position.z) + s.size.y / 2,
z: s.position.z, rotationY: s.rotationY };
  }, links);
}

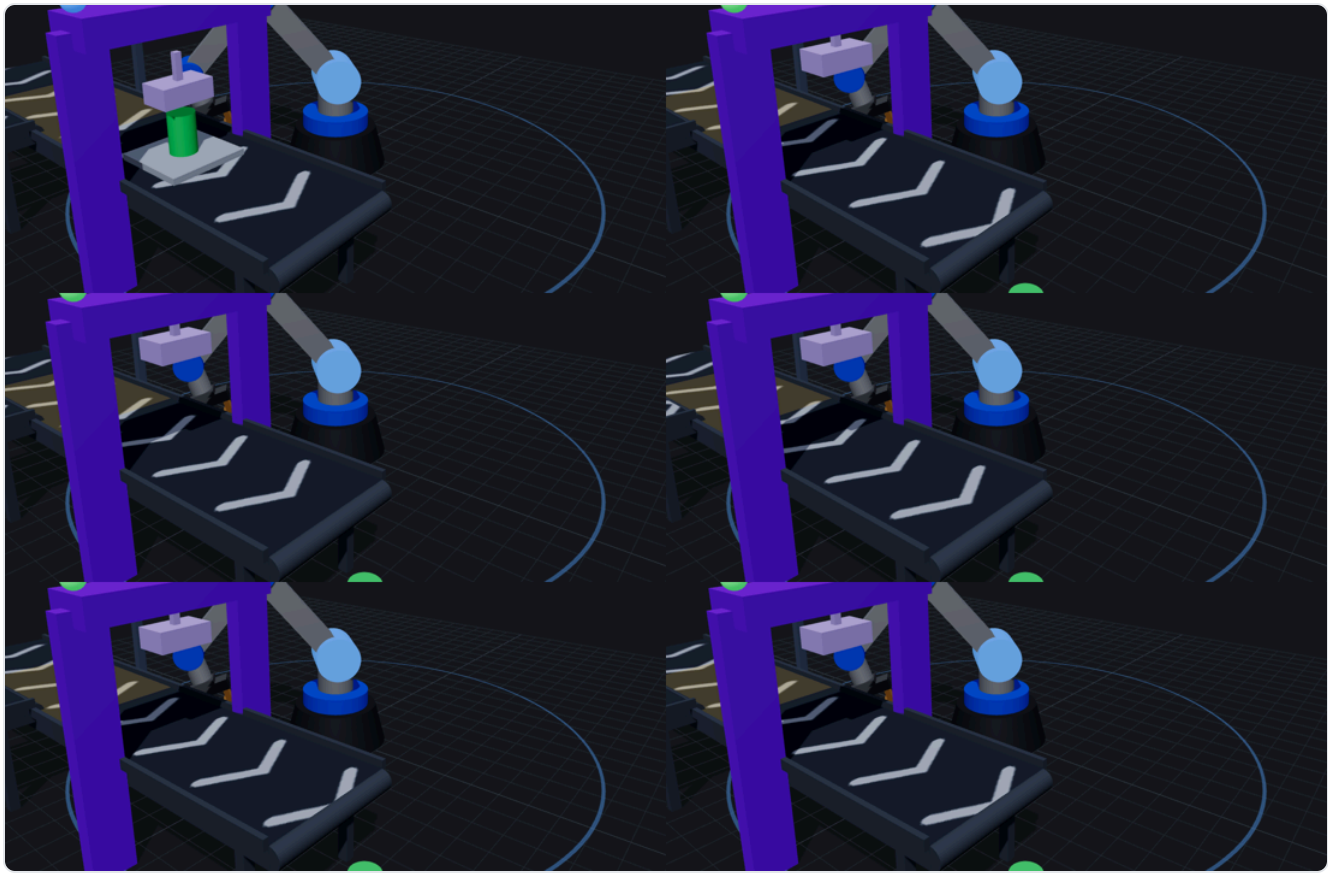
/** Called every frame by main.ts. */
function update(dt: number) {
  if (!bodies) return;
  layer.animate(dt);
  const st = line?.stations;
  const options = {
    gates: gates.list(), route: (b: Body, g: Gate) => gates.route(b.id!, g),
    stops: st?.stops() ?? [], hold: st?.hold,
  };
  for (let left = dt; left > 1e-9; left -= MAX_STEP) {
    const h = Math.min(MAX_STEP, left);
    st?.prepare(bodies);
    step(bodies.values(), belts, h, options);
    for (const j of st?.afterStep(h) ?? []) {
      // The attached shape stops being a free body: from now on it rides on the base.
      bodies.delete(j.attachedId);
      links.set(j.attachedId, { parentId: j.baseId, attach: j.attach });
      joins.push(j);
```

```
    }
    const bins = st?.binStep(h, bodies);
    for (const c of bins?.caught ?? []) bodies.delete(c.shapeId);
    for (const r of bins?.released ?? []) bodies.set(r.body.id!, r.body);
  }
  const stored = st?.stored() ?? new Map();
  for (const [id, p] of currentPoses()) {
    shapes.layer.poseCenter(id, p);
    shapes.layer.setVisible(id, !stored.has(rootOf(id, shapes.shapes, links)));
  }
  line?.frame(dt);
}

runButton.addEventListener("click", () => (bodies ? void stop() : start()));
```

At Run, only free shapes get bodies; shapes already joined ride on theirs. Each step, the stations prepare, the transport steps with their stop lines, and any joins that finish turn the attached item from a body into a **link**: it stops moving by itself and rides on its base. Every frame, `currentPoses` (the `poses` function from the joining part) places every mesh, through any chain of joins.

At Stop, positions are saved first, including where joined items ended up, so that if an assembly is refused they stay at the machine instead of jumping back to where they started. Then each join is saved as an assembly.



A run filmed in front of the camera, panels hidden: the press comes down on the can as it's joined to the plate (top left); then the joined pair carries on and the press goes back up.

Saving assemblies

src/line/controller.ts · lines 142–164

```
/** After a run: save each join as an assembly. The API uses up the parts, or refuses the lot.
*/
async function saveJoins(joins: readonly Join[]) {
  const lines: string[] = [];
  let made = 0;
  for (const j of joins) {
    const base = shapes.shapes.find((s) => s.id === j.baseId);
    const attached = shapes.shapes.find((s) => s.id === j.attachedId);
    const name = `${base?.name ?? "?"} + ${attached?.name ?? "?"}`.slice(0, 40);
    try {
      const a = await lineApi.assemble({
        name, baseShapeId: j.baseId, attachedShapeId: j.attachedId, basePoint: j.basePoint,
attachedPoint: j.attachedPoint, attachment: j.attach,
      });
      made++;
      if (a.consumed.length) lines.push(`${name}: used ${a.consumed.map((c) => `${c.qty} ×
${c.sku}`).join(", ")}`);
    } catch (err) {
      const e = err as ShapesApiError & { message: string };
      lines.push(`${name}: not joined. ${e.message}`);
    }
  }
  message = { text: `Saved ${made} assembl${made === 1 ? "y" : "ies"}.${lines.length ? " " +
lines.join(". ") + "." : ""}`, error: made < joins.length };
  await load();
}
```

Each join becomes one `POST /api/assemblies`. The API does everything else in one operation: attaches the shape, uses up each part once and writes the ledger (Chapter 5). A refusal, usually not enough stock, is reported per join, and that join simply doesn't happen in the database.

Putting feeders in place

src/line/controller.ts · lines 192–227

```
// ---- Putting feeders in place ----

/** Turns and moves a belt so it runs along `travel` (a world direction) and ends at `end`. */
function aim(f: Conveyor, end: { x: number; z: number }, travel: { x: number; z: number }) {
```

```
  const d = f.speed >= 0 ? 1 : -1; // a reversed belt runs toward its -u end
  const rotationY = Math.round((Math.atan2(-(d * travel.z), d * travel.x) / DEG) * 10) / 10;
  const round = (v: number) => Math.round(v * 1000) / 1000;
  return conveyorsApi.update(f.id, {
    rotationY,
    position: { x: round(end.x - (travel.x * f.length) / 2), z: round(end.z - (travel.z *
f.length) / 2) },
  });
}

async function snapFeeders(kind: MergerKind, main: Conveyor, along: number, feeders:
Conveyor[]) {
  const t = main.rotationY * DEG;
  const udir = { x: Math.cos(t), z: -Math.sin(t) }; // the main belt's +u, in the world
  const vdir = { x: Math.sin(t), z: Math.cos(t) }; // and its +v
  const d = main.speed >= 0 ? 1 : -1;
  if (kind === "side") {
    // Square onto the side it's already on, ending at the main belt's edge.
    const f = feeders[0];
    const side = toBelt(main, f.position.x, f.position.z).v >= 0 ? 1 : -1;
    const end = fromBelt(main, along - main.length / 2, side * (main.width / 2));
    await aim(f, end, { x: -side * vdir.x, z: -side * vdir.z });
    return;
  }
  // A y: the two feeders come in from behind the output's start, at ±Y_ANGLE.
  const start = -d * (main.length / 2);
  for (const [i, f] of feeders.entries()) {
    const side = i === 0 ? -1 : 1;
    const end = fromBelt(main, start - d * 0.02, side * main.width * 0.3);
    const a = Y_ANGLE * DEG;
    const travel = { x: d * udir.x * Math.cos(a) - side * vdir.x * Math.sin(a), z: d * udir.z *
Math.cos(a) - side * vdir.z * Math.sin(a) };
    await aim(f, end, travel);
  }
}
```

Placing a feeder by hand so it meets a belt exactly is fiddly, so the tab can do it. For a side merger the feeder is turned to run square onto the main belt, from the side it's already on, and moved so its end touches the main belt's edge at the merge point. For a Y, the two feeders come in at ±35° from

behind the output's start. `aim` turns a direction into a belt angle, allowing for belts that run backwards, and moves the belt's center half its length back from the end. It's done through the same conveyor API as any other move, so the belts are saved where they're shown.

The models

src/line/LineLayer.ts · lines 117–161

```
function buildMerger(main: Conveyor, u: number, side: number, gap: number) {
  const group = new THREE.Group();
  const p = fromBelt(main, 0, 0);
  group.position.set(p.x, 0, p.z);
  group.rotation.y = main.rotationY * DEG;

  const zone = new THREE.Mesh(
    new THREE.PlaneGeometry(Math.min(2 * gap, main.length), main.width).rotateX(-Math.PI / 2),
    new THREE.MeshBasicMaterial({ color: AMBER, transparent: true, opacity: 0.22, depthWrite:
false }),
  );
  zone.position.set(u, main.height + 0.004, 0);
  group.add(zone);

  const post = new THREE.MeshStandardMaterial({ color: 0x475569, roughness: 0.6 });
  const top = main.height + 0.35;
  const z = side * (main.width / 2 + 0.07);
  box(0.03, top, 0.03, post, u, top / 2, z, group);
  const light = new THREE.MeshStandardMaterial({ color: GREEN, emissive: GREEN,
emissiveIntensity: 0.8 });
  const bulb = new THREE.Mesh(new THREE.SphereGeometry(0.035, 16, 12), light);
  bulb.position.set(u, top + 0.03, z);
  group.add(bulb);
  return { group, light };
}

function buildMachine(belt: Conveyor, along: number) {
  const group = new THREE.Group();
  const p = fromBelt(belt, along - belt.length / 2, 0);
  group.position.set(p.x, 0, p.z);
  group.rotation.y = belt.rotationY * DEG;

  const frame = new THREE.MeshStandardMaterial({ color: 0x7c3aed, roughness: 0.5, metalness:
0.2 });
```

```
  const w = belt.width;
  const top = belt.height + 0.42;
  for (const s of [-1, 1]) box(0.05, top, 0.12, frame, 0, top / 2, s * (w / 2 + 0.06), group);
  box(0.16, 0.08, w + 0.17, frame, 0, top, 0, group);
  const headMat = new THREE.MeshStandardMaterial({ color: 0xc4b5fd, roughness: 0.4, metalness:
0.3 });
  const headTop = top - 0.12;
  const head = box(0.1, 0.06, Math.min(0.16, w * 0.6), headMat, 0, headTop, 0, group);
  box(0.02, 0.1, 0.02, headMat, 0, 0.06, 0, head); // the rod up to the beam
  const light = new THREE.MeshStandardMaterial({ color: GREEN, emissive: GREEN,
emissiveIntensity: 0.8 });
  const bulb = new THREE.Mesh(new THREE.SphereGeometry(0.03, 16, 12), light);
  bulb.position.set(0, top + 0.07, w / 2 + 0.06);
  group.add(bulb);
  return { group, light, head, headTop };
}
```

A merger's post stands on the side away from the feeder, with a light that turns red while it holds anything; a translucent patch on the main belt shows the gap it keeps clear. A machine is a portal over the belt whose head comes down and up over a cycle (`sin(π · progress)`), with a light for idle, waiting, joining or error. Both are built in their belt's frame, like the splitter gates.

PART 1.24 · `src/line/stations.ts`

Bins: catching and feeding

Bins hold the kinds of object you choose. They catch items that fall off belts and splitters, and put items back on the line: onto a belt, or straight into a joining machine. Like mergers and machines they're stations, decided each step from the current state.

The model

src/line/types.ts · lines 85–110

```
export type FeedMode = "steady" | "demand";
export type Feed =
  | { type: "none" }
  | { type: "belt"; conveyorId: string; along: number; mode: FeedMode }
  | { type: "machine"; machineId: string };

export interface Bin {
  id: string;
```

```
  name: string;
  /** The kinds of object it holds: one or more. */
  holds: Owner[];
  /** Extra conditions an item must meet to be caught (all of them). */
  accept: Condition[];
  capacity: number;
  position: { x: number; z: number };
  rotationY: number;
  feed: Feed;
  count: number;
  createdAt: string;
  updatedAt: string;
}
export type NewBin = Omit<Bin, "id" | "count" | "createdAt" | "updatedAt">;
export type BinPatch = Partial<Pick<Bin, "name" | "holds" | "accept" | "capacity" | "position"
| "rotationY" | "feed">>;

/** Bins are this size, meters (the API's BIN_SIZE). */
export const BIN_SIZE = { x: 0.5, y: 0.35, z: 0.4 } as const;
```

A bin **holds** a list of kinds (parts and plain shape kinds) and may add **accept** conditions on top, the same conditions a splitter rule uses. Its **feed** is one of three shapes, written as a discriminated union so a feed can't be half one thing and half another: nothing, a point on a belt with a mode, or a machine.

A stored item is just a shape with `binId` set. It keeps its identity, its part and anything joined onto it, so an assembly can go into a bin and come out again whole.

Catching

src/line/stations.ts · lines 278–320

```
// ---- Bins ----

/** Is a machine's slot for this kind of object empty? */
function needs(mc: Machine, owner: string): boolean {
  const slot = slots.get(mc.id);
  return (mc.base.owner === owner && !slot?.base) || (mc.attach.owner === owner &&
!slot?.attach);
}

function inBin(b: Bin, x: number, z: number): boolean {
  const p = toBelt({ ...BIN_SIZE, length: BIN_SIZE.x, width: BIN_SIZE.z, height: BIN_SIZE.y,
```

```
speed: 0, position: b.position, rotationY: b.rotationY }, x, z);
  return Math.abs(p.u) <= BIN_SIZE.x / 2 && Math.abs(p.v) <= BIN_SIZE.z / 2;
}

/**
 * After each step: catch items that came to rest in a bin, and release items from bins that
 * feed the line. Returns the ids caught (the caller removes their bodies) and the new bodies.
 */
function binStep(dt: number, current: Map<string, Body>): { caught: { binId: string; shapeId:
string }[]; released: { binId: string; body: Body }[] } {
  const { bins = [], shapes, parts = [], machines } = world();
  const caught: { binId: string; shapeId: string }[] = [];
  const released: { binId: string; body: Body }[] = [];
  const shapeOf = (id: string) => shapes.find((s) => s.id === id);

  for (const body of current.values()) {
    if (body.y > 1e-4 || body.vy !== 0 || body.guide) continue; // only items lying on the
floor
    const bin = bins.find((b) => inBin(b, body.x, body.z));
    if (!bin) {
      refused.delete(body.id!);
      continue;
    }
    if (refused.has(body.id!)) continue;
    const shape = shapeOf(body.id!);
    const part = shape?.sku ? parts.find((p) => p.sku === shape.sku) : undefined;
    const queue = binQueue.get(bin.id) ?? [];
    const ok = shape && bin.holds.includes(ownerOf(shape)) && bin.accept.every((c) =>
matches(c, shape, part)) && queue.length < bin.capacity;
    if (!ok) {
      refused.add(body.id!);
      continue;
    }
    binQueue.set(bin.id, [...queue, body.id!]);
    binOf.set(body.id!, bin.id);
    caught.push({ binId: bin.id, shapeId: body.id! });
  }
```

A bin is only a footprint on the floor. An item **lying on the floor** (not falling, not on a belt, not being guided) inside that footprint is offered to the bin: it's caught if one of the bin's kinds matches it, every accept condition holds (`matches` from the splitter rules, with the part looked up live) and

there's room. Otherwise it's remembered as refused, so it isn't asked again every step until it leaves.

The test is in the bin's own frame, with `toBelt` reused for the rotation: a turned bin is an axis-aligned rectangle in its own coordinates, exactly like a turned belt.

Feeding the line

src/line/stations.ts · lines 321–389

```
  for (const bin of bins) {
    const queue = binQueue.get(bin.id) ?? [];
    const wait = (cooldown.get(bin.id) ?? 0) - dt;
    cooldown.set(bin.id, wait);
    if (!queue.length || wait > 0 || bin.feed.type === "none") continue;
    let target: { belt: Conveyor; u: number } | null = null;
    /** Which stored items may go: by default any (the oldest goes first). */
    let wanted = (_owner: string) => true;

    if (bin.feed.type === "belt") {
      const belt = beltById(bin.feed.conveyorId);
      if (!belt) continue;
      const u = bin.feed.along - belt.length / 2;
      const onBelt = [...current.values()].filter((b) => Math.abs(b.y - belt.height) < 1e-3 &&
isOver(belt, b.x, b.z));
      if (onBelt.some((b) => Math.abs(toBelt(belt, b.x, b.z).u - u) < 0.25)) continue; // the
spot isn't clear
      if (bin.feed.mode === "demand") {
        // Only kinds a machine further along needs, with none already on its way to it.
        const d = dirOf(belt);
        wanted = (owner) => machines.some((mc) => {
          if (mc.conveyorId !== belt.id || !needs(mc, owner)) return false;
          const mu = mc.along - belt.length / 2;
          if ((mu - u) * d <= 0) return false;
          return !onBelt.some((b) => {
            const s = shapeOf(b.id!);
            const bu = toBelt(belt, b.x, b.z).u;
            return s && ownerOf(s) === owner && (bu - u) * d >= 0 && (mu - bu) * d >= 0;
          });
        });
      }
      target = { belt, u };
```

```
    } else {
      const machineId = bin.feed.machineId;
      const mc = machines.find((m) => m.id === machineId);
      const belt = mc && beltById(mc.conveyorId);
      if (!mc || !belt) continue;
      wanted = (owner) => needs(mc, owner);
      const pending = placed.get(bin.id);
      if (pending && current.has(pending) && current.get(pending)!.heldAt !==
`machine:${mc.id}`) continue; // still arriving
      // Just upstream of the machine's line, so the next step carries it across and the
machine takes it.
      target = { belt, u: mc.along - belt.length / 2 - dirOf(belt) * 0.01 };
    }

    // Drop anything no longer in the scene, then take the oldest item that's wanted.
    const live = queue.filter((id) => shapeOf(id));
    binQueue.set(bin.id, live);
    const id = live.find((sid) => wanted(ownerOf(shapeOf(sid)!)));
    if (!id) continue;
    const shape = shapeOf(id)!;
    const at = fromBelt(target.belt, target.u, 0);
    const body: Body = { id, x: at.x, z: at.z, y: target.belt.height, vx: 0, vy: 0, vz: 0,
size: Math.max(shape.size.x, shape.size.z) };
    binQueue.set(bin.id, live.filter((sid) => sid !== id));
    binOf.delete(id);
    if (bin.feed.type === "machine") placed.set(bin.id, id);
    cooldown.set(bin.id, 0.4);
    released.push({ binId: bin.id, body });
  }
  return { caught, released };
}

/** Where every stored item is now: shape id -> bin id. */
function stored(): ReadonlyMap<string, string> {
  return binOf;
}

function binCounts(): Map<string, number> {
  return new Map([...binQueue].map(([id, q]) => [id, q.length]));
}
```

Each bin with a feed and something in it may release one item, at most every 0.4 s. Which item is a filter: `wanted(owner)`. For a steady belt feed anything is wanted, and the oldest goes. **On demand**, a kind is wanted only if a machine further along that belt has an empty slot for it and no item of that kind is already between the bin's spot and the machine; without that last check the bin would keep sending items while the first is still on its way. For a machine feed, a kind is wanted when the machine's slot for it is empty. A bin with several kinds sends its **oldest wanted** item.

A belt feed also waits for its spot to be clear (nothing within 25 cm), so items go on spaced. A machine feed puts the item a centimeter before the machine's line; the next step carries it across, and the machine holds it like any other arrival.

tests/stations.test.ts · lines 181–236

```
// ---- Bins ----

const bin = (over: Partial<Bin> = {}): Bin => ({
  id: "bin1", name: "Bin", holds: ["kind:box"], accept: [], capacity: 10, position: { x: 1.3,
z: 0 }, rotationY: 0,
  feed: { type: "none" }, count: 0, createdAt: T, updatedAt: T, ...over,
});

function runBins(w: World, bodies: Map<string, Body>, seconds: number) {
  const st = createStations(() => w);
  st.reset();
  const caught: string[] = [];
  const released: string[] = [];
  const joins: Join[] = [];
  const dt = 1 / 120;
  for (let t = 0; t < seconds; t += dt) {
    st.prepare(bodies);
    step(bodies.values(), w.belts, dt, { stops: st.stops(), hold: st.hold });
    for (const j of st.afterStep(dt)) {
      joins.push(j);
      bodies.delete(j.attachedId);
    }
    const r = st.binStep(dt, bodies);
    for (const c of r.caught) {
      caught.push(c.shapeId);
      bodies.delete(c.shapeId);
    }
    for (const x of r.released) {
      released.push(x.body.id!);
      bodies.set(x.body.id!, x.body);
```

```
    }
  }
  return { st, caught, released, joins };
}

test("a bin at the end of a belt catches its kind, and leaves anything else lying in it", () =>
{
  // The main belt ends at x = 1; the bin's footprint is x 1.05..1.55.
  const red = shape("red", { color: "#e11d48" });
  const blue = shape("blue", { color: "#2563eb" });
  const ball = shape("ball", { kind: "sphere" });
  const w = world({ mergers: [], shapes: [red, blue, ball], bins: [bin({ accept: [{ field:
"color", op: "eq", value: "#e11d48" }] })] });
  const bodies = new Map([["red", body("red", 0.5, 0, 0.3)], ["blue", body("blue", 0.2, 0,
0.3)], ["ball", body("ball", -0.1, 0, 0.3)]]);
  const { caught, st } = runBins(w, bodies, 4);
  assert.deepEqual(caught, ["red"]);
  assert.deepEqual([...st.stored()], [["red", "bin1"]]);
  assert.ok(bodies.has("blue") && bodies.has("ball"), "the others lie where they fell");
  assert.equal(bodies.get("blue")!.y, 0);
});

test("a full bin catches nothing more", () => {
  const items = ["a", "b", "c"].map((id) => shape(id));
  const w = world({ mergers: [], shapes: items, bins: [bin({ capacity: 2 })] });
  const bodies = new Map(items.map((s, i) => [s.id, body(s.id, 0.5 - i * 0.3, 0, 0.3)] as
const));
  const { caught } = runBins(w, bodies, 5);
  assert.deepEqual(caught, ["a", "b"]);
});
```

The tests run the real step: a red box caught while a blue one and a ball, which the bin doesn't accept, lie where they fell; a full bin refusing a third item; a steady feed releasing oldest first and never closer than the gap.

tests/stations.test.ts · lines 260–314

```
test("a bin wired to a machine supplies its slot, and the machine joins the item onto the
base", () => {
  const plate = shape("plate", { sku: "PL-200", size: { x: 0.2, y: 0.02, z: 0.2 } });
  const cans = ["c1", "c2"].map((id) => shape(id, { kind: "cylinder", size: { x: 0.08, y: 0.12,
z: 0.08 }, binId: "bin1" }));
```

```
  const w = world({
    mergers: [], machines: [machine], shapes: [plate, ...cans],
    bins: [bin({ holds: ["kind:cylinder"], position: { x: 0.2, z: 0.6 }, feed: { type:
"machine", machineId: "j1" } })],
  });
  const bodies = new Map([["plate", body("plate", -0.6, 0, 0.3)]]);
  const { joins, released, st } = runBins(w, bodies, 4);
  assert.deepEqual(joins.map((j) => [j.baseId, j.attachedId]), [["plate", "c1"]]);
  // As soon as its can slot is empty again, the machine gets the next can, which waits there
for the next plate.
  assert.deepEqual(released, ["c1", "c2"]);
  assert.equal(bodies.get("c2")!.heldAt, "machine:j1");
  assert.equal(st.binCounts().get("bin1"), 0);
});

test("a demand bin feeds a belt only when a machine further along needs its kind", () => {
  const plate = shape("plate", { sku: "PL-200" });
  const cans = ["c1", "c2", "c3"].map((id) => shape(id, { kind: "cylinder", binId: "bin1" }));
  const w = world({
    mergers: [], machines: [machine], shapes: [plate, ...cans],
    bins: [bin({ holds: ["kind:cylinder"], position: { x: -1.5, z: 0.6 }, feed: { type: "belt",
conveyorId: "main", along: 0.2, mode: "demand" } })],
  });
  const bodies = new Map([["plate", body("plate", -0.4, 0, 0.3)]]);
  const { released, joins } = runBins(w, bodies, 6);
  // One can for the one plate: after the join the machine's slot is empty again, and it asks
  // for another. That's demand: a can whenever the cylinder slot is empty and none is on its
way.
  assert.equal(joins.length, 1);
  assert.ok(released.length >= 1 && released.length <= 3, `${released}`);
  assert.equal(released[0], "c1");
});

test("a bin holding several kinds catches each of them, and hands a machine the kind it needs",
() => {
  // One bin holds plates and cans, wired to the machine. The machine needs one of each, so the
  // bin hands over its oldest plate, then a can; once the plate slot is full, it only sends
cans.
  const onBelt = shape("plate1", { sku: "PL-200", size: { x: 0.2, y: 0.02, z: 0.2 } });
  const stored = [
    shape("plate2", { sku: "PL-200", size: { x: 0.2, y: 0.02, z: 0.2 }, binId: "bin1",
```

```
createdAt: "2026-01-01" }),
    shape("can", { kind: "cylinder", size: { x: 0.08, y: 0.12, z: 0.08 }, binId: "bin1",
createdAt: "2026-01-02" }),
  ];
  const mixed = bin({ holds: ["part:PL-200", "kind:cylinder"], position: { x: 0.2, z: 0.6 },
feed: { type: "machine", machineId: "j1" } });
  const w = world({ mergers: [], machines: [machine], shapes: [onBelt, ...stored], bins:
[mixed] });
  const bodies = new Map([["plate1", body("plate1", -0.6, 0, 0.3)]]);
  const { released, joins } = runBins(w, bodies, 1.5);
  assert.deepEqual(released, ["plate2", "can"]);
  assert.deepEqual(joins.map((j) => [j.baseId, j.attachedId]), [["plate2", "can"]]);
  assert.ok(!bodies.get("plate1")!.heldAt, "the belt's own plate found the slot taken and
passed by");

  // Catching: both kinds go in; a ball doesn't.
  const out = bin({ id: "out", holds: ["kind:box", "kind:cylinder"], position: { x: 1.3, z: 0 }
});
  const items = [shape("b", {}), shape("c", { kind: "cylinder" }), shape("s", { kind: "sphere"
})];
  const bodies2 = new Map(items.map((s, i) => [s.id, body(s.id, 0.5 - i * 0.3, 0, 0.3)] as
const));
  const { caught } = runBins(world({ mergers: [], shapes: items, bins: [out] }), bodies2, 5);
  assert.deepEqual(caught.sort(), ["b", "c"]);
});
```

Feeding a machine shows the timing: once a can is joined the slot is empty again, so the next can goes straight in and waits for the next base. The two-kind test checks the “oldest wanted” rule: the bin sends its plate first (the machine needs a base), then a can.

Saving what changed in the bins

`src/conveyors/controller.ts · lines 184–209`

```
  if (!p || storedNow.has(id)) continue; // ended up in a bin: its position doesn't matter
  // Millimeters are plenty, and the API keeps whatever we send.
  const x = Math.round(p.x * 1000) / 1000;
  const z = Math.round(p.z * 1000) / 1000;
  if (Math.hypot(x - from.x, z - from.z) >= 0.001) moves.push({ id, x, z });
}
// What changed in the bins: items that left one (saved with where they ended up), and items
now in one.
```

```
const round = (v: number) => Math.round(v * 1000) / 1000;
const releases = [...storedAtStart].filter(([id, bin]) => storedNow.get(id) !== bin &&
where.has(id))
  .map(([shapeId, binId]) => ({ binId, shapeId, x: round(where.get(shapeId)!.x), z:
round(where.get(shapeId)!.z) }));
const catches = [...storedNow].filter(([id, bin]) => storedAtStart.get(id) !==
bin).map(([shapeId, binId]) => ({ binId, shapeId }));
const made = joins;
bodies = null;
joins = [];
line?.stations.reset();
line?.frame(0); // lights back to idle
shapes.setRunning(false);
if (moves.length === 0 && made.length === 0 && !releases.length && !catches.length) message = {
text: "Nothing moved. Shapes ride a belt when placed on top of it.", error: false };
render();
// Releases first (they put items back on the floor), joins next, catches last (an item can
// be joined and then caught as one assembly in the same run).
if (releases.length && line) await line.saveReleases(releases);
await shapes.saveMoves(moves);
if (made.length && line) await line.saveJoins(made);
if (catches.length && line) await line.saveCatches(catches);
if (line && (releases.length || made.length || catches.length)) await shapes.reload();
```

A run is still a preview. At Stop, the bins' final state is compared with their state at the start: an item that was stored and isn't (or is in another bin) is **released** at the place it ended up; an item that's stored now and wasn't is **caught**. An item that left a bin and came back to it costs nothing.

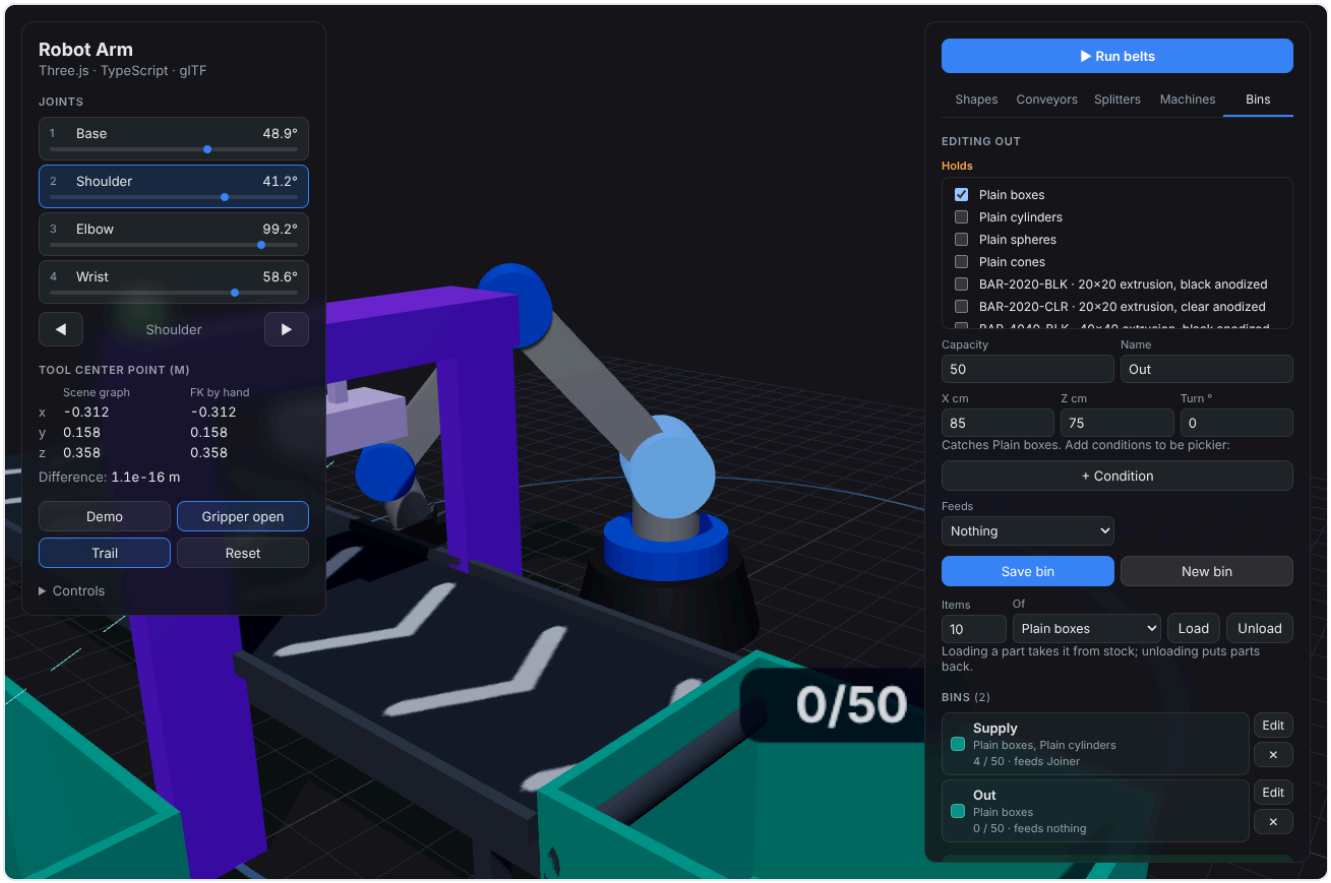
Order matters. Releases go first, so those items exist on the floor before anything else refers to them; then positions, then joins; catches go last, because an item can be joined and then caught as one assembly in the same run.

PART 1.25 · `src/line/BinPanel.ts`

The Bins tab, and loading many items at once

Bins are set up in their own tab: what they hold, where they stand, what they feed. Loading can be a list of specific items, or every kind that matches a rule you write.

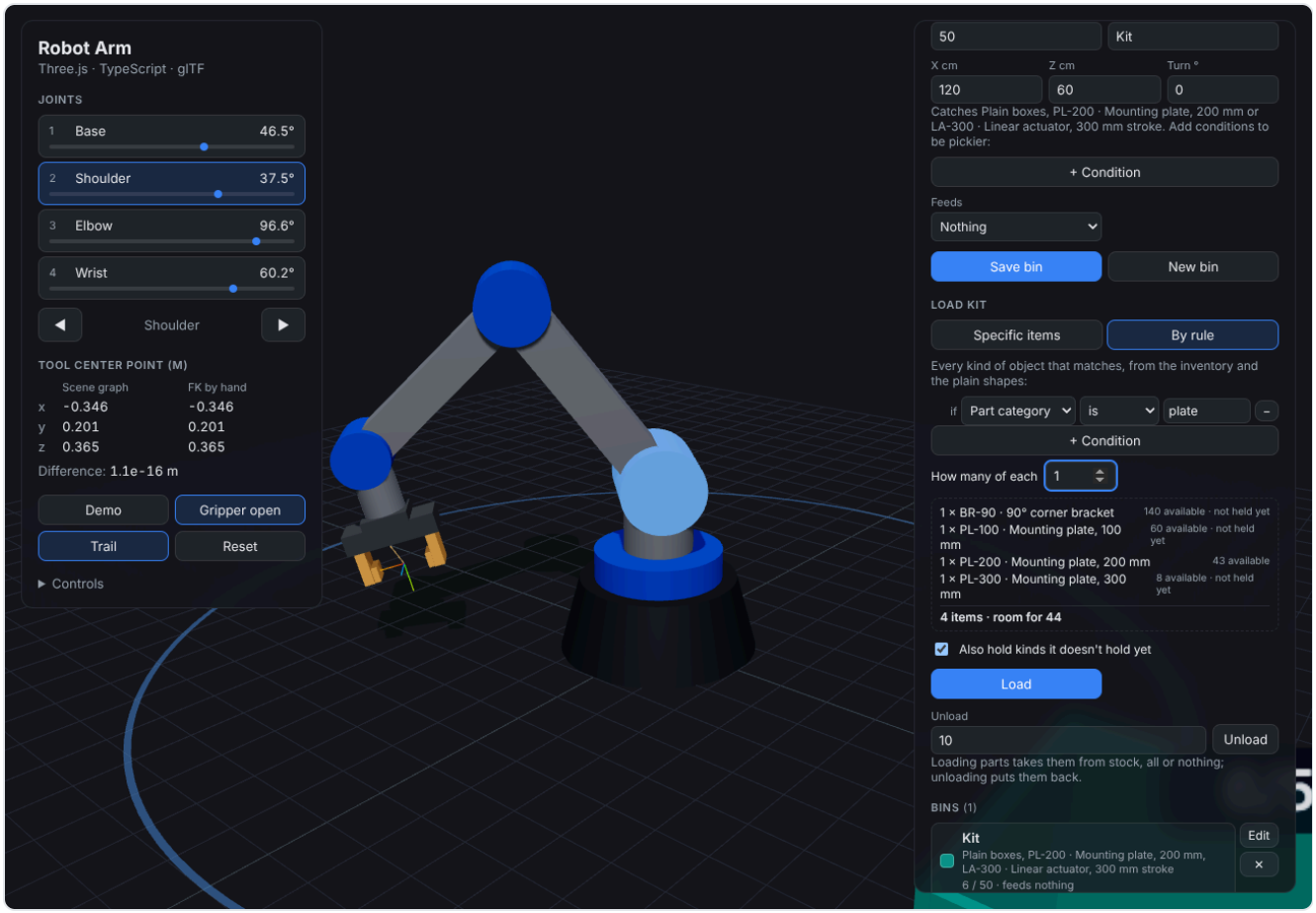
Setting up a bin



Editing a bin: tick the kinds it holds, add conditions, choose what it feeds. The list shows each bin's kinds, fill and feed.

Kinds are a checklist rather than a single choice, and a sentence under it says in words what the bin will catch (“Catches plain boxes or plain cylinders”). The accept conditions reuse the splitter's condition rows; each row carries `data-set` saying which list it edits, so the same rendering serves the accept conditions and the loading rule.

Specific items, or a rule



Loading by rule: every kind whose part category is plate, one of each. The preview shows what's available and which kinds the bin will start holding.

`src/line/BinPanel.ts · lines 413–451`

```
function candidates(): { owner: Owner; shape: Shape; part?: Part }[] {
  const kinds: Owner[] = [...SHAPE_KINDS.map((k) => `kind:${k}` as Owner), ...o.parts().map((p)
=> `part:${p.sku}` as Owner)];
  return kinds.flatMap((owner) => {
    const look = lookOf(owner, o.parts());
    if (!look) return [];
    const sku = owner.startsWith("part:") ? owner.slice(5) : null;
    const shape: Shape = { id: owner, name: look.name, kind: look.kind, size: look.size, color:
look.color, position: { x: 0, z: 0 }, rotationY: 0, sku, createdAt: "", updatedAt: "" };
    return [{ owner, shape, part: sku ? o.parts().find((p) => p.sku === sku) : undefined }];
  });
}

/** What Load would load: the listed items, or every kind matching the rule (none for an empty
rule). */
```

```
function loadPlan(): { owner: Owner; qty: number }[] {
  if (draft.loadMode === "items") {
    const merged = new Map<Owner, number>();
    for (const l of draft.loadLines) merged.set(l.owner, (merged.get(l.owner) ?? 0) + l.qty);
    return [...merged].map(([owner, qty]) => ({ owner, qty }));
  }
  if (!draft.loadRule.length) return [];
  return candidates().filter((c) => draft.loadRule.every((cond) => matches(cond, c.shape,
c.part))).map((c) => ({ owner: c.owner, qty: draft.ruleQty }));
}

/** The plan, kind by kind, with what's available and what the bin doesn't hold yet. */
function renderPreview() {
  const el = host.querySelector<HTMLElement>(".load-preview");
  const bin = o.bins().find((b) => b.id === editing);
  if (!el || !bin) return;
  const plan = loadPlan();
  const total = plan.reduce((n, l) => n + l.qty, 0);
  const room = bin.capacity - bin.count;
  el.innerHTML = !plan.length ? `<li class="empty">${draft.loadMode === "rule" ? "Nothing
matches yet." : "No items listed."}</li>`
    : plan.map((l) => {
        const part = l.owner.startsWith("part:") ? o.parts().find((p) => `part:${p.sku}` ===
l.owner) : undefined;
        const short = part && part.available < l.qty;
        const notHeld = !bin.holds.includes(l.owner);
        return `<li><span>${l.qty} × ${esc(ownerLabel(l.owner))}</span>
          <small class="${short ? "over" : ""}">${part ? `${part.available} available` : "no
stock needed"}${notHeld ? " · not held yet" : ""}</small></li>`;
      }).join("") + `<li class="total ${total > room ? "over" : ""}">${total} item${total === 1
? "" : "s"} · room for ${room}</li>`;
}
```

Specific items is a list of kind and quantity; the same kind listed twice is added together. **By rule** checks every kind there is (each inventory part, each plain shape kind) against the conditions. To do that, each kind becomes a **stand-in shape**, built from how that kind is drawn (`lookOf`) with its SKU, so the splitter's `matches` works unchanged: “part category is plate”, “available > 5” and “kind is cylinder” all mean what they say. An empty rule loads nothing rather than everything.

The preview is the plan, line by line: what's available for each part (in orange when it's short), which kinds the bin doesn't hold yet, and the total against the room left. It's the same `loadPlan()` the Load button sends, so what you see is what's loaded.

src/line/BinPanel.ts · lines 274–290

```
case "load": {
  const b = o.bins().find((x) => x.id === editing);
  const plan = loadPlan();
  if (!b) return;
  if (!plan.length) {
    message = { text: draft.loadMode === "rule" ? "Nothing matches that rule." : "Add an item
to load.", error: true };
    return render(true);
  }
  const lines = plan.map((l) => ({ owner: l.owner, qty: l.qty, look: lookOf(l.owner,
o.parts())! }));
  const total = lines.reduce((n, l) => n + l.qty, 0);
  const addToHolds = draft.addToHolds;
  return void act(async () => {
    // A load may add kinds to the bin; keep the form in step, or Save would take them off
again.
    draft.holds = (await binsApi.load(b.id, lines, addToHolds)).holds;
  },
    `Loaded ${total} item${total === 1 ? "" : "s"} (${lines.length} kind${lines.length === 1 ?
"" : "s"}) into ${b.name}; parts were taken from stock.`);
}
```

The whole plan goes in one request. The API loads all of it or none of it (next chapter), and returns the bin's kinds, which the form adopts: otherwise a Save straight after a load that added kinds would quietly take them off again.

Drawing bins

src/line/BinLayer.ts · lines 152–188

```
function build(b: Bin, color: string, target: { x: number; y: number; z: number } | null):
Model {
  const group = new THREE.Group();
  group.position.set(b.position.x, 0, b.position.z);
  group.rotation.y = b.rotationY * DEG;
  const rim = new THREE.MeshStandardMaterial({ color: 0x0d9488, roughness: 0.6, metalness: 0.1
});
  for (const m of walls(rim)) group.add(m);

  // The fill: a box scaled to the level, in the color of what the bin holds.
  const fill = new THREE.Mesh(
```

```
    new THREE.BoxGeometry(BIN_SIZE.x - 2 * WALL - 0.01, 1, BIN_SIZE.z - 2 * WALL - 0.01),
    new THREE.MeshStandardMaterial({ color, roughness: 0.8, transparent: true, opacity: 0.85
}),
  );
  group.add(fill);

  const canvas = document.createElement("canvas");
  canvas.width = 128;
  canvas.height = 48;
  const texture = new THREE.CanvasTexture(canvas);
  texture.colorSpace = THREE.SRGBColorSpace;
  const label = new THREE.Sprite(new THREE.SpriteMaterial({ map: texture, depthTest: false }));
  label.scale.set(0.24, 0.09, 1);
  label.position.set(0, BIN_SIZE.y + 0.1, 0);
  label.renderOrder = 10;
  group.add(label);

  let chute: THREE.Line | null = null;
  if (target) {
    // Drawn in the world, not the bin's frame: undo the group's transform for the target.
    group.updateMatrixWorld(true);
    const local = group.worldToLocal(new THREE.Vector3(target.x, target.y, target.z));
    const geometry = new THREE.BufferGeometry().setFromPoints([new THREE.Vector3(0, BIN_SIZE.y,
0), local]);
    chute = new THREE.Line(geometry, new THREE.LineDashedMaterial({ color: 0x2dd4bf, dashSize:
0.04, gapSize: 0.03 }));
    chute.computeLineDistances();
    group.add(chute);
  }
  return { group, fill, label: { canvas, texture }, chute, rim, key: "", shown: "" };
}
```

An open box, a fill block scaled to count over capacity, a sprite label with the count (a canvas texture, redrawn only when the text changes), and a dashed line to what the bin feeds. The line's far end is a world point, converted into the bin's own frame with `worldToLocal`, which needs the bin's matrix updated first since it isn't in the scene yet.

CHAPTER 1

Interview questions

Q1 **What's the difference between an object's local and world transform, and how does Three.js get from one to the other?**

Local is relative to the parent; world is relative to the scene. Three.js stores position, quaternion and scale, builds a local 4×4 matrix from them, and computes the world matrix as the parent's world matrix times the local one, recursively from the root. It does this lazily before rendering, or on demand with `updateWorldMatrix`. In the arm, each joint's local transform is just “up by the link length, rotated by my angle”; the hierarchy does the composing.

Q2 **Three.js already knows where the tool is. Why write forward kinematics by hand?**

Three reasons. It's an independent check: if the scene graph were built wrong, the two answers would disagree, and the panel shows the difference live. It runs without a renderer, so tests, IK and clearance checks can run in plain Node. And it documents the conventions (zero pose, signs, axis order) in a dozen readable lines, which is what everyone else on a team builds on.

Q3 **How would FK change if you added a wrist roll joint?**

The planar shortcut breaks as soon as a joint axis isn't parallel to the others, so in general you'd switch to the matrix chain: one rotation and translation per joint, multiplied in order (or DH parameters). In this specific case, a roll about the tool's own axis doesn't move the TCP, only its orientation, so the position formula survives but the orientation needs the full chain.

Q4 **A model looks inside-out after export. What do you check?**

Winding order (front faces must be counter-clockwise from outside), normals pointing outward, and negative scale on a node, which flips winding because the transform's determinant is negative. Setting the material to `DoubleSide` is a useful diagnostic, since the faces reappear, but it's not a fix: it doubles fill cost and lighting will still be wrong if the normals are.

Q5 **Why does glTF have buffers, buffer views and accessors instead of just arrays of numbers?**

Separation of storage from interpretation. Buffers are raw bytes, views are GPU-uploadable ranges (possibly interleaved with a stride), accessors describe types, counts and bounds. Many meshes can share one blob, loaders can upload views directly without parsing each number, and features like sparse accessors and quantization fit in without changing the container.

Q6 **How do you make animation frame-rate independent? What's wrong with `x = lerp(x, target, 0.1)` every frame?**

Scale everything by dt. The lerp-per-frame version converges in a number of frames, not a number of seconds, so it's twice as fast at 120 fps. Using a factor of `1 − e^(−λ·dt)` makes the decay exponential in real time and identical at any frame rate. Also clamp dt, so a backgrounded tab doesn't produce one enormous step.

Q7 **How did you test 3D math?**

In layers. Known poses pin down conventions (zero pose straight up, +90° on the shoulder along −X). An independent implementation, the full matrix chain, is compared on 1,000 seeded random poses. Invariants that must always hold (never beyond full reach) are checked on many poses. And a scenario test samples ten minutes of the demo to prove joint limits and floor clearance.

Q8 **What would you change to show fifty arms in a factory layout?**

Share geometry and materials between arms (today each housing has its own material so it can glow; selection could instead be an outline pass or a single swapped material). Use instancing for repeated parts, turn off matrix auto-updates for static scenery, keep the shadow camera tight or bake shadows, and add level of detail for distant arms.

Q9 **How would you show objects from a database in a Three.js scene, and keep them current?**

Treat the API's list as the state and reconcile the scene to it: add meshes for new ids, update changed ones, remove and dispose missing ones, rebuilding geometry only when the shape's size or kind changed. Poll or subscribe for changes, and skip refreshes during interactions like placing. The ShapeLayer does exactly this, keyed by shape id.

Q10 **Your belts are turned at arbitrary angles. How do you tell whether a shape is on one?**

Transform the shape's position into the belt's frame: subtract the belt's center and rotate by minus its angle. Then it's a rectangle check, $|u| \le \text{length}/2$ and $|v| \le \text{width}/2$. The rotation uses the same convention as Three.js's rotation.y, and a test pins it: a belt turned 90° carries things toward −z.

Q11 **Why not use a physics engine for the conveyors?**

The question the feature answers is where items flow, and a kinematic model answers it exactly, with deterministic tests: ride at belt speed, fall as a projectile off the end, land on whatever is below. A physics engine adds friction, contact and tipping, which matter for jams and transfers, at the cost of tuning and non-exact results. I'd add one when those effects are the question, as Snack 3 does.

Q12 **The belts move shapes every frame. Why aren't positions saved continuously?**

A run is a preview. Writing sixty positions a second per shape would flood the API and the database with intermediate states nobody needs, and another browser polling would see shapes jitter. Saving once on Stop writes only the result, and only for shapes that moved.

Q13 **How does a splitter decide, and how would you let operators change that without a deploy?**

Each splitter has an ordered list of rules stored in the database: conditions on the item or its inventory part, and an exit. When an item crosses the gate, the first rule whose conditions all hold picks left, right or straight; otherwise a default. Because rules are data with a fixed set of fields and operators, they can be edited from the UI, the API or SQL, validated on the way in, and previewed against the items in the scene before saving.

Q14 **What do “left” and “right” mean on a belt that can be turned and reversed?**

Relative to the direction of travel, as for a person standing on the line. In the belt's frame that's +v for right when it runs forward and −v when it runs backwards; one small function holds that rule, and the gate model is built in a frame that's turned with the belt's direction, so the drawn flaps and the motion always agree.

Q15 **Two belts merge. How do you stop items colliding, and how do you decide who goes first?**

Feeder items wait at the end of their belt until the merge point is clear: nothing within a set gap of it on the main belt, and nothing else being merged. Priority decides contention: the main belt never stops, the feeders go first and main-belt items stop a gap short, or they take turns with one bit of state saying who went last. Waiting feeder items queue behind each other, spaced by their sizes.

Q16 **How do you join two 3D objects at named points, so they stay joined when the base moves?**

Define points per kind of object as fractions of its size. To join, place the attached object so its point lands on the base's point (its point turned by the relative heading, subtracted from the base's point), and store that offset in the base's frame with the base's id. Drawing is then parent pose plus turned offset, recursively, exactly like a scene graph.

Q17 **How do you make sure a part isn't counted twice as it moves between bins, belts and assemblies?**

Each shape has a stockTaken flag. Loading a bin, catching an item and joining two items all take stock only for shapes whose flag is false, and set it. Unloading gives a part back and deletes the shape. In Postgres each of those is one function that locks before it checks, so two of them racing can't both take the same part.

Q18 **A user wants to load “one of every plate” into a bin. How does the UI do that without a special feature?**

By reusing the rule engine. Every kind of object is turned into a stand-in shape with its part attached, the user's conditions are evaluated with the same matches() the splitters use, and the matching kinds become the lines of one all-or-nothing load, previewed first with availability.

CHAPTER 1

Exercises

1.1 Add a wrist roll

Add a fifth joint that rotates about the wrist link's own axis (local Y), between `Joint4` and `toolMount`. Update `JOINTS`, `RobotArm.build`, `forwardKinematics` and the matrix-chain test.

Hint. The TCP lies on the roll axis, so its position shouldn't change at all. Write that as a test first.

1.2 Test the limits

Write a test that `setAngle(1, 500)` returns 100 and that the arm's TCP matches FK of the clamped pose.

Hint. Three.js's core runs in Node without a renderer: you can construct a `RobotArm` and call `tcp.getWorldPosition` in a test.

1.3 Map the workspace

Print an ASCII map of the vertical plane (say 4 cm cells) marking every cell the TCP can reach within the joint limits.

Hint. Don't search target points. Sample joint space densely, run FK, and mark the cell each result lands in.

1.4 Extend the gripper model

Add a small camera bracket to the side of the palm, with a third material, then check the file with the Khronos glTF Validator.

Hint. One new material, one `addMesh`, one new node. Gripper.ts shouldn't need to change, which is the point of finding fingers by name.

1.5 A ring-buffer trail

Replace `copyWithin` with a true ring buffer and measure 10,000 `push` calls before and after with `performance.now()`.

Hint. A ring buffer's points are out of order in memory. Drawing it needs two ranges, for example two `Line` objects sharing the buffer.

1.6 Snap to the grid

Add a toggle that snaps placed and moved shapes to the 10 cm floor grid, and rotation to 15° steps.

Hint. Round in floorPoint, before showing the ghost, so the preview shows exactly where the shape will land.

1.7 Collisions on the belt

Make items queue up behind each other instead of passing through when the one in front stops at a rail or the end.

Hint. Sort the bodies on each belt by u, and limit each one's advance so it stays at least its own length behind the next. That keeps the model kinematic: no forces, just a rule about spacing.

1.8 Count what each exit receives

Show a running count per exit on each splitter (left 3, straight 5, right 2), reset when the line starts.

Hint. The route callback already sees every decision. Keep a Map from splitter id to counts, increment it there, and render it in the list. Bonus: store the counts on Stop, as a run summary in the database.

1.9 A buffer for the machine

Let a machine hold a queue of up to three base items on a short side spur, so a second base doesn't have to pass by.

Hint. Model the spur as a feeder belt with a merger into the machine's belt just upstream of it, and make the machine release the spur's stop line only when its base slot is empty.

APPENDIX A

Running the projects

Every project needs Node.js 20 or newer, and Snack 1 needs 22. From `Portfolio/vention/`:

Project	Commands
`snack1-robot-arm`	`npm ci` · `npm run dev` (http://localhost:5173) · `npm test` · `npm run check` (type-check + tests) · `npm run build:gripper` regenerates the .glb · needs Node 22 (Appendix C)
`snack2-control-panel`	`npm install` · `npm run dev` · `npm test` (IK and program tests) · `npm run build`
`snack3-conveyor-physics`	`npm install` · `npm run dev` · `npm test` · `npm run bench` (headless physics benchmark)
`snack4-refactor-kata`	`npm install` · `npm test` (spec + golden master) · `npm run typecheck` · `npm run demo`
`snack5-inventory`	`docker compose --profile postgres up --build` (or `--profile mongo`), then http://localhost:8081 · `docker compose --profile test run --rm --build api-test` runs the contract tests on both databases · without Docker: `cd api; npm install; npm test`

Snack 4's full commit history ships as `history/refactor-steps.bundle`: `git clone history/refactor-steps.bundle steps`, then `git log --stat` in the new folder.

APPENDIX B

Glossary

AABB
Axis-aligned bounding box: a box whose faces line up with the world axes, described by its center and half sizes or its min and max corners.

Accessor (glTF)
Says how to read a range of a bufferView as numbers: component type, element type (VEC3, SCALAR…), count, bounds.

ACID
Atomicity, consistency, isolation, durability: the guarantees a database transaction gives.

Bin packing
Packing items into the fewest fixed-size bins; cut lists into stock bars are an instance.

Broad phase
The cheap first pass of collision detection that finds pairs which might touch.

Buffer view (glTF)
A byte range within a buffer, usable directly as a GPU buffer.

Characterization test
A test that records what code currently does, to detect changes, rather than what it should do.

CHECK constraint
A rule on a table that the database enforces on every write, rejecting rows that break it.

Conditional update
An update whose filter contains the rule (“only if enough is available”), so check and write are one atomic step.

Connection point
A named spot on a kind of object, stored as fractions of its size, where it can be joined to another.

Contract test
One test suite run against every implementation of an interface, so they behave the same.

CORS
Cross-origin resource sharing: headers by which a server lets pages from other origins read its responses in a browser.

Counting sort
A non-comparison sort that buckets items by a small integer key in O(n).

CTE
Common table expression (WITH … AS): names a sub-query; in Postgres it can also chain data-changing statements.

Deadlock
Two transactions each waiting for a lock the other holds; avoided by locking in a consistent order.

Deadlock
A state where each party waits for something only another waiting party can provide, so none can proceed.

Denavit–Hartenberg
A standard way to describe each joint of a robot arm with four parameters, giving one transform matrix per joint.

Diverter
A device on a conveyor that steers items off the line to one side, here a hinged flap.

Document database
A database storing self-contained JSON-like documents, which can embed arrays and sub-objects (MongoDB).

Draw call
One request from the CPU to the GPU to draw geometry with a given state. Each has fixed overhead.

Emissive
Light a material gives off regardless of the scene's lights; used to make the selected housing glow.

Event delegation
One listener on a container handling events from its children, identified by attributes, so re-rendered children need no new listeners.

First-fit decreasing
A bin-packing heuristic: sort items largest first, put each in the first bin with room.

Fixed time step
Advancing a simulation in equal small steps, independent of the display's frame rate, so results don't depend on it.

Foreign key
A column that must match a key in another table, so references are always valid.

Forward kinematics
Computing the tool's position from joint angles. Always one answer.

Frustum culling
Skipping objects whose bounding sphere is outside the camera's view.

Golden master
Recorded outputs of existing code for many inputs, compared against new code.

Idempotency key
A client-chosen key that makes retrying a request safe: the server returns the stored result instead of acting twice.

Impulse
An instant change in momentum, the solver's unit of correction.

Instancing
Drawing one geometry many times in one draw call, with per-instance data such as a matrix and color.

Interpolation (α)
Drawing a moment between the last two physics states, by the fraction α of a step left over.

Inverse kinematics
Computing joint angles that put the tool at a target. Zero, several or infinitely many answers.

Island
A group of bodies connected by contacts; they sleep and wake together.

Isolation level
How much concurrent transactions can see of each other (READ COMMITTED, REPEATABLE READ, SERIALIZABLE).

Jacobian
The matrix of partial derivatives of tool position with respect to joint angles; maps joint speeds to tool speed.

jsonb
PostgreSQL's binary JSON type: stores a document in a column, with operators and indexes to query inside it.

Kanban
Pull-based production: a step makes or sends an item only when the next step signals it needs one.

Kerf
The width of material a saw blade removes with each cut.

Kinematic body
A body moved by code, with a velocity, that pushes others but is never pushed itself.

Kinematic simulation
Motion set directly (positions and velocities) rather than computed from forces.

Kustomize overlay
A directory that builds on a base set of Kubernetes manifests, adding or changing resources.

Little-endian
Byte order with the least significant byte first; required by the glTF container.

Local frame
A coordinate system attached to an object, in which its own geometry is simple; points are transformed into it.

Lost update
A race where two transactions read the same value and one's write silently overwrites the other's.

moveJ / moveL
Robot moves that interpolate joint angles (curved tool path) or the tool position (straight line).

Narrow phase
The exact collision test on candidate pairs, producing contact normal and depth.

Normal
A unit vector perpendicular to a surface; drives lighting and, in physics, the direction of contact forces.

Normalization
Storing each fact once in its own table, related by keys, instead of repeating it.

Object pool
Preallocated objects or slots reused instead of created and destroyed.

ON DELETE CASCADE
A foreign-key option that deletes the referencing rows when the referenced row is deleted.

Origin
The scheme, host and port of a page (http://localhost:5173); browsers isolate pages by origin.

PBR
Physically based rendering: materials described by base color, metalness and roughness, lit consistently in any environment.

Preflight
The OPTIONS request a browser sends before a cross-origin request that isn't simple, asking whether it's allowed.

Pure function
A function whose output depends only on its inputs and which has no side effects.

Quaternion
Four numbers representing a 3D rotation, free of gimbal lock and easy to interpolate.

Raycasting
Intersecting a ray (here from the camera through the mouse) with scene geometry, for picking.

Re-parenting
Moving an object to a new parent in the scene graph; `attach` keeps its world pose, `add` keeps its local one.

Readiness probe
A Kubernetes check that decides whether a pod receives traffic; separate from liveness, which decides restarts.

Reconciliation
React's comparison of a new render's output with the previous one, to find the DOM changes to make.

Refactoring
Changing the structure of code without changing its behaviour, in small steps.

Replica set
A group of MongoDB servers holding the same data; required for multi-document transactions.

Repository pattern
Hiding storage behind an interface of domain operations, so the rest of the code doesn't know the database.

Ring buffer
A fixed array used as a queue by wrapping an index, so nothing is shifted or allocated.

Row lock
A lock on specific rows (SELECT … FOR UPDATE) that makes other writers wait until the transaction ends.

Rule engine
A small interpreter that evaluates conditions stored as data, instead of decisions written as code.

Saga
A multi-step operation made of individually atomic steps, with compensating steps to undo earlier ones on failure.

Scene graph
A tree of objects whose transforms compose from parent to child.

Seam
A place where code's behaviour can be observed or substituted without editing it, which makes testing possible.

Semi-implicit Euler
Integration that updates velocity first, then position from the new velocity; stable and cheap.

Sequential impulses
Solving contact constraints one at a time and repeating, a form of projected Gauss–Seidel.

Shock propagation
Treating the lower body of a vertical contact as immovable on a final, bottom-up pass, so stacks settle.

Singularity
A pose where the arm loses the ability to move the tool in some direction; the Jacobian loses rank.

Slop
Overlap allowed between resting bodies before position correction acts, to prevent jitter.

Smoothstep
The easing curve $3t^2 - 2t^3$: from 0 to 1 with zero slope at both ends.

Spatial hash
A uniform grid of cells stored in a hash table, for finding nearby objects in O(n).

Spiral of death
A slow frame causing more physics steps, making the next frame slower; avoided by capping steps.

Stand-in object
A temporary value built only so existing logic (here, rule matching) can be applied to something that doesn't exist yet.

StatefulSet
A Kubernetes workload with stable pod names and a persistent volume per pod, used for databases.

Structure of arrays
Storing each property of many entities in its own array, instead of one object per entity.

TCP
Tool center point: the point on the tool that motion is planned for; here, between the fingertips.

Tearing (React)
Different components showing different versions of external data in the same commit.

Tessellation
Converting exact surfaces (from CAD) into triangles for rendering.

Upsert
Insert a row, or update it if one with the same key exists, in one statement (INSERT ... ON CONFLICT DO UPDATE).

Winding order
The order a triangle's corners are listed; counter-clockwise as seen marks the front face.

APPENDIX C · `vention/snack1-robot-arm`

Keeping Snack 1 working

The arm is built; this appendix is about keeping it that way. Which libraries it runs on and how they're pinned, what a version number promises and what a breaking change is, and the tests and CI pipeline that check every change before anyone relies on it.

npm · package-lock.json · Semantic Versioning · node:test · GitHub Actions · Dependabot

The one idea to take away

Every change is checked by a machine before a person relies on it. What gets installed is written down exactly:

`package.json (ranges)` → `npm install` → `package-lock.json (exact)` → `npm ci`

and every change, yours or a library's, goes through the same gate:

`a commit or a Dependabot PR` → `type-check + tests + build` → `green` → `merge`

Version numbers say what *should* happen when you upgrade, the lock file controls what *does* get installed, and the tests check what actually happened.

CONCEPTS · C.1

Dependency management

A **dependency** is code you didn't write but ship anyway. Snack 1 declares five of them in `package.json`: `three`, the only one that reaches the browser, and four development tools (`vite`, `typescript`, `tsx` and two sets of type definitions). Those five pull in others, the **transitive dependencies**: the lock file lists 102 packages in all, most of them platform-specific builds of esbuild and Rollup of which a machine installs only its own.

Three files and two commands do the work:

- **The manifest**, `package.json`, lists what the project needs and which versions it accepts, as *ranges*: `"vite": "^6.3.0"` means any 6.x from 6.3.0 up.
- **The lock file**, `package-lock.json`, records the exact version that was installed for every package in the tree, direct or transitive, with a checksum. Commit it: it is what makes your machine, a teammate's and CI install the same bytes.

- `npm install` resolves the ranges, picks versions, and rewrites the lock file. Use it when you mean to change dependencies.
- `npm ci` installs exactly what the lock file says, deletes `node_modules` first, and fails if the lock and the manifest disagree. Use it everywhere else, and always in CI.

Resolution is a puzzle: find one version of each package that satisfies every range in the tree. When two libraries need incompatible versions of a third, npm can nest a second copy (Python's pip and Java's Maven can't, and stop with a conflict or quietly pick one). `npm ls three` shows who needs what; `npm outdated` shows how far behind each package is.

> The arm has almost no runtime surface: one library, `three`, which itself has no dependencies. Everything else is a build or test tool that never reaches a user. That's why the CI audit checks `--omit=dev`: a vulnerability in a test runner is worth fixing, but it isn't shipped.

CONCEPTS · C.2

Semantic versioning and breaking changes

Ranges only work if version numbers mean something. **Semantic Versioning** (SemVer) gives them a meaning: a version is `MAJOR.MINOR.PATCH`, and which number goes up tells users what kind of change they're getting.

Bump	When	In this project, for example
`PATCH` 1.0.0 → 1.0.1	A fix that changes no behaviour anyone relies on	The unused `belts` variable removed from `stations.ts`
`MINOR` 1.0.0 → 1.1.0	New features, old uses still work	Adding an optional `margin = 0` parameter to `maxReach`
`MAJOR` 1.0.0 → 2.0.0	A **breaking change**	Requiring Node 22 instead of 20; renaming `maxReach`

A **breaking change** is anything that can make correct code or a working setup that depends on yours stop working: removing or renaming an export, changing what a function returns or throws, making an optional argument required, or dropping a runtime version. The test is the user's experience, not the size of the diff: a one-line change to `engines` is major, a thousand-line new feature is minor.

Range operators lean on that promise. `^6.3.0` takes minors and patches, `~5.9.0` takes patches only, and an exact `6.3.0` takes nothing.

> **Below 1.0.0 the rules shift one place.** SemVer treats 0.x as unstable, so a minor bump may break. npm follows suit: `^0.170.0` accepts `0.170.x` and nothing else. three.js has stayed at 0.x for its whole life and does rename and remove APIs between minors, which is why its upgrades come as their own pull request (C.4).

SemVer is a promise made by people, not a guarantee. Maintainers misjudge what counts as breaking, and lock files plus tests are how you find out before your users do.

CONCEPTS · C.3

Automated testing and continuous integration

An **automated test** is code that runs your code and checks the result. Snack 1's tests use Node's built-in runner, `node:test`, with `tsx` to load TypeScript. They're almost all **unit tests** of pure logic: kinematics, reach, belts, splitter rules and stations, none of which import Three.js, so they run in plain Node in about half a second.

Tests sit in layers, often drawn as a pyramid: many fast unit tests at the bottom, fewer **integration tests** that wire real parts together (Snack 5's contract tests against real databases), and a handful of slow **end-to-end** tests that drive the whole app. Snack 1 adds a fourth kind that doesn't test behaviour at all: **static checks**. The TypeScript compiler, run with `tsc --noEmit`, rejects whole classes of mistakes before any test runs.

Continuous integration (CI) runs all of it on a clean machine for every push and pull request. "Clean" matters: nothing installed by hand, nothing left over from yesterday, the exact dependencies from the lock file. A red result on a pull request is the cheapest moment to find a problem: before it's merged, while the change is small and fresh.

`push or pull request` → `fresh Ubuntu VM` → `npm ci` → `typecheck` →
`tests + coverage` → `vite build`

Coverage measures which lines and branches the tests executed. It shows what is *untested*, not what is correct: a line can run under a test that asserts nothing about it. Snack 1's pure modules sit near 99% of lines and 89% of branches.

APPENDIX C

Line by line

Six parts: the files that turn the three ideas into checks that run by themselves.

PART C.1 · `vention/snack1-robot-arm/package.json`

The manifest

Two changes: an `engines` field, and scripts that give each check its own name.

package.json · lines 1–18

```
{
  "name": "robot-arm",
  "private": true,
  "version": "1.0.0",
  "type": "module",
  "engines": {
    "node": ">=22"
  },
  "scripts": {
    "dev": "vite",
    "typecheck": "tsc --noEmit",
    "build": "npm run typecheck && vite build",
    "preview": "vite preview",
    "test": "tsx --test \"tests/**/*.test.ts\"",
    "test:coverage": "tsx --test --experimental-test-coverage --test-coverage-
include=\"src/**\" \"tests/**/*.test.ts\"",
    "check": "npm run typecheck && npm test",
    "build:gripper": "tsx scripts/build-gripper.ts"
  },
```

`engines` states the oldest Node this project supports. npm warns when you install on something older (set `engine-strict=true` in an `.npmrc` to make it an error). Raising it from 20 to 22 is this project's first **breaking change**: Node 20 left support in April 2026, and the quoted glob in `test` is expanded by Node itself, which only Node 21 and later do. On Node 20 the test command would fail to find the test files.

`typecheck` is now its own script and `build` calls it, so a type error stops a build in exactly one place. `check` is the pair to run before pushing. `test:coverage` adds Node's built-in coverage,

limited to `src/` so the tests don't count themselves.

package.json · lines 19–28

```
  "dependencies": {
    "three": "^0.170.0"
  },
  "devDependencies": {
    "@types/node": "^22.15.0",
    "@types/three": "^0.170.0",
    "tsx": "^4.19.0",
    "typescript": "^5.8.0",
    "vite": "^6.3.0"
  }
```

Every range is a caret. For the 1.x-and-up packages that means "any later minor or patch"; for `three` and `@types/three`, at 0.170, it means "0.170.x only".

The Node version pin

`.nvmrc` holds one line, `22`. Version managers (`nvm use`, `fnm`) read it to switch Node per folder, and the workflow's audit job reads it to pick its Node. `engines` says what's *allowed*; `.nvmrc` says what to *use*. A test keeps them from disagreeing.

PART C.2 · `tests/dependencies.test.ts`

Testing the dependencies

Most dependency mistakes surface far from their cause: `npm ci` fails in CI with a message about the lock file, or the types say a method exists that the library doesn't have. This file catches them locally, in milliseconds, by reading three files. No install is needed.

Reading the files

tests/dependencies.test.ts · lines 7–16

```
import { test } from "node:test";
import assert from "node:assert/strict";
import { readFileSync } from "node:fs";

const root = new URL("../", import.meta.url);
const readJson = (file: string) => JSON.parse(readFileSync(new URL(file, root), "utf8"));
```

```
const pkg = readJson("package.json");
const lock = readJson("package-lock.json");
const nvmrc = readFileSync(new URL(".nvmrc", root), "utf8").trim();
```

`import.meta.url` is this test file's own URL, so `root` is the project folder however the tests are started. Reading the lock file as plain JSON works because lockfile version 3 is a flat map: `packages["node_modules/three"]` is the installed three.js.

Parsing a version

tests/dependencies.test.ts · lines 18–30

```
type Version = [major: number, minor: number, patch: number];

const SEMVER = /^(\d+)\.(\d+)\.(\d+)(?:-[0-9A-Za-z.-]+)?(?:\+[0-9A-Za-z.-]+)?$/;

function parse(version: string): Version {
  const m = SEMVER.exec(version);
  assert.ok(m, `"${version}" is not a MAJOR.MINOR.PATCH version`);
  return [Number(m[1]), Number(m[2]), Number(m[3])];
}

function compare(a: Version, b: Version): number {
  return a[0] - b[0] || a[1] - b[1] || a[2] - b[2];
}
```

A named tuple type, `[major, minor, patch]`, documents itself and compares naturally. The pattern allows SemVer's optional pre-release (`-beta.1`) and build (`+sha`) suffixes but ignores them; this project doesn't use either. `compare` chains with `||`: the first non-zero difference wins.

Does a version satisfy a range?

tests/dependencies.test.ts · lines 32–51

```
// Enough of npm's range syntax for this project: ^x.y.z, ~x.y.z, >=x.y.z and exact versions.
// A short bound like ">=22" means ">=22.0.0".
function satisfies(version: string, range: string): boolean {
  const v = parse(version);
  const op = /^(\^|~|>=)?/.exec(range)![0];
  const bound = range.slice(op.length).split(".");
  while (bound.length < 3) bound.push("0");
  const base = parse(bound.join("."));
  if (compare(v, base) < 0) return false;
```

```
  if (op === ">=") return true;
  if (op === "~") return v[0] === base[0] && v[1] === base[1];
  if (op === "^") {
    // A caret allows changes that SemVer promises are compatible. Below 1.0.0 the minor
    // number is the breaking one (0.170 → 0.171 may break), and below 0.1.0 even the patch
is.
    if (base[0] > 0) return v[0] === base[0];
    if (base[1] > 0) return v[0] === 0 && v[1] === base[1];
    return v[0] === 0 && v[1] === 0 && v[2] === base[2];
  }
  return compare(v, base) === 0;
}
```

This is the heart of SemVer as npm reads it, written out. Anything below the lower bound fails first. A tilde fixes major and minor. A caret fixes the *leftmost non-zero* number: the major from 1.0.0 up, the minor for 0.x, and even the patch for 0.0.x. The short form `>=22` is padded to `>=22.0.0` so `engines` can be written the usual way.

> In a real tool you'd use the `semver` package, which handles every range form (`||`, hyphen ranges, `x`-ranges, pre-releases). Writing the subset this project uses keeps the test free of dependencies, and the first test pins its behaviour so a mistake in the helper can't hide a mistake in the project.

The checks

`tests/dependencies.test.ts · lines 53–69`

```
const declared: Record<string, string> = { ...pkg.dependencies, ...pkg.devDependencies };

test("the range helper follows npm's rules, including the 0.x special case", () => {
  assert.ok(satisfies("6.4.5", "^6.3.0"));
  assert.ok(!satisfies("7.0.0", "^6.3.0"), "a new major is a breaking change");
  assert.ok(satisfies("0.170.4", "^0.170.0"));
  assert.ok(!satisfies("0.171.0", "^0.170.0"), "below 1.0.0, a new minor is breaking");
  assert.ok(satisfies("5.9.1", "~5.9.0"));
  assert.ok(!satisfies("5.10.0", "~5.9.0"));
  assert.ok(satisfies("24.1.0", ">=22"));
  assert.ok(!satisfies("20.19.0", ">=22"));
});

test("the project version is a valid MAJOR.MINOR.PATCH", () => {
  parse(pkg.version);
```

```
  assert.equal(lock.version, pkg.version, "run npm install so the lock file records the new
version");
});
```

Testing the helper first, with the cases that matter most (a new major, the 0.x rule), means the later failures can be trusted. The version test compares `package.json` with the lock file's copy, which only `npm install` or `npm version` updates: edit the version by hand and this fails.

tests/dependencies.test.ts · lines 71–83

```
test("package-lock.json was regenerated after the last package.json edit", () => {
  const lockRoot = lock.packages[""];
  assert.deepEqual(lockRoot.dependencies ?? {}, pkg.dependencies ?? {}, "run npm install");
  assert.deepEqual(lockRoot.devDependencies ?? {}, pkg.devDependencies ?? {}, "run npm
install");
});

test("every locked direct dependency is inside the range package.json allows", () => {
  for (const [name, range] of Object.entries(declared)) {
    const locked = lock.packages[`node_modules/${name}`]?.version;
    assert.ok(locked, `${name} is in package.json but not in package-lock.json`);
    assert.ok(satisfies(locked, range), `${name}@${locked} is outside "${range}"`);
  }
});
```

The first test is the one `npm ci` would fail on, with a friendlier message. The second catches a lock file that was edited, merged badly, or produced by a different range: every direct dependency's locked version must sit inside its declared range.

tests/dependencies.test.ts · lines 85–91

```
test("three and @types/three are on the same release", () => {
  // @types/three is versioned to match three. A mismatch type-checks against an API the
  // running library doesn't have, so Dependabot updates these two together (see .github).
  const [, threeMinor] = parse(lock.packages["node_modules/three"].version);
  const [, typesMinor] = parse(lock.packages["node_modules/@types/three"].version);
  assert.equal(typesMinor, threeMinor);
});
```

`@types/three` is published separately from three.js, by the DefinitelyTyped project, and versioned to match it. If they drift, the compiler checks your code against an API the running library doesn't have, and the error appears in the browser instead of the build.

tests/dependencies.test.ts · lines 93–107

```
test(".nvmrc pins a Node version that package.json's engines field accepts", () => {
  const range: string = pkg.engines?.node;
  assert.ok(range, 'package.json needs "engines": { "node": ... }');
  const pinned = /^v?(\d+)$/.exec(nvmrc)?.[1];
  assert.ok(pinned, `.nvmrc should hold a major version like "22", not "${nvmrc}"`);
  assert.ok(satisfies(`${pinned}.0.0`, range), `.nvmrc says ${pinned}, engines says
${range}`);
});

test("the Node version running these tests meets the engines field", () => {
  const running = process.versions.node;
  assert.ok(
    satisfies(running, pkg.engines.node),
    `Node ${running} is older than package.json allows (${pkg.engines.node})`,
  );
});
```

The last two keep the Node story consistent: the pin agrees with the requirement, and whoever runs the tests meets it. On CI's Node 22 and 24 jobs both pass; on a laptop still on Node 20 the second one says so plainly.

PART C.3 · `.github/workflows/snack1-robot-arm.yml`

The CI workflow

GitHub Actions reads workflows only from `.github/workflows/` at the repository root, so this file lives in `Portfolio/`, two levels above the project it checks.

.github/workflows/snack1-robot-arm.yml · lines 4–21

```
name: snack1 robot arm

on:
  push:
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
```

```
# A newer push to the same branch cancels the older run.
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

defaults:
  run:
    working-directory: vention/snack1-robot-arm
```

The triggers have no filters: every push to any branch and every pull request runs the checks, whatever it changes. That costs about twenty seconds on a commit that only touches Snack 5, and it matters for branch protection: `main` requires `test (Node 22)` and `test (Node 24)` to pass, and a check that never runs never passes, so a path-filtered workflow would leave unrelated pull requests waiting forever. `workflow_dispatch` adds a "Run workflow" button for a manual run. `permissions: contents: read` gives the job's token the least it needs. `concurrency` cancels a run that a newer push to the same branch has made pointless, and `defaults.run.working-directory` saves a `cd` on every step.

`.github/workflows/snack1-robot-arm.yml · lines 23–50`

```
jobs:
  test:
    name: test (Node ${{ matrix.node }})
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        # The lowest version package.json's "engines" allows, and the current LTS.
        node: [22, 24]
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-node@v5
        with:
          node-version: ${{ matrix.node }}
          cache: npm
          cache-dependency-path: vention/snack1-robot-arm/package-lock.json

      # npm ci installs exactly what package-lock.json records, and fails if the lock file
      # and package.json disagree, instead of quietly resolving new versions.
      - run: npm ci
```

```
      - run: npm run typecheck

      - run: npm run test:coverage

      - name: Production build
        run: npx vite build
```

A **matrix** runs the same job once per value: here Node 22, the oldest `engines` allows, and Node 24, the current long-term-support release. Testing the bounds of what you claim to support is the point; the versions in between are very unlikely to differ. `fail-fast: false` lets both finish, so you see whether a failure is version-specific.

`setup-node`'s `cache: npm` keeps npm's download cache between runs, keyed on the lock file's hash: unchanged dependencies download once. Then the steps mirror `npm run check` plus a build. The build step calls `vite build` directly because `npm run build` would type-check a second time.

`.github/workflows/snack1-robot-arm.yml · lines 52–61`

```
  audit:
    name: audit runtime dependencies
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-node@v5
        with:
          node-version-file: vention/snack1-robot-arm/.nvmrc
      # Only what ships to the browser (three); dev tools never reach users.
      - run: npm audit --omit=dev --audit-level=high
```

The audit job needs no install: `npm audit` sends the lock file's package list to the registry's advisory database. `--omit=dev` limits it to what ships, and `--audit-level=high` fails only on high or critical advisories, so a low-severity notice doesn't turn the build red.

PART C.4 · `.github/dependabot.yml`

Updates as pull requests

Left alone, dependencies drift: a few months later the upgrade is a dozen majors at once and every failure has twelve suspects. Dependabot opens small, regular pull requests instead, and the workflow tests each one.

.github/dependabot.yml · lines 1–27

```yaml
# Dependabot opens pull requests for dependency updates; the snack1 workflow tests each one.
version: 2
updates:
  - package-ecosystem: npm
    directory: /vention/snack1-robot-arm
    schedule:
      interval: weekly
      day: monday
    open-pull-requests-limit: 5
    commit-message:
      prefix: "snack1"
    groups:
      # @types/three is versioned to match three, so they always move together.
      # Below 1.0.0 a minor bump (0.170 → 0.171) can break, which is why CI must pass first.
      three:
        patterns: ["three", "@types/three"]
      # Minor and patch updates to the build and test tools come as one pull request.
      # Major updates (e.g. Vite 6 → 7) still get a pull request each.
      tooling:
        dependency-type: development
        update-types: [minor, patch]

  - package-ecosystem: github-actions
    directory: /
    schedule:
      interval: weekly
      day: monday
```

`directory` points at the project folder, because that's where its `package.json` is. **Groups** decide how updates are bundled, and the first group that matches wins:

- **`three`** holds three.js and its types together, so they can never be merged apart. Dependabot calls 0.170 → 0.171 a "minor" update, but under 0.x it may break, so this pull request is the one to read carefully when CI goes red.
- **`tooling`** bundles every minor and patch update to the dev tools into one pull request a week. A major (Vite 6 → 7) still gets its own, with its changelog, because it may need code changes.

The second entry keeps the workflow's own actions (`checkout`, `setup-node`) current: actions are dependencies too, pinned here by major version.

PART C.5 · `CHANGELOG.md`

The changelog, and releasing

A version number says how big a change is; the changelog says what it was. The file follows the Keep a Changelog layout: an **Unreleased** section collects changes as they're merged, grouped as Added, Changed, Fixed or Removed, and each release moves them under a dated version heading.

CHANGELOG.md · lines 8–15

```
## [Unreleased]

### Changed

- **Breaking:** requires Node.js 22 or newer (was 20). Node 20 reached end of life in April
2026,
  and `npm test` passes a quoted glob (`"tests/**/*.test.ts"`) that only Node 21+ expands
itself.
  Because this can stop the project running for someone on Node 20, the next release is 2.0.0.
- `npm run build` type-checks through the new `typecheck` script.
```

A breaking change is marked as such, says what to do about it, and names the version it forces. To release:

```
npm version major --no-git-tag-version   # 1.0.0 → 2.0.0 in package.json and package-lock.json
# move the Unreleased notes under "## [2.0.0] - <date>", then
git commit -am "snack1: release 2.0.0"
git tag snack1-v2.0.0
git push --follow-tags
```

`--no-git-tag-version` is needed because npm only commits and tags when the package is at the repository root, and this one is two folders down. The tag is prefixed with the project's name because five projects share one repository and one tag namespace.

PART C.6 · `src/line/stations.ts`

The first catch

The new type-check failed the first time it ran, on code that had been in the repository since the bins commit:

```
src/line/stations.ts:296:54 - error TS6133: 'belts' is declared but its value is never read.

296     const { bins = [], shapes, parts = [], machines, belts } = world();
```

`binStep` destructured `belts` from the world but reached belts through `beltById` instead. `tsconfig.json` has `noUnusedLocals` on, so `npm run build` had been failing since that commit. `npm run dev` doesn't type-check (Vite strips types without checking them, for speed), so the app kept running and nobody noticed. The fix was deleting one word.

It was harmless, but it's the whole argument in miniature: a check that exists but doesn't run automatically is a check that doesn't happen. Now it runs on every push.

APPENDIX C

Interview questions

Q1 **What's the difference between `npm install` and `npm ci`, and which one runs in CI?**

`npm install` resolves ranges and may update the lock file; `npm ci` installs exactly what the lock file records, from a clean `node_modules`, and fails if the lock and `package.json` disagree. CI uses `npm ci`, because a build should test the dependencies you committed, not whatever the registry has today.

Q2 **Why commit a lock file if `package.json` already lists the dependencies?**

`package.json` lists ranges and only direct dependencies. The lock file pins the exact version and checksum of every package in the tree, transitive ones included. Without it, two installs a week apart can differ, which is where "works on my machine" comes from.

Q3 **Is changing `engines` from 20 to 22 a breaking change? It's one line.**

Yes. Anyone running the project on Node 20 can no longer do so, and that's what a breaking change is: it's judged by its effect on users, not the size of the diff. Under SemVer it takes a major bump, here 1.0.0 → 2.0.0.

Q4 **Why does `^0.170.0` not accept 0.171.0?**

SemVer treats 0.x as unstable: below 1.0.0, the minor number signals breaking changes. npm's caret fixes the leftmost non-zero number, so for 0.170.0 that's the minor, and only 0.170.x patches are accepted. three.js has always been 0.x and does break APIs between minors.

Q5 **The tests passed but the build failed. How?**

The tests run through `tsx`, which strips types without checking them, and they only import the pure modules. The type-check reads every file under `tsconfig.json`'s `include`, with all its strictness flags. That gap is exactly how the unused `belts` variable survived, and why CI runs the type-check as its own step.

Q6 **Your coverage is 99%. Is the code correct?**

Not necessarily. Coverage records which lines ran during the tests, not whether the tests checked them. It's good at showing what's untested (here, `joints.ts` lines 100–109) and poor at proving anything else. Assertions on behaviour, like the 1,000-pose FK comparison, are what give confidence.

Q7 **How would you handle a dependency update that breaks your build?**

Don't merge it; that's what the red pull request is for. Read the library's changelog for the breaking change, then either adapt the code on the Dependabot branch, or hold the range (`ignore` that version in `dependabot.yml`) until there's time. Avoid patching around it in `node_modules`: `npm ci` throws that away.

Q8 **Why test on Node 22 and 24, but not 23?**

22 is the lowest version `engines` promises and 24 is the current LTS; bugs cluster at the edges of what you support. Odd-numbered Node releases are short-lived and never reach long-term support, so few people run them.

APPENDIX C

Exercises

Each one breaks something on purpose so you can watch a check catch it. Work on a branch (`git switch -c try-safety-nets`), from `vention/snack1-robot-arm`; after each, `git restore .` undoes the edits and `npm ci` restores `node_modules`.

C.1 A stale lock file

Change `"three": "^0.170.0"` to `"^0.171.0"` in `package.json` without reinstalling, then run `npm test` and `npm ci`.

Expect. Two dependency tests fail ("run npm install" and `three@0.170.0 is outside "^0.171.0"`), and `npm ci` refuses to install.

C.2 Types that don't match the library

Run `npm install @types/three@0.171`, upgrading only the types.

Expect. "three and @types/three are on the same release" fails. Then upgrade both together and run `npm run check`; read `git diff package-lock.json` to see everything that moved.

C.3 Feel a breaking change

Rename `maxReach` to `maxRadius` in `src/arm/kinematics.ts` and run `npm run check`. Then restore it and instead add an optional parameter, `margin = 0`, subtracted from the result.

Expect. The rename breaks `reach.ts` and two test files: a major change. The optional parameter breaks nothing: a minor one.

C.4 A version edited by hand

Run `npm version minor --no-git-tag-version`, then change only `package.json`'s version to 1.2.0.

Expect. The first passes (both files say 1.1.0); the hand edit fails "the project version is a valid MAJOR.MINOR.PATCH" because the lock still says 1.1.0.

C.5 Watch CI go red, then green

In `forwardKinematics`, change `-r * Math.sin(base)` to `r * Math.sin(base)`. Commit, push the branch, and open a pull request.

Expect. The kinematics tests fail locally and both test jobs fail on the pull request. Fix the sign, push again, and they turn green. Close the pull request without merging.

C.6 Make red unmergeable

In the repository's **Settings → Rules → Rulesets**, add a branch ruleset that targets the default branch, with **Require a pull request before merging** (0 approvals is enough when you work alone) and **Require status checks to pass** listing `test (Node 22)` and `test (Node 24)`. Add **Repository admin** to the bypass list if you still want to push straight to `main`.

Hint. The checks only appear in the search after they've run once. Repeat C.5: GitHub now disables the merge button until the checks pass. One approval would lock you out of your own pull requests, because GitHub doesn't let authors approve their own.

C.7 Cover the uncovered

Run `npm run test:coverage` and pick an uncovered range, such as `joints.ts` lines 100–109. Write a test that runs it *and* asserts what it does.

Hint. Read the function first and decide what should be true of its result; a test that only calls it raises coverage without checking anything (Q6).
